

数据分析之 SQL 基础知识

目录

- 一、EXCEL 基础技能 1
 - 核心函数（必须掌握） 1
 - 数据透视表 (PivotTables) 10
 - 数据可视化（基础图表） 15
 - （进阶选学）：条件格式 17
 - （进阶选学）：Power Query 19
- 二、SQL (数据查询与提取)..... 21
 - 1. 数据库基础概念详解： 21
 - 2. 核心查询语句 (DQL - Data Query Language) 详解： 22
 - 3. 数据聚合与分组详解： 25
 - 4. 多表连接 (Joins) 详解： 27
 - 5. 子查询 (Subqueries) 详解： 29
 - 6. (进阶选学) 简述： 30
 - 7. 实践环境： 31
- 三、统计学基础 (数据分析的理论支撑)..... 31
 - 1. 什么是数据分析与统计学？ 31
 - 2. 数据类型详解： 32
 - 3. 描述性统计 (Descriptive Statistics) 详解： 32
 - 4. 概率基础 (概念理解) 详解： 34
 - 5. 相关性与因果关系 详解： 35
 - 6. 推断性统计入门 (概念理解) 详解： 35
 - 7. (入门选学): 线性回归 (Linear Regression) 的基本概念： 36

四、SQL 应用举例	37
1. 查询表中的所有数据	39
2. 查询特定列并按条件筛选行	40
3. 对查询结果进行排序	41
4. 计算总行数	41
5. 计算数值列的总和、平均值、最大值、最小值	42
6. 按分组对数据进行聚合	43
7. 对分组后的结果进行过滤	43
8. 使用 INNER JOIN 连接两个表	44
9. 使用 LEFT JOIN 连接两个表	45
10. 使用 CASE 语句进行条件判断	46
11. 使用 AND 组合多个过滤条件	49
12. 使用 OR 组合多个过滤条件	50
13. 使用 BETWEEN 过滤日期范围	51
14. 使用 LIKE 进行模糊匹配	51
15. 查找唯一值	52
16. 计算衍生列	53
17. 过滤 NULL 值	53
18. 统计唯一值的数量	54
19. 使用窗口函数计算总量的百分比	54
20. 查找没有关联记录的行 (LEFT JOIN + IS NULL)	55
21. 使用 UNION 合并查询结果	56
22. 在 WHERE 子句中使用子查询 (非关联子查询)	57
23. 在 FROM 子句中使用子查询 (派生表/内联视图)	57
24. 查找前 N 个/最后的 N 个记录	58
25. 使用窗口函数 RANK() 进行排名	59
26. 使用窗口函数 ROW_NUMBER() 分配行号	60
27. 使用窗口函数 SUM() OVER() 计算累计总和 (Running Total)	61
28. 使用窗口函数 LAG() 获取前一行的值	62
29. 使用 COALESCE() 处理 NULL 值	63

30. 使用 NOT IN 排除特定值列表	64
31. 使用 NOT LIKE 进行反向模糊匹配	65
32. 提取日期中的年份和月份	65
33. 计算日期之间的差异	66
34. 条件聚合 (使用 CASE 在聚合函数内)	67
35. 查找具有重复值的行	68
36. 使用 CTE (Common Table Expression) 简化复杂查询	69
37. 使用 RIGHT JOIN 连接两个表	70
38. 使用 CROSS JOIN 连接两个表	71
39. 更新现有数据 (UPDATE)	71
40. 删除数据 (DELETE)	72
五、SQL 应用模型-完整代码版	72
1.应用模式: 聚合与汇总 (Aggregation & Summarization)	72
2.应用模式: 数据筛选与过滤 (Filtering Data)	77
3.数据关联与整合	81
4.数据转换与衍生 (Data Transformation & Derivation)	89
5.窗口函数应用 (Window Function Applications)	92
6.处理缺失数据 (Handling Missing Data)	96
7. 数据去重 (Deduplication)	98
8. 数据透视 (Pivoting Data)	102
9. 逆透视 (Unpivoting Data)	105
10. 分解复杂逻辑 (Breaking Down Complex Logic)	108
11. 查找序列中的“缺失”或“间隔” (Identifying Gaps)	110
12. 分组内 Top N 查询 (Top N per Group Query)	114
13. 识别连续序列或“岛” (Identifying Consecutive Sequences or "Islands")	116
14. 计算期环比/同比变化 (Calculating Period-over-Period Change)	118
15. 数据分箱/离散化 (Data Binning / Discretization)	121
16. 计算百分位数或分位数 (Calculating Percentiles or Quantiles)	122
17. 连接多个 (多于两个) 表 (Joining Multiple Tables)	124
18. 数据验证规则检查 (Data Validation Rules Check)	126

19. 数据抽样 (Sampling Data).....	129
20. 分组内的首/末事件查询 (Finding First/Last Event per Group).....	130
21. 计算移动平均/总和 (Calculating Moving Average/Sum).....	134
22. 填充数据中的间隔或缺失值 (Filling Gaps or Imputing Missing Data).....	135
23. 处理层级结构数据 (Handling Hierarchical Data).....	138
24. 比较相关行的数据 (Comparing Related Rows).....	141
25. 比较个体与群体规范 (Comparing Item to Group Norm).....	143

一、 EXCEL 基础技能

核心函数（必须掌握）

这些函数是 Excel 进行数据处理和分析的基础工具，掌握它们能极大提高你的效率。

1. 数学计算 (Mathematical Calculations)

这些函数用于执行基本的算术运算。

- SUM
 - **概念：** 计算指定单元格区域内所有数值的总和。
 - **语法：** SUM(number1, [number2], ...) 可以是单个数字、单元格引用（如 A1）或单元格区域（如 A1:A10）。
 - **示例：** 假设 A1 到 A3 单元格的值分别是 10, 20, 30。
 - 公式：=SUM(A1:A3)
 - 结果：60
- AVERAGE
 - **概念：** 计算指定单元格区域内所有数值的算术平均值。
 - **语法：** AVERAGE(number1, [number2], ...)
 - **示例：** 假设 A1 到 A3 单元格的值分别是 10, 20, 30。
 - 公式：=AVERAGE(A1:A3)
 - 结果：20 ((10+20+30)/3)
- COUNT
 - **概念：** 计算指定单元格区域内包含 **数值** 的单元格数量。它会忽略文本、空白单元格和错误值。
 - **语法：** COUNT(value1, [value2], ...)
 - **示例：** 假设 A1:A5 的值是 10, "Hello", 30, , 50 (空单元格)
 - 公式：=COUNT(A1:A5)
 - 结果：3 (只计算了 10, 30, 50)
- COUNTA
 - **概念：** 计算指定单元格区域内 **非空** 的单元格数量。它计算包含任何类型信息（数字、文本、逻辑值、错误值）的单元格，但不计算空白单元格。
 - **语法：** COUNTA(value1, [value2], ...)
 - **示例：** 假设 A1:A5 的值是 10, "Hello", 30, , 50
 - 公式：=COUNTA(A1:A5)
 - 结果：4 (计算了 10, "Hello", 30, 50)
- MAX
 - **概念：** 返回指定单元格区域内数值的最大值。
 - **语法：** MAX(number1, [number2], ...)

- **示例：** 假设 A1 到 A3 单元格的值分别是 10, 20, 5。
 - 公式：=MAX(A1:A3)
 - 结果：20
- **MIN**
 - **概念：** 返回指定单元格区域内数值的最小值。
 - **语法：** MIN(number1, [number2], ...)
 - **示例：** 假设 A1 到 A3 单元格的值分别是 10, 20, 5。
 - 公式：=MIN(A1:A3)
 - 结果：5

2. 逻辑判断 (Logical Functions)

这些函数用于根据条件执行逻辑测试。

- **IF**
 - **概念：** 进行逻辑判断。如果条件为 TRUE，返回一个值；如果条件为 FALSE，返回另一个值。
 - **语法：** IF(logical_test, value_if_true, value_if_false)
 - **示例：** 假设 A1 单元格的值是 75。判断是否及格 (≥ 60)。
 - 公式：=IF(A1 $\geq$ 60, "及格", "不及格")
 - 结果："及格"
 - 如果 A1 是 50，则结果为 "不及格"。
- **AND**
 - **概念：** 检查 **所有** 参数是否都为 TRUE。只有所有参数都为 TRUE 时才返回 TRUE，否则返回 FALSE。通常嵌套在 IF 函数中使用。
 - **语法：** AND(logical1, [logical2], ...)
 - **示例：** 假设 A1=75, B1=80。判断是否两门都及格 (都 ≥ 60)。
 - 公式：=AND(A1 $\geq$ 60, B1 $\geq$ 60)
 - 结果：TRUE
 - 如果 A1=50, B1=80，则结果为 FALSE。
 - 结合 IF：=IF(AND(A1 $\geq$ 60, B1 $\geq$ 60), "双优", "非双优") 结果为 "双优"。
- **OR**
 - **概念：** 检查 **任何一个** 参数是否为 TRUE。只要有一个参数为 TRUE 就返回 TRUE，只有所有参数都为 FALSE 时才返回 FALSE。
 - **语法：** OR(logical1, [logical2], ...)
 - **示例：** 假设 A1=50, B1=80。判断是否至少有一门及格 (≥ 60)。
 - 公式：=OR(A1 $\geq$ 60, B1 $\geq$ 60)
 - 结果：TRUE
 - 如果 A1=50, B1=55，则结果为 FALSE。
 - 结合 IF：=IF(OR(A1 $\geq$ 60, B1 $\geq$ 60), "至少一门及格", "全不及格") 结果为 "至少一门及格"。
- **IFS (新版本 Excel)**

- **概念：** 检查多个条件，并返回与第一个 TRUE 条件相对应的值。避免了写很多层嵌套的 IF 函数，使公式更简洁。
- **语法：** IFS(logical_test1, value_if_true1, [logical_test2, value_if_true2], ...)
- **示例：** 假设 A1 是分数，评定等级：>=90 为“优秀”，>=75 为“良好”，>=60 为“及格”，<60 为“不及格”。
 - 公式：=IFS(A1>=90, “优秀”, A1>=75, “良好”, A1>=60, “及格”, A1<60, “不及格”)
 - 如果 A1=85，结果为“良好”。
 - 如果 A1=55，结果为“不及格”。

3. 查找引用 (Lookup & Reference Functions)

这些函数用于在表格中查找和引用数据。

- **VLOOKUP (Vertical Lookup)**
 - **概念：** 在表格的 **第一列** 中垂直查找某个值，并返回同一行中指定列的值。
 - **语法：** VLOOKUP(lookup_value, table_array, col_index_num, [range_lookup])
 - lookup_value: 要查找的值。
 - table_array: 要查找的区域（包含查找列和返回列）。查找总是在这个区域的第一列进行。
 - col_index_num: 返回值所在的列号（在 table_array 中，第一列是 1，第二列是 2，以此类推）。
 - range_lookup: 可选。TRUE 或省略表示近似匹配（要求查找列升序排序），FALSE 表示精确匹配（常用）。
 - **示例：** 假设有如下表格在 D1:E3:

	D (学号)	E (姓名)
101	张三	李四
102	王五	

 在 A1 输入学号 102，想在 B1 查找对应的姓名。
 - 公式：=VLOOKUP(A1, D1:E3, 2, FALSE)
 - 结果：“李四”
 - **注意：** VLOOKUP 只能在查找区域的第一列查找，且只能向右查找返回值。
- **HLOOKUP (Horizontal Lookup)**
 - **概念：** 在表格的 **第一行** 中水平查找某个值，并返回同一列中指定行的值。
 - **语法：** HLOOKUP(lookup_value, table_array, row_index_num, [range_lookup])（参数类似 VLOOKUP，但 row_index_num 指的是行号）。
 - **示例：** 用的相对较少，逻辑与 VLOOKUP 类似，只是查找方向是水平的。
- **INDEX**

- **概念：** 返回表格或区域中指定行号和列号交叉处的单元格的值或引用。非常灵活。
- **语法：** INDEX(array, row_num, [column_num])
 - array: 单元格区域。
 - row_num: 行号（在 array 内的相对行号）。
 - column_num: 列号（可选，在 array 内的相对列号）。
- **示例：** 假设区域 D1:E3 同上。
 - 公式: =INDEX(D1:E3, 2, 2)（查找区域第 2 行，第 2 列的值）
 - 结果: “李四”
- **MATCH**
 - **概念：** 在指定区域中查找特定项，并返回该项在区域中的相对位置（行号或列号）。
 - **语法：** MATCH(lookup_value, lookup_array, [match_type])
 - lookup_value: 要查找的值。
 - lookup_array: 要查找的单行或单列区域。
 - match_type: 可选。1 或省略表示查找小于等于 lookup_value 的最大值（要求 lookup_array 升序）；0 表示精确匹配；-1 表示查找大于等于 lookup_value 的最小值（要求 lookup_array 降序）。最常用的是 0（精确匹配）。
 - **示例：** 假设区域 D1:D3 是学号 101, 102, 103。查找 102 的位置。
 - 公式: =MATCH(102, D1:D3, 0)
 - 结果: 2（102 在区域 D1:D3 中的第 2 个位置）
- **INDEX / MATCH 组合（重要！）**
 - **概念：** 结合 INDEX 和 MATCH 可以实现比 VLOOKUP 更灵活的查找。MATCH 找到目标值所在的行号（或列号），INDEX 根据这个行号（或列号）在指定区域中返回值。
 - **优点：**
 - 可以在查找区域的 **任意列** 进行查找（不像 VLOOKUP 只能在第一列）。
 - 可以向左查找返回值。
 - 在插入或删除列时不易出错。
 - **示例：** 假设区域 D1:E3 同上。在 A1 输入姓名“李四”，想在 B1 查找对应的学号。（VLOOKUP 无法直接实现）
 - 步骤 1: 找到“李四”在姓名列（E1:E3）的位置。
MATCH(“李四”, E1:E3, 0) 结果是 2。
 - 步骤 2: 在学数列（D1:D3）中返回第 2 行的值。
 - 组合公式: =INDEX(D1:D3, MATCH(“李四”, E1:E3, 0))
 - 结果: 102
- **XLOOKUP（新版本 Excel，强烈推荐）**
 - **概念：** 最新、更强大的查找函数，旨在取代 VLOOKUP, HLOOKUP, INDEX/MATCH。更简单、更灵活。

- **语法：** XLOOKUP(lookup_value, lookup_array, return_array, [if_not_found], [match_mode], [search_mode])
 - lookup_value: 要查找的值。
 - lookup_array: 在哪个区域查找。
 - return_array: 从哪个区域返回值。
 - if_not_found (可选): 找不到时返回什么值。
 - match_mode (可选): 0 精确匹配 (默认), -1 精确匹配或下一个较小项, 1 精确匹配或下一个较大项, 2 通配符匹配。
 - search_mode (可选): 1 从第一项开始搜索 (默认), -1 从最后一项反向搜索, 2 二分法搜索 (升序), -2 二分法搜索 (降序)。
- **示例：** 假设区域 D1:E3 同上。
 - 查找学号 102 对应的姓名:
 - 公式: =XLOOKUP(102, D1:D3, E1:E3)
 - 结果: "李四"
 - 查找姓名 "李四" 对应的学号 (向左查找):
 - 公式: =XLOOKUP("李四", E1:E3, D1:D3)
 - 结果: 102
 - 查找学号 105, 如果找不到返回 "未找到":
 - 公式: =XLOOKUP(105, D1:D3, E1:E3, "未找到")
 - 结果: "未找到"

4. 条件聚合 (Conditional Aggregation)

这些函数根据一个或多个条件进行求和、计数或求平均值。

- SUMIF / COUNTIF / AVERAGEIF (单条件)
 - **概念：** 根据 **单个条件** 对指定区域进行求和/计数/求平均值。
 - **语法：**
 - SUMIF(range, criteria, [sum_range])
 - COUNTIF(range, criteria)
 - AVERAGEIF(range, criteria, [average_range])
 - range: 用于条件判断的单元格区域。
 - criteria: 判断条件 (如 ">50", "苹果", A1)。
 - sum_range/average_range (可选): 实际要求和/求平均值的区域。如果省略, 则对 range 区域本身进行操作。
 - **示例：** 假设有如下数据在 A1:B4:

	A (产品)	B (销量)
1	苹果	100
2	香蕉	150
3	苹果	120
4	橙子	80

 - 计算苹果的总销量:
 - 公式: =SUMIF(A1:A4, "苹果", B1:B4)
 - 结果: 220 (100 + 120)

- 计算苹果出现的次数：
 - 公式：=COUNTIF(A1:A4, "苹果")
 - 结果：2
- 计算苹果的平均销量：
 - 公式：=AVERAGEIF(A1:A4, "苹果", B1:B4)
 - 结果：110 ((100+120)/2)
- SUMIFS / COUNTIFS / AVERAGEIFS (多条件)
 - 概念： 根据 **多个条件** 对指定区域进行求和/计数/求平均值。所有条件必须 **同时** 满足。
 - 语法： (注意参数顺序与 SUMIF/AVERAGEIF 不同)
 - SUMIFS(sum_range, criteria_range1, criterial, [criteria_range2, criteria2], ...)
 - COUNTIFS(criteria_range1, criterial, [criteria_range2, criteria2], ...)
 - AVERAGEIFS(average_range, criteria_range1, criterial, [criteria_range2, criteria2], ...)
 - sum_range/average_range: 实际要求和/求平均值的区域 (放在第一个参数)。
 - criteria_range1: 第一个条件判断区域。
 - criterial: 第一个条件。
 - criteria_range2, criteria2...: 后续的条件区域和条件。
 - 示例： 假设有如下数据在 A1:C5:

A (区域)	B (产品)	C (销量)
华北	苹果	100
华南	香蕉	90
华北	苹果	120
华南	香蕉	110

 - 计算华北区域苹果的总销量：
 - 公式：=SUMIFS(C1:C5, A1:A5, "华北", B1:B5, "苹果")
 - 结果：220 (100 + 120)
 - 计算华北区域苹果的销售次数：
 - 公式：=COUNTIFS(A1:A5, "华北", B1:B5, "苹果")
 - 结果：2
 - 计算华北区域苹果的平均销量：
 - 公式：=AVERAGEIFS(C1:C5, A1:A5, "华北", B1:B5, "苹果")
 - 结果：110

5. 文本处理 (Text Functions)

这些函数用于操作文本字符串。

- LEFT
 - 概念： 从文本字符串的 **左侧 (开头)** 提取指定数量的字符。

- **语法:** LEFT(text, [num_chars]) (num_chars 省略时默认为 1)。
- **示例:** 假设 A1 单元格是 "Hello World"。
 - 公式: =LEFT(A1, 5)
 - 结果: "Hello"
- **RIGHT**
 - **概念:** 从文本字符串的 **右侧（末尾）** 提取指定数量的字符。
 - **语法:** RIGHT(text, [num_chars])
 - **示例:** 假设 A1 单元格是 "Hello World"。
 - 公式: =RIGHT(A1, 5)
 - 结果: "World"
- **MID**
 - **概念:** 从文本字符串的 **指定位置开始** 提取指定数量的字符。
 - **语法:** MID(text, start_num, num_chars)
 - start_num: 开始提取的位置（第一个字符是 1）。
 - num_chars: 要提取的字符数。
 - **示例:** 假设 A1 单元格是 "Hello World"。
 - 公式: =MID(A1, 7, 5) (从第 7 个字符开始, 提取 5 个字符)
 - 结果: "World"
- **CONCATENATE / &**
 - **概念:** 将两个或多个文本字符串连接（合并）成一个字符串。使用 & 符号通常更简洁。
 - **语法:**
 - CONCATENATE(text1, [text2], ...)
 - text1 & text2 & ...
 - **示例:** 假设 A1="Hello", B1="World"。
 - 公式 1: =CONCATENATE(A1, " ", B1)
 - 公式 2: =A1 & " " & B1
 - 结果: "Hello World" (注意中间加了空格)
- **LEN**
 - **概念:** 返回文本字符串中的字符数（长度）。
 - **语法:** LEN(text)
 - **示例:** 假设 A1 单元格是 "Hello"。
 - 公式: =LEN(A1)
 - 结果: 5
- **FIND**
 - **概念:** 在一个文本字符串中查找另一个文本字符串, 并返回 **第一个** 匹配项的起始位置（字符编号）。**区分大小写**。如果找不到, 返回 #VALUE! 错误。
 - **语法:** FIND(find_text, within_text, [start_num])
(start_num 可选, 指定从 within_text 的哪个位置开始查找)。
 - **示例:** 假设 A1 单元格是 "Hello World"。
 - 公式: =FIND("W", A1) (查找 "W" 的位置)
 - 结果: 7

- 公式: =FIND("w", A1) (区分大小写, 找不到 "w")
- 结果: #VALUE! (可以用 SEARCH 函数进行不区分大小写的查找)
- **SUBSTITUTE**
 - **概念:** 将文本字符串中的旧文本替换为新文本。
 - **语法:** SUBSTITUTE(text, old_text, new_text, [instance_num]) (instance_num 可选, 指定替换第几个出现的 old_text, 省略则全部替换)。
 - **示例:** 假设 A1 单元格是 "apple,banana,apple"。
 - 公式: =SUBSTITUTE(A1, "apple", "orange") (全部替换)
 - 结果: "orange,banana,orange"
 - 公式: =SUBSTITUTE(A1, "apple", "orange", 1) (只替换第一个)
 - 结果: "orange,banana,apple"
- **TEXT**
 - **概念:** 将数值转换为按指定格式表示的文本。常用于将数字或日期嵌入到文本字符串中。
 - **语法:** TEXT(value, format_text)
 - value: 要转换的数值或日期。
 - format_text: 要应用的格式代码 (用引号括起来, 如 "yyyy-mm-dd", "0.00%", "# ##0")。
 - **示例:**
 - 假设 A1=0.85 (表示 85%)。
 - 公式: ="完成率: " & TEXT(A1, "0.0%")
 - 结果: "完成率: 85.0%"
 - 假设 A2 包含日期 2025/4/21。
 - 公式: ="报告日期: " & TEXT(A2, "yyyy 年 m 月 d 日")
 - 结果: "报告日期: 2025 年 4 月 21 日" (这里的日期基于我回复时的时间, 实际会根据 A2 的值变化)

6. 日期时间 (Date & Time Functions)

这些函数用于处理日期和时间。Excel 将日期存储为序列号 (从 1900-1-1 开始为 1), 将时间存储为小数。

- **TODAY()**
 - **概念:** 返回 **当前日期** 的序列号。每次打开或重新计算工作簿时都会更新 (易失性函数)。
 - **语法:** TODAY() (无参数)
 - **示例:**
 - 公式: =TODAY()

- 结果：单元格显示当前日期，例如 2025/4/21 （需要设置单元格格式为日期）。
- NOW()
 - 概念： 返回 **当前日期和时间** 的序列号。也是易失性函数。
 - 语法： NOW() （无参数）
 - 示例：
 - 公式： =NOW()
 - 结果： 单元格显示当前日期和时间，例如 2025/4/21 12:06 （需要设置单元格格式为日期时间）。
- YEAR / MONTH / DAY
 - 概念： 从一个日期序列号中分别提取年、月、日。
 - 语法： YEAR(serial_number), MONTH(serial_number), DAY(serial_number)
 - 示例： 假设 A1 包含日期 2025/4/21。
 - 公式 YEAR： =YEAR(A1) 结果： 2025
 - 公式 MONTH： =MONTH(A1) 结果： 4
 - 公式 DAY： =DAY(A1) 结果： 21
- DATE
 - 概念： 根据给定的年、月、日数字，返回表示该日期的序列号。
 - 语法： DATE(year, month, day)
 - 示例：
 - 公式： =DATE(2025, 4, 21)
 - 结果： 单元格显示日期 2025/4/21。
- DATEDIF
 - 概念： 计算两个日期之间的 **完整** 时间单位数（天数、月数、年数）。这是一个非常有用的“隐藏”函数（在函数列表中可能找不到，但可以直接输入使用）。
 - 语法： DATEDIF(start_date, end_date, unit)
 - start_date： 开始日期。
 - end_date： 结束日期（end_date 必须晚于或等于 start_date）。
 - unit： 要返回的时间单位：
 - "Y"： 整年数。
 - "M"： 整月数。
 - "D"： 天数。
 - "YM"： 忽略年份的整月数。
 - "YD"： 忽略年份的天数。
 - "MD"： 忽略年份和月份的天数。
 - 示例： 假设 A1=2024/1/15, B1=2025/4/21。
 - 计算相差多少 **整年**：
 - 公式： =DATEDIF(A1, B1, "Y")
 - 结果： 1
 - 计算相差多少 **整月**：
 - 公式： =DATEDIF(A1, B1, "M")

- 结果：15（从 2024/1/15 到 2025/4/15 是 15 个整月）
- 计算相差多少 天：
 - 公式：=DATEDIF(A1, B1, "D")
 - 结果：462（可以通过 B1-A1 得到同样结果）
- 计算不足一年的 整月数 ("YM")：
 - 公式：=DATEDIF(A1, B1, "YM")
 - 结果：3（15 个月 = 1 年 + 3 个月）
- 计算不足一年的 天数 ("YD")（不常用）：
 - 公式：=DATEDIF(A1, B1, "YD")
 - 结果：96（从 2025/1/15 到 2025/4/21 的天数）
- 计算不足一月的 天数 ("MD")（不常用，有时结果不稳定）：
 - 公式：=DATEDIF(A1, B1, "MD")
 - 结果：6（从 2025/4/15 到 2025/4/21 的天数）

数据透视表 (PivotTables)

目标：熟练使用数据透视表对大量数据进行快速、灵活的汇总、分析和报告。

核心概念：

数据透视表是一种强大的工具，它可以让你根据一个或多个键（字段）对数据进行聚合。你可以快速地将行转换为列，列转换为行，或者按任意顺序组织字段，以便对数据进行各种计算（如求和、计数、平均值等）。它的最大优点是**交互性和灵活性**，你可以轻松地拖放字段来改变分析视角。

想象你有一张包含了几万行销售记录的表格（日期、区域、产品、销售额、数量等）。如果你想知道：

- 每个区域的总销售额是多少？
- 每个产品在不同区域的销售额分布？
- 每月或每年的销售趋势？
- 哪个区域的平均销售额最高？

手动计算这些将非常耗时且容易出错，但使用数据透视表可以在几秒钟内完成。

学习内容详解与举例：

假设我们有如下简单的销售数据表：

日期	区域	产品	销售额	数量
2023-01-05	华东	A	1000	10

奕道远信息

2023-01-06	华北	B	1500	15
2023-01-07	华东	A	1200	12
2023-01-07	华南	C	2000	20
2023-01-08	华北	A	800	8
2023-01-09	华东	B	1800	18

1. 创建数据透视表：

- **步骤：**
 - 选中你的原始数据区域（包括标题行）。
 - 点击 Excel 菜单栏的“插入”选项卡。
 - 在左侧找到并点击“数据透视表”。
 - 在弹出的对话框中，确认数据范围正确。
 - 选择放置数据透视表的位置：通常选择“新工作表”更清晰。
 - 点击“确定”。
- **结果：**Excel 会新建一个工作表，并在右侧出现“数据透视表字段列表”窗格，左侧是空的透视表框架。

2. 理解行、列、值、筛选器区域：

在“数据透视表字段列表”窗格底部，有四个区域：

- **筛选器 (Filters)：**放在这里的字段将成为整个数据透视表的全局筛选条件。你可以选择一个或多个值来过滤显示的数据。
 - **例：**将“区域”字段拖到“筛选器”区域。透视表上方会出现一个区域筛选框，你可以选择只看“华东”或“华北”的数据。
- **列 (Columns)：**放在这里的字段的唯一值将显示为数据透视表的列标题。
 - **例：**将“产品”字段拖到“列”区域。数据透视表的列标题将显示“A”，“B”，“C”等产品名称。
- **行 (Rows)：**放在这里的字段的唯一值将显示为数据透视表的行标题。
 - **例：**将“区域”字段拖到“行”区域。数据透视表的行标题将显示“华东”，“华北”，“华南”等区域名称。
- **值 (Values)：**放在这里的字段通常是需要进行计算（汇总）的数值字段。默认情况下，数值字段会进行求和（SUM）。
 - **例：**将“销售额”字段拖到“值”区域。数据透视表的核心区域将显示每个区域（或产品，取决于行/列的设置）的总销售额。

组合应用举例：

- 将“区域”拖到 **行**，将“产品”拖到 **列**，将“销售额”拖到 **值**。你会得到一个交叉表，显示每个区域下，每个产品的总销售额。
- 将“区域”拖到 **行**，将“销售额”拖到 **值**。你会得到每个区域的总销售额列表。

3. 更改值汇总方式（求和、计数、平均值等）：

默认是求和，但你可以轻松改变。

- **步骤：**
 - 在“值”区域中，点击你想要更改的字段（例如“求和项：销售额”）。
 - 选择“值字段设置...”。
 - 在弹出的对话框的“汇总方式”选项卡中，选择你想要的计算类型，如“计数”（Count）、“平均值”（Average）、“最大值”（Max）、“最小值”（Min）等。
 - 你还可以更改字段的自定义名称（例如将“求和项：销售额”改为“总销售额”）。
 - 点击“确定”。
- **例：**将“销售额”的汇总方式从“求和”改为“平均值”，透视表将显示每个区域（或产品）的平均销售额。将“数量”拖到“值”区域并选择“计数”，可以知道每个区域或产品有多少条销售记录。

4. 值显示方式（占比、差异等）：

除了基本的汇总计算，你还可以用不同的方式显示值，比如占总体的百分比、与某个基准的差异等。

- **步骤：**
 - 在“值”区域中，点击你想要更改的字段。
 - 选择“值字段设置...”。
 - 切换到“值显示方式”选项卡。
 - 在下拉列表中选择你想要的显示方式。
- **常用显示方式举例：**
 - **无计算（No Calculation）：**默认方式，显示汇总结果。
 - **总计的百分比（% of Grand Total）：**显示该值占整个数据透视表总计的百分比。
 - **列汇总的百分比（% of Column Total）：**显示该值占其所在列总计的百分比。
 - **行汇总的百分比（% of Row Total）：**显示该值占其所在行总计的百分比。
 - **差异（Difference From）：**显示该值与指定基本字段和基本项之间的差异。常用于比较与上期或上一年的差异。
 - **汇总百分比（Running Total In）：**计算累积百分比。
 - **排名（Rank Smallest to Largest / Largest to Smallest）：**根据值进行排名。
- **例：**将“销售额”字段再次拖到“值”区域，这次将第二个“销售额”字段的显示方式设置为“列汇总的百分比”。这样，你就可以在同一张表中看到每个区域的总销售额以及该区域销售额占总销售额的百分比。

5. 分组（日期、数值）：

当你有日期或连续数值字段时，分组功能非常有用。

- **日期分组：**
 - 将日期字段拖到 **行** 或 **列** 区域。
 - **右键点击** 日期字段的任意一个单元格（例如 “2023-01-05”）。
 - 选择 **“创建组...”**。
 - 在弹出的对话框中，选择你想要的分组单位（如 “月”，“季度”，“年”）。你可以同时选择多个单位，例如先按年分组，再按月分组。
 - 点击 **“确定”**。
 - **例：**将 “日期” 字段分组为 “年” 和 “月”，然后将 “销售额” 拖到 “值”。透视表会显示每年的总销售额，展开年份可以看到每个月的销售额。
- **数值分组：**
 - 将数值字段（如 “销售额” 或 “数量”）拖到 **行** 或 **列** 区域。
 - **右键点击** 该数值字段的任意一个单元格。
 - 选择 **“创建组...”**。
 - Excel 会尝试建议一个分组的起始值、结束值和步长。你可以根据需要修改这些值。
 - 点击 **“确定”**。
 - **例：**将 “销售额” 字段分组，步长设为 500，你可以看到销售额在 0-500, 501-1000, 1001-1500 等各个区间的记录数或总销售额。

6. 添加计算字段和计算项：

- **计算字段 (Calculated Field)：**基于源数据中的现有字段进行计算，结果作为一个新的字段添加到数据透视表中。
 - **场景：**计算利润（销售额 - 成本），计算每单位销售额（销售额 / 数量），计算佣金（销售额 * 佣金率）等。
 - **步骤：**
 - 选中数据透视表中的任意单元格。
 - 在 Excel 菜单栏上会出现 **“数据透视表分析”**（或 “选项”）选项卡。
 - 点击 **“字段、项目和集”** (Fields, Items, & Sets)。
 - 选择 **“计算字段...”**。
 - 输入字段名称（例如 “利润”）。
 - 输入公式（例如 = 销售额 - 成本，这里的 “销售额” 和 “成本” 是源数据中的字段名，双击字段列表中的名称可以将其添加到公式框）。
 - 点击 **“添加”**，然后 **“确定”**。
 - 新的计算字段 “利润” 会出现在字段列表中，你可以像其他字段一样使用它。

- **计算项 (Calculated Item)**: 在现有字段的 项 之间进行计算。相对较少用, 且有较多限制 (如不能用于某些汇总方式, 不能用于分组字段)。
 - **场景**: 如果你的 “区域” 字段有 “华东”, “华北”, “华南”, 你可能想计算 “北方区” 的总和, 它等于 “华北” + “西北” (如果存在)。
 - **步骤**: (与计算字段类似, 但在 “字段、项目和集” 中选择 “计算项...”)
 - **注意**: 对于大多数常见的数据分析需求, 计算字段更常用且灵活。初学阶段可以先重点掌握计算字段。

7. 使用切片器 (Slicers) 和日程表 (Timelines) 进行交互式筛选:

切片器和日程表是比传统筛选器更直观、交互性更强的筛选工具。

- **切片器 (Slicers)**:
 - **步骤**:
 - 选中数据透视表中的任意单元格。
 - 在 “数据透视表分析” (或 “选项”) 选项卡中, 点击 “插入切片器”。
 - 选择你想要用作筛选器的字段 (例如 “区域”, “产品”)。
 - 点击 “确定”。
 - Excel 会在工作表中插入一个或多个切片器窗口。每个切片器显示该字段的所有唯一值按钮。点击按钮即可快速筛选透视表数据。你可以按住 Ctrl 键选择多个项目, 也可以清除筛选。切片器可以连接到多个数据透视表, 实现联动筛选。
- **日程表 (Timelines)**:
 - **步骤**:
 - 选中数据透视表中的任意单元格。
 - 在 “数据透视表分析” (或 “选项”) 选项卡中, 点击 “插入日程表”。
 - 选择数据源中的日期字段。
 - 点击 “确定”。
 - Excel 会插入一个日程表窗口。你可以选择按年、季度、月或天来拖动时间条, 快速筛选数据。
- **例**: 插入一个 “区域” 切片器和一个 “日期” 日程表。你可以通过点击切片器中的 “华东”, 然后拖动日程表选择 “2023 年第一季度”, 快速查看华东区域在 2023 年第一季度的销售数据。

总结数据透视表:

它是 Excel 数据分析的核心工具之一。通过灵活运用行、列、值、筛选器这四个区域, 配合值汇总/显示方式、分组、计算字段以及切片器/日程表, 你可以从各个维度深入分析数据, 快速生成各种汇总报告。多练习是掌握它的关键。

数据可视化（基础图表）

目标：掌握创建常用图表的方法，理解不同图表的适用场景，能够对图表进行基本编辑，使数据分析结果更直观易懂。

核心概念：

数据可视化是将数据转化为图形或图像的过程。它能帮助我们更快地发现数据中的模式、趋势、异常值，并更容易地向他人传达分析结果。俗话说“一图胜千言”，恰如其分地描述了数据可视化的作用。

学习内容详解与举例：

基于之前的数据或数据透视表的汇总结果，我们可以创建图表。

1. 创建常用图表：

- **步骤：**
 - 选择包含你想要绘制图表的数据区域（例如，如果用数据透视表得到了各区域的总销售额，就选中区域名称和对应的总销售额这两列数据）。
 - 点击 Excel 菜单栏的“插入”选项卡。
 - 在“图表”组中，你可以：
 - 点击“推荐的图表”：Excel 会根据你的数据推荐合适的图表类型。
 - 直接点击特定的图表类型图标（如柱状图、折线图等）。
 - 选择你想要的具体图表样式。
 - 点击“确定”。

2. 理解不同图表的适用场景：

选择合适的图表类型是有效可视化的关键。

- **柱状图（Column Chart） / 条形图（Bar Chart）：**
 - **适用场景：**比较不同类别之间的数值大小。
 - **区别：**柱状图是垂直的柱子，通常用于表示时间序列数据（如每月销售额）或类别名称较短的情况。条形图是水平的条形，更适合类别名称较长或者比较的项目数量较多的情况。
 - **例：**比较不同区域的总销售额（用柱状图或条形图）。
- **折线图（Line Chart）：**
 - **适用场景：**显示数据随时间或其他有序类别变化的趋势。强调数据的变化方向和速率。
 - **例：**显示销售额按月或按年变化的趋势（用折线图，X 轴为日期/月份，Y 轴为销售额）。

- **饼图 (Pie Chart):**
 - **适用场景:** 显示一个整体中各部分所占的比例, 所有部分的比例之和必须是 100%。最好只用于少量类别 (通常不超过 5-7 个), 否则难以辨认。
 - **例:** 显示不同区域的销售额占总销售额的百分比 (用饼图)。
- **散点图 (Scatter Chart):**
 - **适用场景:** 显示两个数值变量之间的关系或相关性。每个点代表一对数值。
 - **例:** 分析销售额和数量之间是否存在线性关系 (用散点图, X 轴为数量, Y 轴为销售额)。
- **其他常用图表 (简述):**
 - **面积图 (Area Chart):** 类似于折线图, 但线条下方区域被填充颜色, 可以显示随时间变化的趋势, 并强调总量。
 - **组合图 (Combo Chart):** 在同一个图表中结合两种或多种图表类型, 通常用于展示有不同度量单位或尺度的数据系列 (如销售额和利润率)。

3. 图表元素编辑 (标题、坐标轴、图例、数据标签):

创建图表后, 你需要编辑其元素使其更清晰易懂。

- **步骤:**
 - 点击图表选中它。
 - 图表右上角会出现三个小按钮: + (图表元素), 画刷 (图表样式), 漏斗 (图表筛选器)。
 - 点击 "+" (图表元素) 按钮。一个列表会展开, 你可以勾选/取消勾选各种图表元素, 并点击右侧的小箭头进行进一步设置。
- **常用图表元素:**
 - **图表标题 (Chart Title):** 描述图表内容的标题。点击标题文本框即可编辑。
 - **坐标轴标题 (Axis Titles):** 说明 X 轴和 Y 轴代表什么。特别是在有多个系列或单位时非常重要。
 - **数据标签 (Data Labels):** 直接在数据点旁边显示具体的数值。
 - **数据表 (Data Table):** 在图表下方显示用于绘制图表的数据表格。
 - **误差线 (Error Bars):** 显示数据的变异性或不确定性 (常用于统计分析)。
 - **网格线 (Gridlines):** 帮助读取图表数值的水平和垂直线条。
 - **图例 (Legend):** 说明图表中不同颜色或标记代表的数据系列。
 - **趋势线 (Trendline):** 显示数据系列的发展趋势 (常用于折线图或散点图)。
- **其他编辑:**
 - 点击图表中的任何元素 (如坐标轴、数据系列、标题), 右键选择 "设置格式..." 可以打开详细的格式设置窗格, 调整颜色、字体、线条、填充等。

- 点击 **画刷** 按钮可以快速更改图表整体样式和配色方案。
- 点击 **漏斗** 按钮可以筛选图表中显示的数据系列或类别。
- **例：**给柱状图添加“区域销售额分布”的标题，给 X 轴添加“区域”标题，给 Y 轴添加“销售额（元）”标题，并在每个柱子上方显示数据标签。

4. 简单的组合图表：

在一个图表中展示两个不同类型或不同量级的数据系列。

- **场景：**显示每月销售额（柱状图）和每月利润率（折线图）。销售额单位是元，利润率是百分比，两者尺度不同，需要双坐标轴。
- **步骤：**
 - 选择包含两个数据系列以及类别（如月份）的数据区域。
 - 点击 **“插入”** 选项卡 -> **“推荐的图表”**。
 - 切换到 **“所有图表”** 选项卡。
 - 选择 **“组合图”**。
 - 在组合图设置界面，为每个数据系列选择合适的图表类型（例如，销售额选“簇状柱形图”，利润率选“折线图”）。
 - 对于需要不同尺度的数据系列（如利润率），勾选旁边的 **“次坐标轴”**。
 - 点击 **“确定”**。
- **例：**创建一个柱状图+折线图的组合图，柱状图显示各区域的总销售额，折线图显示各区域的平均数量。由于销售额和数量的量级可能不同，将数量折线图设置到次坐标轴。

总结基础图表：

学会根据数据类型和分析目的选择合适的图表，并通过编辑图表元素使其清晰、准确、美观，是有效沟通数据分析结果的关键技能。

(进阶选学)：条件格式

目标：使用条件格式根据单元格的值自动应用不同的格式，突出显示数据中的模式、趋势或异常值。

核心概念：

条件格式允许你设置规则，当单元格满足特定条件时，自动改变其字体颜色、填充颜色、边框样式等。这比手动格式化效率高得多，并且会随着数据变化而更新。

学习内容详解与举例：

奕道远信息

- 应用场景：
 - 突出显示高于某个阈值的销售额。
 - 标记出重复的客户 ID。
 - 用颜色条表示进度或大小。
 - 用图标集表示高、中、低等级。
 - 标示出排名前 10% 的优秀销售员。
- 创建条件格式规则：
 - 步骤：
 - 选中你想要应用条件格式的单元格区域。
 - 点击 Excel 菜单栏的“开始”选项卡。
 - 在“样式”组中，点击“条件格式”。
 - 在下拉菜单中选择你想要的规则类型，或者选择“新建规则...”进行更详细的设置。
- 常见规则类型举例：
 - 突出显示单元格规则 (Highlight Cells Rules):
 - 大于 (Greater Than)...: 突出显示大于某个值的单元格。
 - 例：选中销售额列，条件格式 -> 突出显示单元格规则 -> 大于... -> 输入 1500 -> 选择“绿色填充深绿色文本”。所有销售额大于 1500 的单元格都会变绿。
 - 小于 (Less Than)...
 - 介于 (Between)...
 - 等于 (Equal To)...
 - 文本包含 (Text that Contains)...
 - 发生日期 (A Date Occurring)...: (用于日期)
 - 重复值 (Duplicate Values): 突出显示选中区域中的重复值。
 - 例：选中客户 ID 列，条件格式 -> 突出显示单元格规则 -> 重复值。所有重复的客户 ID 会被突出显示。
 - 最前/最后规则 (Top/Bottom Rules):
 - 前 10 项 (Top 10 Items)...: 突出显示区域内最大的 N 个值。
 - 前 10% (Top 10%)...: 突出显示区域内最大的前 N% 的值。
 - 后 10 项 (Bottom 10 Items)...
 - 后 10% (Bottom 10%)...
 - 高于平均值 (Above Average):
 - 低于平均值 (Below Average):
 - 数据条 (Data Bars): 在单元格内绘制一个与单元格值成比例的彩色条。
 - 例：选中销售额列，条件格式 -> 数据条 -> 选择一个颜色。每个单元格的背景会出现一个彩色条，条的长短反映销售额的大小。
 - 色阶 (Color Scales): 根据单元格的值，应用一个渐变颜色。

- **例：**选中销售额列，条件格式 -> 色阶 -> 选择一个红-黄-绿色阶。高销售额的单元格显示绿色，低销售额显示红色，中间值显示黄色或橙色。
- **图标集 (Icon Sets)：**根据单元格值与设定阈值的比较结果，显示不同的图标（如箭头、红绿灯、星级）。
 - **例：**选中销售额列，条件格式 -> 图标集 -> 选择三色交通灯。Excel 会自动将值分为三组（高、中、低）并显示相应的图标（绿色圆表示高，黄色方块表示中，红色菱形表示低）。你可以在“管理规则”中修改分组的阈值。
- **管理规则：**通过“条件格式” -> “管理规则...”可以查看、编辑、删除或重新排序已经应用的条件格式规则。

总结条件格式：

是数据展示和初步分析的利器，能让你快速识别数据中的关键信息、高低值、趋势和异常情况。

(进阶选学)：Power Query

目标：了解 Power Query 的作用，它是 Excel 中一个强大的数据获取、转换和加载 (ETL) 工具。

核心概念：

Power Query (在 Excel 2016 及更高版本通常内置在“数据”选项卡下的“获取和转换”组中，旧版本需要安装插件) 允许你：

1. **连接 (Connect)：**从各种不同的数据源获取数据（数据库、本地文件、网页、其他 Excel 工作簿、SharePoint 等）。
2. **转换 (Transform)：**清理、重塑和合并数据。这是 Power Query 的核心功能，你可以在一个可视化的用户界面中完成许多复杂的数据处理任务，如：
 - 更改列名和数据类型。
 - 删除不需要的列或行。
 - 处理缺失值或错误值。
 - 筛选和排序数据。
 - 拆分列（例如将“姓名”拆分为“姓”和“名”）。
 - 合并列。
 - 逆透视列 (Unpivot Columns)：将交叉表格式的数据转换为规范的表格格式。
 - 添加自定义列（使用 M 语言编写简单公式）。

- 合并查询 (Merge Queries): 类似于 SQL 的 JOIN, 根据共同列连接两个或多个表。
 - 追加查询 (Append Queries): 将两个或多个结构相同的表堆叠在一起。
3. **加载 (Load)**: 将转换后的干净数据加载到 Excel 工作表、数据模型 (用于 Power Pivot) 或只创建连接。

与 Excel 常规操作的区别:

- **可重复性**: Power Query 记录你进行的每一步转换操作。下次数据源更新时, 你只需要点击 **“刷新”**, Power Query 就会自动重跑所有步骤, 无需手动重复。
- **处理大数据**: Power Query 在加载到 Excel 之前对数据进行处理, 可以更有效地处理比 Excel 行数上限更大的数据集 (虽然最终加载到工作表仍受限制, 但可以加载到数据模型)。
- **连接外部数据源更方便和稳定**。
- **数据清洗能力更强更自动化**。

学习 Power Query 的意义:

在实际工作中, 原始数据通常是不干净、不规范的。你可能需要从多个来源获取数据, 进行大量的清洗和整合。Power Query 能极大地提高这部分工作的效率和准确性, 让你将更多时间用于数据分析本身, 而不是耗费在繁琐的数据准备上。

举例 (概念性):

假设你有三个月的销售数据, 分别保存在三个不同的 CSV 文件中, 每个文件列名稍有不同, 且包含一些空白行和错误数据。

使用 Power Query 的流程可能是:

1. 连接到第一个 CSV 文件。
2. 在 Power Query 编辑器中, 重名列使其规范, 删除空白行和错误数据。
3. 连接到第二个 CSV 文件, 进行相同的清洗步骤。
4. 连接到第三个 CSV 文件, 进行相同的清洗步骤。
5. 使用 **“追加查询”** 将这三三个月的干净数据合并到一个查询中。
6. 将合并后的查询加载到 Excel 工作表。

下次有新的月份数据时, 你只需要将新文件放到指定位置, 然后在 Excel 中点击 **“刷新”**, Power Query 就会自动完成上述所有步骤, 将新数据追加到现有数据中。

总结 Power Query:

奕道远信息

它是 Excel 中用于数据预处理的瑞士军刀。虽然初学可能需要一点时间适应其界面和逻辑，但掌握后能极大地提升数据处理效率，是进阶 Excel 用户和数据分析师的必备技能。

二、 SQL (数据查询与提取)

目标：掌握 SQL 基础语法，能够从数据库中查询、筛选、聚合和连接所需数据。

核心概念：

SQL (Structured Query Language, 结构化查询语言) 是用于管理关系型数据库系统的标准语言。它不是一种编程语言，而是一种声明式语言，你告诉数据库你想要什么数据，而不是告诉它如何一步步去获取。

关系型数据库将数据存储在**表 (Tables)** 中。每个表都像一个电子表格：

- **行 (Rows)：**代表一个独立的记录或实体（例如，一个客户、一个订单、一个产品）。
- **列 (Columns)：**代表记录的属性或字段（例如，客户的姓名、订单的日期、产品的价格）。

1. 数据库基础概念详解：

- **理解关系型数据库（表、行、列）：**
 - 就像上面提到的，数据以二维表格的形式组织。
 - **表** 是数据的集合，有特定的结构（由列定义）。
 - **行** 是表中的一条记录。
 - **列** 是表中的一个字段，存储特定类型的数据。
 - **举例：**
 - **customers 表：**包含 `customer_id` (客户 ID)，`customer_name` (客户姓名)，`city` (城市) 等列。
 - **orders 表：**包含 `order_id` (订单 ID)，`customer_id` (客户 ID)，`order_date` (订单日期)，`amount` (金额) 等列。
- **主键 (Primary Key) 和外键 (Foreign Key) 的概念：**
 - **主键 (Primary Key - PK)：**表中的一列或一组列，其值能唯一标识表中的每一行。
 - **特点：**值唯一，不能为空 (NOT NULL)。
 - **作用：**确保每条记录都有一个唯一的身份，是建立表之间关系的基础。
 - **举例：**在 `customers` 表中，`customer_id` 通常被设为主键，因为每个客户 ID 是唯一的。在 `orders` 表中，`order_id` 通常被设为主键。

- **外键 (Foreign Key - FK)**: 表中的一列或一组列, 它引用了另一个表中的主键。外键用来在两个表之间建立联系。
 - 作用: 维护数据之间的引用完整性, 确保引用的数据是存在的。
 - **举例**: 在 orders 表中, customer_id 列是一个外键, 它引用了 customers 表中的 customer_id 主键。这意味着每个订单都必须对应一个在 customers 表中存在的客户。
- **常见数据类型**:
 - SQL 支持多种数据类型, 不同的数据库系统可能略有差异, 但核心类型类似。
 - **整数**: INT, INTEGER, SMALLINT (用于存储整数, 根据大小范围选择)
 - **浮点数**: FLOAT, DOUBLE, REAL (用于存储近似数值)
 - **精确小数**: DECIMAL, NUMERIC (用于存储精确数值, 如货币)
 - **字符串/文本**: VARCHAR(n), TEXT, CHAR(n) (用于存储文本, VARCHAR(n) 可变长度, CHAR(n) 固定长度, TEXT 用于大块文本)
 - **日期/时间**: DATE, TIME, DATETIME, TIMESTAMP (用于存储日期和时间信息)
 - **布尔型**: BOOLEAN, BOOL (用于存储真/假值)
 - **举例**:
 - customer_id: INT
 - customer_name: VARCHAR(50)
 - amount: DECIMAL(10, 2) (总共 10 位数, 小数点后 2 位)
 - order_date: DATE

2. 核心查询语句 (DQL - Data Query Language) 详解:

DQL 主要指 SELECT 语句, 它是 SQL 中最常用的部分, 用于从数据库中检索数据。基本结构通常是: SELECT 列名 FROM 表名 WHERE 条件 GROUP BY 列名 HAVING 条件 ORDER BY 列名 LIMIT 行数; (并非所有部分都必须出现)

- **SELECT**: 指定你想要从表中 retrieve (检索) 哪些列。
 - *: 代表选择所有列。
 - 列名 1, 列名 2, ...: 选择指定的列。
 - **举例**:
 - 选择 products 表中的所有列:

SQL

```
SELECT * FROM products;
```

- 选择 customers 表中的 customer_name 和 city 列:

SQL

奕道远信息

```
SELECT customer_name, city FROM customers;
```

- **FROM:** 指定你要从哪个表或哪些表中查询数据。
 - FROM 表名: 从单个表查询。
 - FROM 表名 1 JOIN 表名 2 ON ...: 从多个表查询 (见下文的连接)。
 - **举例:** (已包含在 SELECT 举例中)
- **WHERE:** 用于设置过滤条件, 只选择满足特定条件的行。
 - 可以使用各种比较操作符: = (等于), > (大于), < (小于), >= (大于等于), <= (小于等于), != 或 <> (不等于)。
 - 可以使用逻辑操作符组合条件: AND (并且), OR (或者), NOT (不)。
 - 可以使用特殊操作符:
 - LIKE '模式%': 模糊匹配, % 代表任意多个字符, _ 代表任意一个字符。
 - IN (值 1, 值 2, ...): 检查值是否在列表内。
 - BETWEEN 值 1 AND 值 2: 检查值是否在某个范围内 (包含边界)。
 - IS NULL: 检查值是否为 NULL。
 - IS NOT NULL: 检查值是否不为 NULL。
 - **举例:**
 - 选择 products 表中价格大于 50 的产品:

SQL

```
SELECT * FROM products WHERE price > 50;
```

- 选择 orders 表中金额大于 1000 且日期在 2023 年的订单:

SQL

```
SELECT * FROM orders WHERE amount > 1000 AND  
order_date BETWEEN '2023-01-01' AND '2023-12-31';
```

- 选择 customers 表中城市是 'New York' 或 'London' 的客户:

SQL

```
SELECT * FROM customers WHERE city IN ('New York',  
'London');
```

- 选择 products 表中名称以 'Laptop' 开头的产品:

尖道远信息

SQL

```
SELECT * FROM products WHERE product_name LIKE  
'Laptop%';
```

- 选择 customers 表中城市信息为空的客户：

SQL

```
SELECT * FROM customers WHERE city IS NULL;
```

- **ORDER BY:** 用于对查询结果进行排序。
 - ASC: 升序（默认）。
 - DESC: 降序。
 - 可以按多个列排序，先按第一个列排序，如果值相同，再按第二个列排序，以此类推。
 - **举例:**

- 选择 products 表中所有产品，按价格从低到高排序：

SQL

```
SELECT * FROM products ORDER BY price ASC; -- ASC  
可以省略
```

- 选择 orders 表中所有订单，按订单日期从新到旧排序：

SQL

```
SELECT * FROM orders ORDER BY order_date DESC;
```

- 选择 customers 表中的客户，先按城市升序排序，再按客户姓名升序排序：

SQL

```
SELECT * FROM customers ORDER BY city ASC,  
customer_name ASC;
```

- **LIMIT / TOP:** 用于限制查询结果返回的行数。这在只需要查看少量数据进行预览或获取排行榜等场景很有用。
 - 语法因数据库而异：
 - MySQL, PostgreSQL, SQLite 使用 LIMIT 行数。
 - SQL Server, MS Access 使用 TOP 行数（放在 SELECT 后面）。
 - **举例:**

奕道远信息

- (MySQL/PostgreSQL/SQLite) 获取价格最高的前 10 个产品:

SQL

```
SELECT * FROM products ORDER BY price DESC LIMIT 10;
```

- (SQL Server) 获取价格最高的前 10 个产品:

SQL

```
SELECT TOP 10 * FROM products ORDER BY price DESC;
```

3. 数据聚合与分组详解:

当你需要对一组行进行计算（如求总和、平均值、计数）而不是处理单行时，使用聚合函数和 GROUP BY。

- **常用聚合函数:**

- COUNT(列名): 计算指定列中非 NULL 值的行数。COUNT(*) 计算所有行的数量（包括 NULL）。
- SUM(列名): 计算指定列的总和（仅适用于数值类型）。
- AVG(列名): 计算指定列的平均值（仅适用于数值类型）。
- MIN(列名): 找出指定列的最小值。
- MAX(列名): 找出指定列的最大值。
- **举例:**

- 计算 orders 表中总共有多少行（多少个订单）:

SQL

```
SELECT COUNT(*) FROM orders;
```

- 计算 products 表中所有产品的总价格:

SQL

```
SELECT SUM(price) FROM products;
```

- 计算 sales 表中平均销售额:

SQL

```
SELECT AVG(amount) FROM sales;
```

奕道远信息

- **GROUP BY:** 用于将具有相同值的行分组到一起，然后对每个组应用聚合函数。
 - 规则: SELECT 语句中，除了聚合函数外，所有未被聚合的列都必须出现在 GROUP BY 子句中。

- **举例:**

- 计算每个客户的总订单金额:

SQL

```
SELECT customer_id, SUM(amount) FROM orders GROUP BY customer_id;
```

解释: SQL 会找到所有 customer_id 相同的行，将它们分为一组，然后计算该组行的 amount 的总和。

- 计算不同城市中客户的数量:

SQL

```
SELECT city, COUNT(*) FROM customers GROUP BY city;
```

- 计算不同产品在不同区域的总销售额:

SQL

```
SELECT region, product_name, SUM(sales_amount) FROM sales GROUP BY region, product_name;
```

- **HAVING:** 用于对 GROUP BY 分组后的结果进行过滤。它类似于 WHERE，但 WHERE 过滤的是原始行，而 HAVING 过滤的是组。
 - HAVING 通常用于过滤那些基于聚合函数结果的组。

- **举例:**

- 找到总订单金额大于 5000 的客户:

SQL

```
SELECT customer_id, SUM(amount) FROM orders GROUP BY customer_id HAVING SUM(amount) > 5000;
```

解释: 先按 customer_id 分组计算每个客户的总金额，然后 HAVING 过滤掉那些总金额不大于 5000 的组。

- 找到客户数量大于 10 个的城市:

奕道远信息

SQL

```
SELECT city, COUNT(*)
FROM customers
GROUP BY city
HAVING COUNT(*) > 10;
```

总结执行顺序（简化版）：FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT/TOP。理解这个顺序有助于理解为什么 WHERE 不能用聚合函数，而 HAVING 可以。

4. 多表连接 (Joins) 详解：

连接用于将来自两个或多个相关表的数据组合起来。连接基于表之间共同的列（通常是一个表的外键引用另一个表的主键）。

- **理解连接的必要性：**数据被分解到不同的表中（规范化）是为了减少冗余和提高数据完整性。例如，客户信息和订单信息分开存储。当你需要查看某个订单是由哪个客户下的，或者某个客户下了哪些订单时，就需要将 orders 表和 customers 表连接起来。
- **INNER JOIN：**返回两个表中都满足连接条件的行。这是最常见的连接类型。
 - **举例：**获取每个订单的订单 ID 以及对应的客户姓名：

SQL

```
SELECT o.order_id, c.customer_name
FROM orders o -- 为 orders 表起别名 o
INNER JOIN customers c ON o.customer_id = c.customer_id;
-- 为 customers 表起别名 c，并指定连接条件
```

*解释：*它会找到 orders 表和 customers 表中 customer_id 相等的行，然后将这些匹配的行的指定列组合在一起。如果某个订单的 customer_id 在 customers 表中找不到对应的客户，这个订单就不会出现在结果中。

- **LEFT JOIN (或 LEFT OUTER JOIN)：**返回 左表 (FROM 子句中先写的表) 的所有行，以及右表中满足连接条件的匹配行。如果左表的某一行在右表中没有匹配，右表的列将显示 NULL。
 - **举例：**获取所有客户的姓名，以及他们下过的订单 ID（如果他们下过订单的话）：

SQL

奕道远信息

```
SELECT c.customer_name, o.order_id
FROM customers c -- customers 是左表
LEFT JOIN orders o ON c.customer_id = o.customer_id; --
orders 是右表
```

解释：结果会包含 customers 表中的所有客户。对于那些下过订单的客户，会显示他们的订单 ID；对于那些没有下过订单的客户，order_id 列将显示 NULL。

- **RIGHT JOIN (或 RIGHT OUTER JOIN)：**返回 右表 的所有行，以及左表中满足连接条件的匹配行。如果右表的某一行在左表中没有匹配，左表的列将显示 NULL。
 - **举例：**(与 LEFT JOIN 类似，但表顺序相反，实际使用中较少直接用 RIGHT JOIN，通常倾向于使用 LEFT JOIN 并调整 FROM 后表的顺序)

SQL

```
SELECT c.customer_name, o.order_id
FROM customers c -- customers 是左表
RIGHT JOIN orders o ON c.customer_id = o.customer_id; --
orders 是右表
```

解释：结果会包含 orders 表中的所有订单。对于在 customers 表中找到对应客户的订单，会显示客户姓名；如果某个订单的 customer_id 在 customers 表中不存在（这通常意味着数据有问题），customer_name 列将显示 NULL。

- **FULL OUTER JOIN：**返回当使用 LEFT JOIN 和 RIGHT JOIN 时，所有未被匹配的行。即，返回两个表中的所有行，无论是否匹配。不匹配的行在另一表的对应列将显示 NULL。
 - **举例：**获取所有客户和所有订单的组合，无论它们是否匹配。

SQL

```
SELECT c.customer_name, o.order_id
FROM customers c
FULL OUTER JOIN orders o ON c.customer_id = o.customer_id;
```

解释：结果会包含：匹配的客户和订单；没有订单的客户（订单 ID 为 NULL）；没有对应客户的订单（客户姓名 NULL）。某些数据库系统（如 MySQL 旧版本）不支持 FULL OUTER JOIN，但可以通过 UNION 来模拟实现 LEFT JOIN 和 RIGHT JOIN 的结果。

奕道远信息

- **ON 子句：**用于指定连接两个表时，用来匹配行的条件。通常是 表 1. 列名 = 表 2. 列名。
 - **举例：**（已包含在上述连接类型的例子中）

5. 子查询（Subqueries）详解：

子查询是一个嵌套在另一个 SQL 语句中的 SELECT 语句。内层子查询先执行，其结果被外层查询使用。子查询必须用括号（）括起来。

- **理解嵌套查询的概念：**将一个查询的结果作为另一个查询的输入或条件。
- **在 WHERE 子句中使用子查询：**最常见的用法，将子查询结果作为过滤条件。
 - **举例：**找到价格高于平均价格的所有产品：

SQL

```
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products); -- 内层
子查询计算平均价格
```

- **举例：**找到下过订单的客户的姓名（使用 IN）：

SQL

```
SELECT customer_name
FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders); --
内层子查询获取所有下过订单的客户 ID 列表
```

- **在 SELECT 子句中使用子查询：**通常用作**标量子查询**，即子查询只返回一个行和一个列的值。
 - **举例：**获取每个订单的订单 ID，以及所有订单的总金额（总金额对每行订单都重复显示）：

SQL

```
SELECT order_id, (SELECT SUM(amount) FROM orders) AS
total_orders_amount
FROM orders;
```

注意：这种用法较少见，除非需要将一个汇总值与每条明细记录一起显示。

- **在 FROM 子句中使用子查询：** 子查询的结果被视为一个临时表（也称为派生表或内联视图），外层查询再从这个临时表中查询。**在 FROM 子句中的子查询必须有别名。**
 - **举例：**找到总销售额大于 100000 的区域：

SQL

```
SELECT region, total_sales
FROM (
    SELECT region, SUM(sales_amount) AS total_sales -- 内
    层子查询计算每个区域的总销售额
    FROM sales
    GROUP BY region
) AS RegionSales -- 给内层子查询结果起的别名
WHERE total_sales > 100000; -- 外层查询从 RegionSales 这
个临时表中筛选
```

这个例子也可以用 GROUP BY 和 HAVING 实现，但使用 FROM 子查询有时能让逻辑更清晰，特别是在多步计算时。

6. (进阶选学) 简述：

这些是 SQL 中更高级的功能，能解决更复杂的问题，但初学阶段可以先了解概念，后续深入学习。

- **UNION 和 UNION ALL：**合并两个或多个 SELECT 查询的结果集。UNION 会去除重复行，UNION ALL 会保留所有行（包括重复的）。要求合并的查询列数和对应数据类型兼容。
 - **用处：**合并来自不同表或不同条件的相似结构的数据。
- **CASE 语句：**在查询中实现条件逻辑（IF-THEN-ELSE）。根据不同的条件返回不同的值。
 - **用处：**对数据进行分类、转换格式或计算新的值。
- **窗口函数 (Window Functions)：**在与当前行相关的“窗口”行集上执行计算。与 GROUP BY 的聚合函数不同，窗口函数不会将行折叠成单个汇总行，而是为每一行返回一个结果。
 - **常用函数：**ROW_NUMBER(), RANK(), DENSE_RANK() (排名), SUM() OVER(...), AVG() OVER(...) (窗口聚合), LAG(), LEAD() (访问前/后行的值)。
 - **用处：**计算排名、累计总额、移动平均、比较当前行与前/后行的值等。非常强大的分析工具。
- **CTE (Common Table Expressions - WITH 语句)：**提供一种方法来定义一个临时、命名的结果集，可以在一个 SELECT, INSERT, UPDATE, DELETE 或另一个 CTE 中引用它。
 - **用处：**简化复杂的查询，提高可读性；用于递归查询。

7. 实践环境：

掌握 SQL 最关键的是**实践**。

- **在线平台：**
 - LeetCode 数据库题目：适合练习各种查询技巧，特别是面试准备。
 - HackerRank SQL：提供不同难度级别的 SQL 练习题。
 - SQLZoo：很好的互动式教程，从基础到进阶。
 - Mode Analytics SQL Tutorial：包含教程和交互式练习。
- **本地数据库：**
 - **SQLite**：一个轻量级的、基于文件的数据库，无需复杂的安装和配置，非常适合个人学习和练习。下载一个 SQLite 客户端工具即可使用。
 - PostgreSQL / MySQL：更强大的客户端-服务器数据库系统，需要一定的安装配置，但更接近实际企业环境。

选择一个或多个环境，从简单的 `SELECT * FROM table;` 开始，一步步练习 WHERE, GROUP BY, JOIN 等，不断尝试解决实际的数据提取问题。

三、 统计学基础 (数据分析的理论支撑)

目标：理解核心统计概念，为数据解释和后续分析打下基础。重点在于理解概念而非复杂计算。

核心概念：

统计学 是一门关于数据收集、组织、分析、解释和呈现的科学。在数据分析中，统计学提供了理解数据特性、找出模式、以及基于数据做出可靠结论和预测的方法和工具。

数据分析 是一个更广泛的过程，它利用统计学等工具和技术从数据中提取有用的信息和洞见，以支持决策。可以说，统计学是数据分析的重要理论和方法基础。

学习内容详解与举例：

1. 什么是数据分析与统计学？

- **数据分析的目标和作用：**
 - **目标：**从原始数据中发现有价值的信息、识别趋势、找出异常、验证假设，并最终为解决问题或制定策略提供依据。

奕道远信息

- **作用:** 帮助我们理解发生了什么（描述性分析）、为什么会发生（诊断性分析）、预测未来可能发生什么（预测性分析），以及如何让事情朝着期望的方向发展（规范性分析）。简而言之，数据分析让数据“说话”，从而做出更明智的决策。
- **统计学在数据分析中的角色:**
 - 提供描述和概括数据的工具（如均值、标准差）。
 - 提供可视化数据的手段（如直方图、箱线图）。
 - 提供基于样本推断总体的理论和方法（如假设检验、置信区间）。
 - 提供建模数据之间关系的方法（如回归分析）。
 - 帮助我们理解数据中的变异性和不确定性。

2. 数据类型详解:

理解数据类型是进行正确统计分析的第一步。

- **定性数据 (Qualitative Data) / 分类数据 (Categorical Data):** 描述事物的品质、特征或类别，通常是非数值的，或者数值本身没有数学意义（如身份证号、邮编）。
 - **名义数据 (Nominal):** 类别之间没有顺序或等级之分。
 - **举例:** 性别（男/女）、血型（A/B/AB/O）、城市名称（北京/上海/广州）、产品颜色（红/蓝/绿）。
 - **等级数据 (Ordinal):** 类别之间有明显的顺序或等级关系，但类别之间的差值没有固定的或可量化的意义。
 - **举例:** 教育程度（小学/初中/高中/大学）、客户满意度（不满意/一般/满意/非常满意）、比赛排名（第一名/第二名/第三名）。
- **定量数据 (Quantitative Data) / 数值数据 (Numerical Data):** 描述事物的数量，是数值形式的，且数值具有数学意义，可以进行加减乘除等运算。
 - **离散数据 (Discrete):** 只能取有限个或可数个特定值，通常是整数，值之间有明确的间隔。
 - **举例:** 家庭人口数、销售的商品数量、班级人数、网站访问次数。
 - **连续数据 (Continuous):** 可以在某一区间内取任意数值，值之间没有明确的间隔，精度取决于测量工具。
 - **举例:** 身高、体重、温度、销售额、时间、产品价格。

3. 描述性统计 (Descriptive Statistics) 详解:

描述性统计用于概括和描述数据集的主要特征，帮助我们快速了解数据的全貌。

- **集中趋势度量 (Measures of Central Tendency):** 描述数据的中心位置。
 - **均值 (Mean) / 平均数:** 所有数据点之和除以数据点的数量。是最常见的集中趋势度量。

- **计算方法:** $\bar{x} = \frac{1}{n} \sum x_i$
 - **适用场景:** 适用于对称分布的定量数据。
 - **缺点:** 对极端值（异常值）非常敏感。
 - **举例:** 销售额数据 [100, 120, 150, 130, 110] 的均值是 $(100+120+150+130+110)/5 = 122$ 。如果数据是 [100, 120, 150, 130, 110, 1000]，均值会大幅提高。
- **中位数 (Median):** 将数据按大小排序后位于最中间的值。如果数据点数量是偶数，则是中间两个数的平均值。
 - **适用场景:** 适用于有等级顺序或定量数据，特别是数据分布偏斜或存在极端值时，中位数比均值更能代表“典型”值。
 - **举例:** 销售额数据 [100, 110, 120, 130, 150] 的中位数是 120。如果数据是 [100, 110, 120, 130, 150, 1000]，排序后 [100, 110, 120, 130, 150, 1000]，中位数是 $(120+130)/2 = 125$ 。中位数受极端值影响较小。
- **众数 (Mode):** 数据集中出现次数最多的值。
 - **适用场景:** 适用于任何类型的数据（名义、等级、定量），特别是分类数据。一个数据集可以有多个众数或没有众数。
 - **举例:** 产品颜色数据 [红, 蓝, 红, 绿, 红, 蓝] 的众数是“红”。销售额数据 [100, 120, 100, 150, 120, 100] 的众数是 100。
- **离散程度度量 (Measures of Dispersion/Variability):** 描述数据分布的离散程度或波动性。
 - **极差 (Range):** 最大值与最小值之差。
 - **计算方法:** 最大值 - 最小值
 - **优点:** 计算简单。
 - **缺点:** 只利用了两个极端值，对异常值非常敏感，不能反映中间数据的离散程度。
 - **举例:** 销售额数据 [100, 120, 150, 130, 110] 的极差是 $150 - 100 = 50$ 。数据 [100, 120, 150, 130, 110, 1000] 的极差是 $1000 - 100 = 900$ 。
 - **四分位数 (Quartiles) 与四分位距 (IQR - Interquartile Range):** 四分位数将排序后的数据分为四等份。Q1 是第 25 百分位数，Q2 是第 50 百分位数（中位数），Q3 是第 75 百分位数。
 - **四分位距 (IQR):** $Q3 - Q1$ 。表示中间 50% 数据的分布范围。
 - **适用场景:** 适用于定量数据，特别在数据偏斜时比标准差更稳健，常用于检测异常值（通常将小于 $Q1 - 1.5 \times IQR$ 或大于 $Q3 + 1.5 \times IQR$ 的值视为异常值）。
 - **举例:** 数据 [10, 20, 30, 40, 50, 60, 70, 80]。Q1 = 25, Q2 = 45, Q3 = 65。IQR = $65 - 25 = 40$ 。
 - **方差 (Variance):** 衡量所有数据点与其均值的平均离散程度。是每个数据点与均值之差的平方的平均数。
 - **计算方法 (样本方差):** $s^2 = \frac{1}{n-1} \sum (x_i - \bar{x})^2$ （除以 n-1 而不是 n 是为了无偏估计总体方差）
 - **优点:** 考虑了所有数据点的离散程度。

- **缺点:** 单位是原始数据的平方，解释性不强。
- **举例:** 数据 [10, 20, 30]。均值 = 20。方差 = $((10-20)^2 + (20-20)^2 + (30-20)^2) / (3-1) = (100+0+100)/2=100$ 。
- **标准差 (Standard Deviation):** 方差的平方根。与原始数据具有相同的单位，是最常用的离散程度度量。
 - **计算方法 (样本标准差):** $s = \sqrt{s^2} = \sqrt{\frac{1}{n-1} \sum (x_i - \bar{x})^2}$
 - **优点:** 易于理解和解释，反映了数据点平均偏离均值的程度。标准差越大，数据越分散；标准差越小，数据越集中在均值附近。
 - **举例:** 基于上方方差的例子，标准差 = $\sqrt{100} = 10$ 。这表示数据点平均偏离均值 20 的距离大约是 10。
- **分布形态 (Shape of Distribution):** 描述数据分布的形状。
 - **频数分布 (Frequency Distribution):** 显示数据集中各个值或各个区间的值出现的次数。
 - **直方图 (Histogram):** 频数分布的图形表示，用柱子的面积表示落在每个区间内的数据的频率。
 - **正态分布 (Normal Distribution) / 钟形曲线 (Bell Curve):** 一种非常重要的对称分布，形似钟形。数据的均值、中位数和众数大致相等并位于中心。许多自然现象和社会现象（如身高、考试分数）近似服从正态分布。它的数学性质在推断统计中非常重要。
 - **偏度 (Skewness):** 衡量数据分布的对称性。
 - **左偏 / 负偏 (Left Skewed / Negative Skew):** 尾部在左边更长，数据集中在右侧。通常 均值 < 中位数 < 众数。
 - **右偏 / 正偏 (Right Skewed / Positive Skew):** 尾部在右边更长，数据集中在左侧。通常 众数 < 中位数 < 均值。
 - **无偏 / 对称 (Symmetric):** 两侧尾部大致对称，如正态分布。通常 均值 $\approx$ 中位数 $\approx$ 众数。
 - **举例:** 收入分布通常是右偏的（少数高收入者拉高了平均值）。考试分数分布可能左偏（大多数人考得不错）。
 - **峰度 (Kurtosis):** 衡量数据分布的“尖锐”或“平阔”程度，以及尾部的厚度，与正态分布比较。高峰度表示数据更集中在中心，尾部更厚（有更多极端值）；低峰度表示数据较平坦，尾部较薄。
- **概念: 百分位数 (Percentiles):** 将一组数据从小到大排序后，将数据分为 100 等份。第 P 百分位数表示有 P% 的数据值小于或等于这个值。
 - **举例:** 如果你的考试分数是 80 分，处于第 90 百分位数，意味着 80 分这个分数高于 90% 的考生分数。中位数是第 50 百分位数。

4. 概率基础（概念理解）详解：

概率是理解推断统计的基础，但在这里只需理解基本概念。

- **随机事件与概率：**
 - **随机事件：**在给定条件下，可能发生也可能不发生的结果。
 - **概率：**衡量随机事件发生的可能性大小的数值，范围在 0 到 1 之间（或 0% 到 100%）。0 表示不可能发生，1 表示必然发生。
 - **举例：**抛硬币正面朝上是一个随机事件，其概率是 0.5。扔骰子得到 6 是一个随机事件，其概率是 1/6。
- **条件概率（初步了解）：**在另一个事件已经发生的情况下，某个事件发生的概率。
 - **举例：**已知今天多云，明天可能下雨的概率（这是一个条件概率）。

5. 相关性与因果关系 详解：

这是数据分析中最容易混淆和误解的概念之一。

- **理解相关性 (Correlation) 的概念：**衡量两个变量之间线性关联的**强度和方向**。它只是描述了两个变量变化的同步程度，并不涉及谁导致了谁。
 - **皮尔逊相关系数 (Pearson Correlation Coefficient)：**最常用的衡量线性相关性的指标，取值范围在 -1 到 +1 之间。
 - +1 表示完全正线性相关（一个变量增加时，另一个变量也等比例增加）。
 - -1 表示完全负线性相关（一个变量增加时，另一个变量等比例减少）。
 - 0 表示没有线性相关性（可能存在非线性关系或其他关系）。
 - **举例：**身高和体重通常是正相关的。汽车速度和行驶时间（固定距离）是负相关的。鞋码和智商通常没有相关性。
- **重要：区分相关性不等于因果关系 (Correlation does not imply causation)：**仅仅观察到两个变量一起变化（相关）并不能得出其中一个导致另一个变化的结论（因果）。
 - 可能存在**混淆变量 (Confounding Variable)**，它同时影响了这两个变量。
 - 可能仅仅是**巧合**。
 - 可能因果关系是**反向的**。
 - **举例：**
 - 夏季冰淇淋销量和溺水事件数量高度相关。原因：夏天天气热（混淆变量），导致冰淇淋销量增加，也导致更多人去游泳从而增加了溺水风险。冰淇淋不导致溺水。
 - 研究发现，经常使用某个 APP 的用户学习成绩更好。可能的原因：学习成绩好的学生更自律，同时也更倾向于使用这类有助于学习的 APP。APP 本身不一定是成绩提高的唯一或主要原因。

6. 推断性统计入门（概念理解） 详解：

推断性统计使用样本数据来对更广泛的总体做出结论或预测。我们分析样本的目的是为了了解总体，而不是仅仅描述样本本身。

- **总体 (Population) 与样本 (Sample) 的概念:**
 - **总体:** 是你研究兴趣的所有个体或事物构成的完整集合。例如, 如果你想研究全中国的成年男性平均身高, 总体就是全中国的成年男性。
 - **样本:** 从总体中抽取的一个子集。由于研究总体通常不现实 (成本高、时间长、不可能), 我们通常抽取一个具有代表性的样本进行研究。例如, 从全中国的成年男性中随机抽取 1000 人测量身高。
 - **挑战:** 如何从样本的特征 (**样本统计量**, 如样本均值) 来推断总体的特征 (**总体参数**, 如总体均值), 并评估这种推断的可靠性。
- **假设检验 (Hypothesis Testing):** 一种统计方法, 用于评估关于总体参数的某个断言 (假设) 是否与样本数据相符。
 - **基本思想:**
 - 提出一个 **零假设 (Null Hypothesis, H0)**: 通常是一个没有效应、没有差异或没有关系的陈述。例如, H_0 : 成年男性平均身高 = 175cm。
 - 提出一个 **备择假设 (Alternative Hypothesis, H1)**: 通常是零假设的对立面, 是你希望找到证据支持的陈述。例如, H_1 : 成年男性平均身高 $\neq$ 175cm。
 - 收集样本数据。
 - 进行统计检验, 计算样本数据在零假设为真时出现的概率。
 - 根据这个概率 (P 值) 来决定是拒绝零假设还是不拒绝零假设。
 - **P 值的含义 (Meaning of P-value):** 是在零假设为真的前提下, 观察到当前样本结果或更极端结果的概率。
 - **小的 P 值 (通常 ≤ 0.05):** 意味着如果零假设是真的, 我们得到当前样本结果的可能性非常小。这提供了**有力证据**反对零假设, 因此我们**拒绝 H0**, 转而支持备择假设。
 - **大的 P 值 (> 0.05):** 意味着如果零假设是真的, 我们得到当前样本结果的可能性较大。这说明我们**没有足够的证据**拒绝零假设, 因此我们**不拒绝 H0** (注意: 不拒绝 H0 不等于接受 H0 为真, 只是说当前数据不足以推翻它)。
 - **举例:** 假设检验结果 P 值为 0.03。这意味着如果成年男性平均身高确实是 175cm (H_0 为真), 我们抽取样本得到当前平均身高 (或更极端值) 的概率只有 3%。由于 3% 远小于常用的 5% 显著性水平, 我们认为这个结果在 H_0 为真时不太可能发生, 因此我们拒绝 H_0 , 得出“有统计学证据表明成年男性平均身高不等于 175cm”的结论。
- **置信区间 (Confidence Interval):** 是一个根据样本数据计算出的区间, 用来估计总体参数 (如总体均值或总体比例) 的可能范围。它提供了一个估计的**精度**概念。
 - **了解其含义:** 一个 95% 置信区间意味着, 如果我们从总体中重复抽取大量随机样本并计算每个样本的置信区间, 那么大约有 95% 的这些区间会包含真实的总体参数值。它量化了我们对样本估计值的信心程度。
 - **举例:** 通过样本数据计算得到某 APP 用户平均每天使用时长的 95% 置信区间是 [45 分钟, 55 分钟]。这意味着我们有 95% 的信心认为, 所有该 APP 用户的真实平均每天使用时长介于 45 分钟到 55 分钟之间。

7. (入门选学): 线性回归 (Linear Regression) 的基本概念:

- **基本概念:** 是一种建模一个**因变量**（你想要预测或解释的变量）与一个或多个**自变量**（用来预测或解释因变量的变量）之间**线性关系**的统计方法。
 - 最简单的是**简单线性回归**（一个自变量），模型可以表示为一条直线方程：

$$y = a + bx + \epsilon$$
 - y: 因变量 (Dependent Variable)
 - x: 自变量 (Independent Variable)
 - a: 截距 (Intercept), 当 $x=0$ 时 y 的期望值
 - b: 斜率 (Slope), x 每增加一个单位时 y 的期望变化量
 - ϵ : 误差项 (Error Term), 模型未能解释的部分
 - **用处:**
 - 预测: 根据自变量的值预测因变量的值。
 - 解释: 理解自变量对因变量的影响方向和强度（通过斜率 b 的正负和大小）。
 - **举例:**
 - 分析广告投入（自变量）对销售额（因变量）的影响，并预测在特定广告投入下的销售额。
 - 分析工作年限（自变量）对薪资（因变量）的影响。
 - 分析房屋面积（自变量）对房价（因变量）的影响。

总结统计学基础:

统计学为我们提供了理解数据、描述数据、以及基于有限数据做出可靠结论的科学方法。描述性统计帮助我们认识数据的基本特征，推断性统计则帮助我们超越样本本身，对总体得出有意义的结论。理解数据类型、集中趋势、离散程度、分布形态、相关性与因果的区别以及假设检验和置信区间的基本思想，将为你进行有效的数据分析奠定坚实的基础。

四、 SQL 应用举例

本节主要介绍 SQL 中最基础也是最重要的概念和应用。通过实际运行这些代码，观察输入数据和输出结果，并对照解释，将能更深入地理解每个语句的作用。将使用几个简单的假设表来进行演示：customers（客户表），orders（订单表），products（产品表）。

1. 创建 customers（客户）表

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY, -- 客户 ID, 整数, 主键 (唯一标识每一行, 非空)
    customer_name VARCHAR(50),    -- 客户姓名, 最大 50 字符的可变字符串
    city VARCHAR(50),             -- 客户所在城市, 最大 50 字符的可变字符串
    join_date DATE                 -- 客户加入日期, 日期类型
```

```
);
```

```
-- -----  
-- 插入示例数据到 customers 表  
-- -----
```

```
INSERT INTO customers (customer_id, customer_name, city, join_date)  
VALUES  
(1, 'Alice', 'New York', '2022-01-10'),  
(2, 'Bob', 'London', '2022-02-15'),  
(3, 'Charlie', 'New York', '2022-03-20'),  
(4, 'David', 'Paris', '2022-04-01'),  
(5, 'Eve', 'London', '2023-05-10'),  
(6, 'Frank', 'New York', '2023-06-18'),  
(7, 'Grace', 'London', '2023-07-22');
```

```
-- -----  
-- 2. 创建 orders (订单) 表  
-- -----
```

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,    -- 订单 ID, 整数, 主键  
    customer_id INT,            -- 客户 ID, 整数 (外键, 将关联  
customers 表)  
    order_date DATE,            -- 订单日期, 日期类型  
    amount DECIMAL(10, 2),      -- 订单总金额, 精确小数类型 (总共最多  
10 位, 小数点后 2 位)  
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id) -- 定  
义 customer_id 是外键, 引用 customers 表的 customer_id  
);
```

```
-- -----  
-- 插入示例数据到 orders 表  
-- -----
```

```
INSERT INTO orders (order_id, customer_id, order_date, amount) VALUES  
(101, 1, '2023-01-15', 150.00),  
(102, 3, '2023-01-16', 250.50),  
(103, 1, '2023-01-17', 50.00),  
(104, 2, '2023-02-20', 300.00),  
(105, 5, '2023-05-15', 120.00),  
(106, 1, '2023-06-01', 450.75),  
(107, 3, '2023-06-05', 80.00),  
(108, 2, '2023-06-10', 180.00),  
(109, 5, '2023-06-20', 90.00),  
(110, 7, '2023-07-25', 210.00),
```

```
(111, 1, '2023-07-30', 600.00),
(112, 3, '2023-08-01', 110.00),
(113, 2, '2023-08-05', 220.00),
(114, 5, '2023-08-10', 150.00),
(115, 7, '2023-08-15', 350.00);
-- 注意：客户 ID 为 4 (David) 和 6 (Frank) 没有对应的订单记录，这将在
LEFT JOIN 示例中用到
```

```
-- -----
-- 3. 创建 products (产品) 表
-- -----
```

```
CREATE TABLE products (
    product_id INT PRIMARY KEY, -- 产品 ID, 整数, 主键
    product_name VARCHAR(50),    -- 产品名称, 最大 50 字符的可变字符串
    category VARCHAR(50),        -- 产品类别, 最大 50 字符的可变字符串
    price DECIMAL(10, 2),        -- 产品价格, 精确小数类型
    stock INT                    -- 库存数量, 整数
);
```

```
-- -----
-- 插入示例数据到 products 表
-- -----
```

```
INSERT INTO products (product_id, product_name, category, price,
stock) VALUES
(1, 'Laptop', 'Electronics', 800.00, 15),
(2, 'Keyboard', 'Electronics', 75.00, 50),
(3, 'Mouse', 'Electronics', 25.00, 100),
(4, 'SQL Book', 'Books', 45.00, 30),
(5, 'Python Book', 'Books', 60.00, 25),
(6, 'Desk Chair', 'Furniture', 150.00, 10),
(7, 'Desk Lamp', 'Furniture', 35.00, 40),
(8, 'Monitor', 'Electronics', 300.00, 20),
(9, 'Novel', 'Books', 15.00, 80),
(10, 'Headphones', 'Electronics', 90.00, 35);
```

执行上面的 SQL 代码来创建表并插入数据，然后你可以使用下面的查询代码进行练习。

1. 查询表中的所有数据

- 应用案例：查看 customers 表的完整内容。

- **需求：**获取 customers 表中的所有列和所有行。
- **查询代码：**

SQL

```
SELECT
    * -- 选择所有列
FROM
    customers; -- 从 customers 表中查询
```

- **预期输出：**

customer_id	customer_name	city	join_date
1	Alice	New York	2022-01-10
2	Bob	London	2022-02-15
3	Charlie	New York	2022-03-20
4	David	Paris	2022-04-01
5	Eve	London	2023-05-10
6	Frank	New York	2023-06-18
7	Grace	London	2023-07-22

- **应用场景：**快速预览整个表的数据。

2. 查询特定列并按条件筛选行

- **应用案例：**找出所有居住在 'London' 的客户的姓名和加入日期。
- **需求：**从 customers 表中筛选出 city 为 'London' 的行，并仅返回 customer_name 和 join_date 列。
- **查询代码：**

SQL

```
SELECT
    customer_name, -- 选择 customer_name 列
    join_date      -- 选择 join_date 列
FROM
    customers      -- 从 customers 表中查询
WHERE
    city = 'London'; -- 设置过滤条件，只保留 city 列的值等于
                    'London' 的行
```

- **预期输出：**
- customer_name | join_date

奕道远信息

- -----|-----
 - Bob | 2022-02-15
 - Eve | 2023-05-10
 - Grace | 2023-07-22
 - **应用场景：**提取符合特定标准的记录，例如查找某个地区、某个时间段、某个状态的数据。
-

3. 对查询结果进行排序

- **应用案例：**按产品价格从高到低列出所有产品。
- **需求：**获取 products 表中的所有列，按 price 降序排列。
- **查询代码：**

SQL

```
SELECT
    * -- 选择所有列
FROM
    products -- 从 products 表中查询
ORDER BY
    price DESC; -- 按 price 列进行排序，使用 DESC 关键字表示降序
```

- **预期输出：**（只显示前几行，实际输出包含所有产品按价格排序）
 - | product_id | product_name | category | price | stock |
|------------|--------------|-------------|--------|-------|
| 1 | Laptop | Electronics | 800.00 | 15 |
| 8 | Monitor | Electronics | 300.00 | 20 |
| 6 | Desk Chair | Furniture | 150.00 | 10 |
| 10 | Headphones | Electronics | 90.00 | 35 |
| ... | ... | ... | ... | ... |
 - **应用场景：**排行榜、按时间顺序/倒序查看事件、按字母顺序排列列表等。
-

4. 计算总行数

- **应用案例：**统计 orders 表中共有多少条订单记录。
- **需求：**计算 orders 表的总行数。
- **查询代码：**

奕道远信息

SQL

```
SELECT
    COUNT(*)          -- 使用 COUNT(*) 聚合函数计算表中的总行数
FROM
    orders;          -- 从 orders 表中查询
```

- **预期输出:**
 - count
 - -----
 - 15
 - **应用场景:** 快速获取数据表的总记录量、统计符合特定条件的记录数量（结合 WHERE）。
-

5. 计算数值列的总和、平均值、最大值、最小值

- **应用案例:** 计算所有订单的总金额、平均订单金额、单笔最高订单金额和单笔最低订单金额。
- **需求:** 对 orders 表的 amount 列进行聚合计算。
- **查询代码:**

SQL

```
SELECT
    SUM(amount) AS TotalAmount,      -- 计算 amount 列的总和，并
    命名为 TotalAmount
    AVG(amount) AS AverageAmount,    -- 计算 amount 列的平均值，
    并命名为 AverageAmount
    MAX(amount) AS MaxAmount,        -- 计算 amount 列的最大值，
    并命名为 MaxAmount
    MIN(amount) AS MinAmount         -- 计算 amount 列的最小值，
    并命名为 MinAmount
FROM
    orders;                          -- 从 orders 表中查询
```

- **预期输出:**
- TotalAmount | AverageAmount | MaxAmount | MinAmount
- -----|-----|-----|-----
- 2931.25 | 195.416666 | 600.00 | 50.00
- **应用场景:** 计算总销售额、平均成绩、最高温度、最低库存等，对整体数据集进行汇总描述。

奕道远信息

6. 按分组对数据进行聚合

- **应用案例：**统计每个客户的总订单金额。
- **需求：**按 `customer_id` 分组，计算每个分组下的 `amount` 总和。
- **查询代码：**

SQL

```
SELECT
    customer_id,          -- 选择用于分组的列: customer_id
    SUM(amount) AS CustomerTotalAmount -- 计算每个分组 (客户)
    的 amount 总和, 并命名为 CustomerTotalAmount
FROM
    orders                -- 从 orders 表中查询
GROUP BY
    customer_id;          -- 按 customer_id 列进行分组
```

- **预期输出：**
- | customer_id | CustomerTotalAmount |
|-------------|---------------------|
| 1 | 1250.75 |
| 2 | 700.00 |
| 3 | 440.50 |
| 5 | 360.00 |
| 7 | 560.00 |
- **应用场景：**统计各区域销售额、各产品类别库存量、各部门员工数等，分析不同分组的汇总情况。

7. 对分组后的结果进行过滤

- **应用案例：**找出总订单金额大于 500 的客户及其总金额。
- **需求：**按 `customer_id` 分组计算总订单金额，然后只保留总金额大于 500 的分组。
- **查询代码：**

SQL

```
SELECT
    customer_id,          -- 选择客户 ID
```

变道远信息

```

SUM(amount) AS CustomerTotalAmount -- 计算客户总订单金额
FROM
    orders -- 从 orders 表中查询
GROUP BY
    customer_id -- 按客户 ID 分组
HAVING
    SUM(amount) > 500; -- 使用 HAVING 过滤分组，只保留总金额大于 500 的分组

```

- **预期输出：**

customer_id	CustomerTotalAmount
1	1250.75
2	700.00
7	560.00

- **应用场景：**筛选出销售业绩超过目标的分组、找出问题比例高于阈值的产品批次等，对聚合结果进行筛选。

8. 使用 INNER JOIN 连接两个表

- **应用案例：**查看每个订单是由哪个客户下的，获取订单 ID、订单日期、订单金额以及对应的客户姓名。
- **需求：**连接 orders 表和 customers 表，基于 customer_id 匹配，并返回指定的列。
- **查询代码：**

SQL

```

SELECT
    o.order_id, -- 选择 orders 表的 order_id (使用别名 o)
    o.order_date, -- 选择 orders 表的 order_date
    o.amount, -- 选择 orders 表的 amount
    c.customer_name -- 选择 customers 表的 customer_name (使用别名 c)
FROM
    orders o -- 从 orders 表查询，并使用别名 o
INNER JOIN
    customers c ON o.customer_id = c.customer_id; -- 与 customers 表 (别名 c) 进行内连接，连接条件是 o.customer_id 等于 c.customer_id

```

- **预期输出：**(只显示部分结果)

奕道远信息

• order_id	order_date	amount	customer_name
• -----	-----	-----	-----
• 101	2023-01-15	150.00	Alice
• 102	2023-01-16	250.50	Charlie
• 103	2023-01-17	50.00	Alice
• 104	2023-02-20	300.00	Bob
• ...	...	...	...

- **应用场景：**合并来自不同表的关联数据，例如订单与客户信息、产品与类别信息、员工与部门信息等。只返回两个表都有匹配的数据。

9. 使用 LEFT JOIN 连接两个表

- **应用案例：**获取所有客户的列表，并显示他们下过的订单 ID。即使客户没有下过订单，也应出现在列表中。
- **需求：**连接 customers 表（左表）和 orders 表（右表），基于 customer_id 匹配。返回所有客户信息，并附加匹配的订单信息。
- **查询代码：**

SQL

```
SELECT
    c.customer_id,    -- 选择 customers 表的 customer_id (左表所有行都会保留)
    c.customer_name, -- 选择 customers 表的 customer_name
    o.order_id,       -- 选择 orders 表的 order_id (如果没有匹配的订单，此处显示 NULL)
    o.order_date,     -- 选择 orders 表的 order_date (如果没有匹配的订单，此处显示 NULL)
    o.amount          -- 选择 orders 表的 amount (如果没有匹配的订单，此处显示 NULL)
FROM
    customers c        -- 从 customers 表查询 (作为左表)
LEFT JOIN
    orders o ON c.customer_id = o.customer_id; -- 与 orders 表 (作为右表) 进行左连接，连接条件是 c.customer_id 等于 o.customer_id
```

- **预期输出：**(注意客户 David 和 Frank)

• customer_id	customer_name	order_id	order_date	amount
• -----	-----	-----	-----	-----
• 1	Alice	101	2023-01-15	150.00
• 1	Alice	103	2023-01-17	50.00

- | | | | | |
|-----|---------|------|------------|--------|
| • 1 | Alice | 106 | 2023-06-01 | 450.75 |
| • 1 | Alice | 111 | 2023-07-30 | 600.00 |
| • 2 | Bob | 104 | 2023-02-20 | 300.00 |
| • 2 | Bob | 108 | 2023-06-10 | 180.00 |
| • 2 | Bob | 113 | 2023-08-05 | 220.00 |
| • 3 | Charlie | 102 | 2023-01-16 | 250.50 |
| • 3 | Charlie | 107 | 2023-06-05 | 80.00 |
| • 3 | Charlie | 112 | 2023-08-01 | 110.00 |
| • 4 | David | NULL | NULL | NULL |
| • 5 | Eve | 105 | 2023-05-15 | 120.00 |
| • 5 | Eve | 109 | 2023-06-20 | 90.00 |
| • 5 | Eve | 114 | 2023-08-10 | 150.00 |
| • 6 | Frank | NULL | NULL | NULL |
| • 7 | Grace | 110 | 2023-07-25 | 210.00 |
| • 7 | Grace | 115 | 2023-08-15 | 350.00 |
- **应用场景：**查找没有子项的父项（如没有订单的客户）、保留主表所有信息并附加关联表的匹配信息。

10. 使用 CASE 语句进行条件判断

- **应用案例：**根据产品库存数量，判断其库存状态：库存小于等于 10 为 'Low Stock'，在 11 到 50 之间为 'Medium Stock'，大于 50 为 'High Stock'。
- **需求：**获取产品名称、库存数量，并根据库存数量添加一个表示库存状态的列。
- **查询代码：**

SQL

```
SELECT
    product_name,  -- 选择产品名称
    stock,         -- 选择库存数量
    CASE           -- 开始 CASE 语句
        WHEN stock <= 10 THEN 'Low Stock'           -- 当库存小于等于 10 时，结果为 'Low Stock'
        WHEN stock > 10 AND stock <= 50 THEN 'Medium Stock' -- 当库存大于 10 且小于等于 50 时，结果为 'Medium Stock'
        ELSE 'High Stock'                           -- 否则（库存大于 50）结果为 'High Stock'
    END AS StockStatus -- 结束 CASE 语句，并将结果列命名为 'StockStatus'
FROM
```

奕道远信息

```
products;      -- 从 products 表中查询
```

- **预期输出:**

product_name	stock	StockStatus
Laptop	15	Medium Stock
Keyboard	50	Medium Stock
Mouse	100	High Stock
SQL Book	30	Medium Stock
Python Book	25	Medium Stock
Desk Chair	10	Low Stock
Desk Lamp	40	Medium Stock
Monitor	20	Medium Stock
Novel	80	High Stock
Headphones	35	Medium Stock

- **应用场景:** 根据业务规则对数据进行分类或标记、数据转换、在查询结果中创建自定义列。

为了演示一些新的场景（例如处理 NULL 值），我们在 customers 表中新增一条记录。以下是完整的建表和数据插入代码，包含了之前的数据和新增的数据：

SQL

```
-- -----  
-- 1. 创建 customers（客户）表  
-- -----
```

```
DROP TABLE IF EXISTS orders; -- 如果 orders 表存在，先删除（因为  
orders 引用 customers）  
DROP TABLE IF EXISTS customers; -- 如果 customers 表存在，先删除  
DROP TABLE IF EXISTS products; -- 如果 products 表存在，先删除（通常  
先删依赖它的表）
```

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY, -- 客户 ID，整数，主键（唯一标识每一  
    行，非空）  
    customer_name VARCHAR(50),    -- 客户姓名，最大 50 字符的可变字符串  
    city VARCHAR(50),             -- 客户所在城市，最大 50 字符的可变字  
    符串  
    join_date DATE                 -- 客户加入日期，日期类型  
);
```

```
-- -----  
-- 插入示例数据到 customers 表  
-- -----
```

奕道远信息

```
INSERT INTO customers (customer_id, customer_name, city, join_date)
VALUES
(1, 'Alice', 'New York', '2022-01-10'),
(2, 'Bob', 'London', '2022-02-15'),
(3, 'Charlie', 'New York', '2022-03-20'),
(4, 'David', 'Paris', '2022-04-01'),
(5, 'Eve', 'London', '2023-05-10'),
(6, 'Frank', 'New York', '2023-06-18'),
(7, 'Grace', 'London', '2023-07-22'),
(8, 'Wendy', NULL, '2024-01-01'); -- 新增一个城市为 NULL 的客户
```

-- 2. 创建 orders (订单) 表

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY, -- 订单 ID, 整数, 主键
    customer_id INT, -- 客户 ID, 整数 (外键, 将关联
customers 表)
    order_date DATE, -- 订单日期, 日期类型
    amount DECIMAL(10, 2), -- 订单总金额, 精确小数类型 (总共最多
10 位, 小数点后 2 位)
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id) -- 定
义 customer_id 是外键, 引用 customers 表的 customer_id
);
```

-- 插入示例数据到 orders 表

```
INSERT INTO orders (order_id, customer_id, order_date, amount) VALUES
(101, 1, '2023-01-15', 150.00),
(102, 3, '2023-01-16', 250.50),
(103, 1, '2023-01-17', 50.00),
(104, 2, '2023-02-20', 300.00),
(105, 5, '2023-05-15', 120.00),
(106, 1, '2023-06-01', 450.75),
(107, 3, '2023-06-05', 80.00),
(108, 2, '2023-06-10', 180.00),
(109, 5, '2023-06-20', 90.00),
(110, 7, '2023-07-25', 210.00),
(111, 1, '2023-07-30', 600.00),
(112, 3, '2023-08-01', 110.00),
(113, 2, '2023-08-05', 220.00),
(114, 5, '2023-08-10', 150.00),
```

```
(115, 7, '2023-08-15', 350.00);
-- 注意: 客户 ID 为 4 (David) 和 6 (Frank) 以及 8 (Wendy) 没有对应的订单记录
```

```
-- -----
-- 3. 创建 products (产品) 表
-- -----
```

```
CREATE TABLE products (
    product_id INT PRIMARY KEY, -- 产品 ID, 整数, 主键
    product_name VARCHAR(50),    -- 产品名称, 最大 50 字符的可变字符串
    category VARCHAR(50),        -- 产品类别, 最大 50 字符的可变字符串
    price DECIMAL(10, 2),        -- 产品价格, 精确小数类型
    stock INT                    -- 库存数量, 整数
);
```

```
-- -----
-- 插入示例数据到 products 表
-- -----
```

```
INSERT INTO products (product_id, product_name, category, price,
stock) VALUES
(1, 'Laptop', 'Electronics', 800.00, 15),
(2, 'Keyboard', 'Electronics', 75.00, 50),
(3, 'Mouse', 'Electronics', 25.00, 100),
(4, 'SQL Book', 'Books', 45.00, 30),
(5, 'Python Book', 'Books', 60.00, 25),
(6, 'Desk Chair', 'Furniture', 150.00, 10),
(7, 'Desk Lamp', 'Furniture', 35.00, 40),
(8, 'Monitor', 'Electronics', 300.00, 20),
(9, 'Novel', 'Books', 15.00, 80),
(10, 'Headphones', 'Electronics', 90.00, 35);
```

请先执行上面的 SQL 代码来设置数据库环境和数据。

SQL 应用案例 (续 11-30)

11. 使用 AND 组合多个过滤条件

- **应用案例:** 查找居住在 'New York' 且在 '2023 年' 加入的客户。
- **需求:** 从 customers 表中筛选 city 等于 'New York' 并且 join_date 在 2023 年的客户。

- 查询代码:

SQL

```
SELECT
    customer_id, -- 选择客户 ID
    customer_name, -- 选择客户姓名
    city, -- 选择城市
    join_date -- 选择加入日期
FROM
    customers -- 从 customers 表查询
WHERE
    city = 'New York' -- 第一个条件: 城市是 New York
    AND join_date BETWEEN '2023-01-01' AND '2023-12-31'; -- 第
二个条件: 加入日期在 2023 年范围内 (使用 BETWEEN)
```

- 预期输出:
- | customer_id | customer_name | city | join_date |
|-------------|---------------|----------|------------|
| 6 | Frank | New York | 2023-06-18 |
- 应用场景: 同时满足多个业务规则的数据提取。

12. 使用 OR 组合多个过滤条件

- 应用案例: 查找居住在 'London' 或 'Paris' 的客户。
- 需求: 从 customers 表中筛选 city 是 'London' 或者 'Paris' 的客户。
- 查询代码:

SQL

```
SELECT
    customer_id, -- 选择客户 ID
    customer_name, -- 选择客户姓名
    city -- 选择城市
FROM
    customers -- 从 customers 表查询
WHERE
    city = 'London' -- 第一个条件: 城市是 London
    OR city = 'Paris'; -- 第二个条件: 城市是 Paris (使用 OR 关
键字)
```

- 预期输出:

奕道远信息

- | customer_id | customer_name | city |
|-------------|---------------|--------|
| 2 | Bob | London |
| 4 | David | Paris |
| 5 | Eve | London |
| 7 | Grace | London |
- 应用场景：提取满足多个可能条件之一的数据。

13. 使用 BETWEEN 过滤日期范围

- 应用案例：查找发生在 '2023 年 6 月' 的所有订单。
- 需求：从 orders 表中筛选 order_date 在 '2023-06-01' 到 '2023-06-30' 之间的订单。
- 查询代码：

SQL

```
SELECT
    order_id,      -- 选择订单 ID
    order_date,    -- 选择订单日期
    amount         -- 选择订单金额
FROM
    orders         -- 从 orders 表查询
WHERE
    order_date BETWEEN '2023-06-01' AND '2023-06-30'; -- 使用
BETWEEN 过滤日期范围（包含起始和结束日期）
```

- 预期输出：
- | order_id | order_date | amount |
|----------|------------|--------|
| 106 | 2023-06-01 | 450.75 |
| 107 | 2023-06-05 | 80.00 |
| 108 | 2023-06-10 | 180.00 |
| 109 | 2023-06-20 | 90.00 |
- 应用场景：按日期区间筛选数据、按数值区间筛选数据。

14. 使用 LIKE 进行模糊匹配

- 应用案例：查找所有产品名称中包含 'Book' 的产品。
- 需求：从 products 表中筛选 product_name 包含 'Book' 的产品。

- **查询代码：**

SQL

```
SELECT
    product_id,    -- 选择产品 ID
    product_name,  -- 选择产品名称
    category       -- 选择类别
FROM
    products       -- 从 products 表查询
WHERE
    product_name LIKE '%Book%'; -- 使用 LIKE 进行模式匹配，% 代表任意零个或多个字符
```

- **预期输出：**
- | product_id | product_name | category |
|------------|--------------|----------|
| 4 | SQL Book | Books |
| 5 | Python Book | Books |
- **应用场景：**搜索含有特定关键词的文本、查找符合某种模式的记录。

15. 查找唯一值

- **应用案例：**查看 customers 表中所有不重复的城市名称。
- **需求：**获取 customers 表中 city 列的唯一值列表。
- **查询代码：**

SQL

```
SELECT DISTINCT -- 使用 DISTINCT 关键字去除重复的行
    city         -- 选择 city 列
FROM
    customers;   -- 从 customers 表查询
```

- **预期输出：**（注意 NULL 也被视为一个唯一值）
- city
- -----
- New York
- London
- Paris
- NULL
- **应用场景：**统计不重复的类别、来源、用户等。

16. 计算衍生列

- **应用案例：**计算每个产品的潜在总价值（单价 * 库存）。
- **需求：**获取产品名称、价格、库存，并计算一个新列显示潜在总价值。
- **查询代码：**

SQL

```
SELECT
    product_name,      -- 选择产品名称
    price,              -- 选择价格
    stock,              -- 选择库存
    price * stock AS PotentialValue -- 计算价格乘以库存，并将结果命名为 PotentialValue
FROM
    products;          -- 从 products 表查询
```

- **预期输出：**（只显示部分结果）
- | product_name | price | stock | PotentialValue |
|--------------|--------|-------|----------------|
| Laptop | 800.00 | 15 | 12000.00 |
| Keyboard | 75.00 | 50 | 3750.00 |
| Mouse | 25.00 | 100 | 2500.00 |
| ... | ... | ... | ... |
- **应用场景：**计算利润、税费、折扣价、总金额等。

17. 过滤 NULL 值

- **应用案例：**找出 customers 表中那些城市信息缺失（为 NULL）的客户。
- **需求：**从 customers 表中筛选 city 列是 NULL 的行。
- **查询代码：**

SQL

```
SELECT
    customer_id,      -- 选择客户 ID
    customer_name,    -- 选择客户姓名
    city              -- 选择城市（这里会显示 NULL）
FROM
    customers         -- 从 customers 表查询
WHERE
    city IS NULL;    -- 使用 IS NULL 检查列值是否为 NULL（不能用 = NULL）
```

- 预期输出:
- | customer_id | customer_name | city |
|-------------|---------------|------|
| 8 | Wendy | NULL |
- 应用场景: 数据清洗、找出信息不完整的记录。

18. 统计唯一值的数量

- 应用案例: 统计 customers 表中不重复的城市数量。
- 需求: 计算 customers 表中 city 列的唯一值数量。
- 查询代码:

SQL

```
SELECT
    COUNT(DISTINCT city) AS UniqueCityCount -- 使用 COUNT() 函数并结合 DISTINCT 统计 city 列的唯一非 NULL 值数量
FROM
    customers; -- 从 customers 表查询
```

- 预期输出: (注意 COUNT(DISTINCT) 通常不计算 NULL 值)
- UniqueCityCount
- | |
|---|
| 3 |
|---|
- 应用场景: 统计用户来源地数量、产品类别数量、访问页面数量等。

19. 使用窗口函数计算总量的百分比

- 应用案例: 计算每个订单金额占所有订单总金额的百分比。
- 需求: 在不聚合订单明细行的情况下, 计算每行订单金额占所有订单总金额的比例。
- 查询代码:

SQL

```
SELECT
    order_id, -- 选择订单 ID
    amount, -- 选择订单金额
    SUM(amount) OVER() AS TotalAmount, -- 使用 SUM() 窗口函数计算所有行的 amount 总和, OVER() 表示计算整个结果集的总和
```

尖道远信息

```

        (amount / SUM(amount) OVER()) * 100 AS PercentageOfTotal --
计算当前行 amount 除以总和，再乘以 100 得到百分比
FROM
    orders;          -- 从 orders 表查询

```

- **预期输出：**（只显示部分结果）

order_id	amount	TotalAmount	PercentageOfTotal
101	150.00	2931.25	5.1172...
102	250.50	2931.25	8.5455...
103	50.00	2931.25	1.7057...
...	...	...	...

- **应用场景：**计算各部分占总体的比例、市场份额计算、用户行为占比分析等。窗口函数在不折叠原始行的情况下进行聚合计算，非常适用于这类分析。

20. 查找没有关联记录的行 (LEFT JOIN + IS NULL)

- **应用案例：**找出那些从未下过订单的客户。
- **需求：**通过连接 customers 和 orders 表，找到在 customers 表中存在但在 orders 表中没有匹配记录的客户。
- **查询代码：**

SQL

```

SELECT
    c.customer_id,    -- 选择客户 ID
    c.customer_name   -- 选择客户姓名
FROM
    customers c       -- 从 customers 表查询（作为左表）
LEFT JOIN
    orders o ON c.customer_id = o.customer_id -- 与 orders 表
（作为右表）进行左连接
WHERE
    o.order_id IS NULL; -- 过滤条件：只保留右表（orders）中没有
匹配项（order_id 为 NULL）的左表（customers）的行

```

- **预期输出：**
- | customer_id | customer_name |
|-------------|---------------|
| 4 | David |
| 6 | Frank |

- 8 | Wendy
 - **应用场景：**查找未购买产品的客户、查找没有完成某个特定行为的用户、数据一致性检查（如查找没有对应部门的员工）。
-

21. 使用 UNION 合并查询结果

- **应用案例：**获取所有客户和所有产品的名称列表（不重复）。
- **需求：**将 customers 表中的 customer_name 列和 products 表中的 product_name 列合并到一个结果集中，并去除重复名称。
- **查询代码：**

SQL

```
SELECT
    customer_name AS Name -- 选择 customer_name 并取别名 Name
FROM
    customers             -- 从 customers 表查询
UNION
    -- 使用 UNION 合并结果集，并自动去除重复行
SELECT
    product_name AS Name -- 选择 product_name 并取别名 Name
    (UNION 要求合并的列数和类型兼容，且通常建议列名一致或使用别名统一)
FROM
    products;            -- 从 products 表查询
```

- **预期输出：**（顺序可能不同，且重复项被移除）
- Name
- -----
- Alice
- Bob
- Charlie
- David
- Eve
- Frank
- Grace
- Wendy
- Laptop
- Keyboard
- Mouse
- SQL Book
- Python Book

- Desk Chair
 - Desk Lamp
 - Monitor
 - Novel
 - Headphones
 - **应用场景：**合并来自不同表的相似结构数据、合并多个 SELECT 语句的结果进行分析。UNION ALL 会保留所有行，包括重复的。
-

22. 在 WHERE 子句中使用子查询（非关联子查询）

- **应用案例：**查找下过金额大于 300 的订单的客户。
- **需求：**先找出金额大于 300 的订单的 customer_id，然后用这些 customer_id 去筛选客户表。
- **查询代码：**

SQL

```
SELECT
    customer_id,    -- 选择客户 ID
    customer_name   -- 选择客户姓名
FROM
    customers       -- 从 customers 表查询
WHERE
    customer_id IN ( -- 过滤条件：客户 ID 在以下子查询的结果集中
        SELECT customer_id -- 子查询：选择 orders 表中的
        customer_id
            FROM orders      -- 从 orders 表查询
            WHERE amount > 300 -- 子查询过滤条件：订单金额大于 300
    ); -- 子查询结束
```

- **预期输出：**
 - customer_id | customer_name
 - -----|-----
 - 1 | Alice
 - 2 | Bob
 - 7 | Grace
 - **应用场景：**基于一个表的过滤条件去筛选另一个表的数据、复杂的多步过滤。
-

23. 在 FROM 子句中使用子查询（派生表/内联视图）

奕道远信息

- **应用案例：**找出平均订单金额超过 200 的客户。
- **需求：**先计算每个客户的平均订单金额（子查询作为派生表），然后从这个结果集中筛选平均金额大于 200 的客户。
- **查询代码：**

SQL

```
SELECT
    customer_id,          -- 从派生表中选择客户 ID
    AverageOrderAmount    -- 从派生表中选择平均订单金额
FROM
    ( -- 开始子查询，作为 FROM 子句的来源
        SELECT
            customer_id,      -- 选择客户 ID
            AVG(amount) AS AverageOrderAmount -- 计算每个客户的
平均订单金额
        FROM
            orders            -- 从 orders 表查询
        GROUP BY
            customer_id       -- 按客户 ID 分组
    ) AS CustomerAverages -- 给予查询的结果集命名为
CustomerAverages (在 FROM 子句中的子查询必须有别名)
WHERE
    AverageOrderAmount > 200; -- 从 CustomerAverages 这个派生表
中过滤 AverageOrderAmount 大于 200 的行
```

- **预期输出：**
- | customer_id | AverageOrderAmount |
|-------------|--------------------|
| 1 | 312.6875 |
| 2 | 233.3333 |
| 7 | 280.0000 |
- **应用场景：**在进行更复杂的分析前，先对数据进行聚合或转换、分解复杂查询为更小的步骤。

24. 查找前 N 个/最后的 N 个记录

- **应用案例：**找出库存数量最多的 3 个产品。
- **需求：**获取 products 表中 stock 最高的 3 条记录。
- **查询代码：**（这里使用 LIMIT，兼容 MySQL/PostgreSQL/SQLite。SQL Server 使用 TOP 3）

SQL

奕道远信息

```

SELECT
    product_id,    -- 选择产品 ID
    product_name,  -- 选择产品名称
    stock          -- 选择库存
FROM
    products       -- 从 products 表查询
ORDER BY
    stock DESC     -- 先按库存降序排序
LIMIT 3;         -- 限制只返回前 3 行

```

- **预期输出:**

product_id	product_name	stock
3	Mouse	100
9	Novel	80
2	Keyboard	50

- **应用场景:** 排行榜、最新发布的文章、销量最高的产品等。

25. 使用窗口函数 RANK() 进行排名

- **应用案例:** 对每个客户的订单按金额进行排名。
- **需求:** 在每个客户内部, 根据订单金额 (amount) 对其下的订单进行排名。
- **查询代码:**

SQL

```

SELECT
    order_id,      -- 选择订单 ID
    customer_id,   -- 选择客户 ID
    order_date,    -- 选择订单日期
    amount,        -- 选择订单金额
    RANK() OVER (  -- 使用 RANK() 窗口函数进行排名
        PARTITION BY customer_id -- PARTITION BY 指定分组, 这里
        按客户 ID 分组, 排名在每个客户内部独立计算
        ORDER BY amount DESC     -- ORDER BY 指定排序, 这里按订
        单金额降序排序 (金额大的排在前面)
    ) as customer_order_rank -- 给排名结果列命名为
customer_order_rank
FROM
    orders;        -- 从 orders 表查询

```

- **预期输出：**（只显示部分客户的结果）
- | order_id | customer_id | order_date | amount | customer_order_rank |
|----------|-------------|------------|--------|---------------------|
| 111 | 1 | 2023-07-30 | 600.00 | 1 |
| 106 | 1 | 2023-06-01 | 450.75 | 2 |
| 101 | 1 | 2023-01-15 | 150.00 | 3 |
| 103 | 2023-01-17 | 50.00 | 4 | |
| 113 | 2 | 2023-08-05 | 220.00 | 1 |
| 108 | 2 | 2023-06-10 | 180.00 | 2 |
| 104 | 2 | 2023-02-20 | 300.00 | 1 |
| ... | ... | ... | ... | ... |
- **应用场景：**在分组内进行排名（如按销售额对区域内的销售员排名）、按时间对用户的行为进行排序。RANK() 会跳过重复的排名数字（如两个并列第一后是第三）。DENSE_RANK() 则不会跳过（两个并列第一后是第二）。ROW_NUMBER() 即使值相同也会给出连续的行号。

26. 使用窗口函数 ROW_NUMBER() 分配行号

- **应用案例：**给每个客户的第一笔订单分配一个标记。
- **需求：**在每个客户内部，根据订单日期对订单进行排序，并分配一个从 1 开始的连续行号。
- **查询代码：**

SQL

```
SELECT
    order_id,      -- 选择订单 ID
    customer_id,   -- 选择客户 ID
    order_date,    -- 选择订单日期
    amount,        -- 选择订单金额
    ROW_NUMBER() OVER ( -- 使用 ROW_NUMBER() 窗口函数分配连续行号
        PARTITION BY customer_id -- 按客户 ID 分组
        ORDER BY order_date ASC   -- 在每个客户内部按订单日期升序排序（最早的排最前面）
    ) as rn -- 给行号结果列命名为 rn
FROM
    orders;      -- 从 orders 表查询
```

奕道远信息

- **预期输出：**（只显示部分客户的结果）
- | order_id | customer_id | order_date | amount | rn |
|----------|-------------|------------|--------|-----|
| 101 | 1 | 2023-01-15 | 150.00 | 1 |
| 103 | 1 | 2023-01-17 | 50.00 | 2 |
| 106 | 1 | 2023-06-01 | 450.75 | 3 |
| 111 | 1 | 2023-07-30 | 600.00 | 4 |
| 104 | 2 | 2023-02-20 | 300.00 | 1 |
| 108 | 2 | 2023-06-10 | 180.00 | 2 |
| 113 | 2 | 2023-08-05 | 220.00 | 3 |
| ... | ... | ... | ... | ... |
- **应用场景：**给分组内的行分配唯一的序号、查找分组内的第一/最后 N 条记录（结合子查询或 CTE 和 WHERE rn = 1）。

27. 使用窗口函数 SUM() OVER() 计算累计总和 (Running Total)

- **应用案例：**计算每个客户随时间推移的累计订单总金额。
- **需求：**在每个客户内部，按订单日期排序，计算到当前订单为止该客户的所有订单金额总和。
- **查询代码：**

SQL

```
SELECT
    order_id,      -- 选择订单 ID
    customer_id,   -- 选择客户 ID
    order_date,    -- 选择订单日期
    amount,        -- 选择订单金额
    SUM(amount) OVER ( -- 使用 SUM() 窗口函数计算总和
        PARTITION BY customer_id -- 按客户 ID 分组，累计计算在每个客户内部进行
        ORDER BY order_date ASC -- 按订单日期升序排序，累计按时间顺序进行
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW -- 指定窗口范围：从分组开始到当前行（这是默认的累计求和范围，可以省略但写出来更清晰）
    ) as RunningTotalAmount -- 给结果列命名为 RunningTotalAmount
FROM
    orders;          -- 从 orders 表查询
```

- **预期输出：**（只显示部分客户的结果）

奕道远信息

- order_id | customer_id | order_date | amount | RunningTotalAmount
- -----|-----|-----|-----|-----
- ---
- 101 | 1 | 2023-01-15 | 150.00 | 150.00 -- 150
- 103 | 1 | 2023-01-17 | 50.00 | 200.00 --
150+50
- 106 | 1 | 2023-06-01 | 450.75 | 650.75 --
150+50+450.75
- 111 | 1 | 2023-07-30 | 600.00 | 1250.75 --
150+50+450.75+600
- 104 | 2 | 2023-02-20 | 300.00 | 300.00
- 108 | 2 | 2023-06-10 | 180.00 | 480.00
- 113 | 2 | 2023-08-05 | 220.00 | 700.00
- ... | ... | ... | ... | ...
- **应用场景：**计算随时间变化的累计值、账户余额变化、库存累计进出量等。

28. 使用窗口函数 LAG() 获取前一行的值

- **应用案例：**查看每个订单与其前一个订单（按日期排序）的金额差异（在同一个客户内部）。
- **需求：**在每个客户内部，按订单日期排序，获取当前订单金额以及前一个订单的金额。
- **查询代码：**

SQL

```
SELECT
    order_id,      -- 选择订单 ID
    customer_id,   -- 选择客户 ID
    order_date,    -- 选择订单日期
    amount,        -- 选择订单金额
    LAG(amount, 1, 0) OVER ( -- 使用 LAG() 窗口函数获取前一行的数据
        PARTITION BY customer_id -- 按客户 ID 分组
        ORDER BY order_date ASC -- 在每个客户内部按订单日期升序排序
    ) as PreviousOrderAmount, -- 获取前一行的 amount 值，如果没有前一行，则使用默认值 0（参数 1 是偏移量，参数 2 是默认值）
    amount - LAG(amount, 1, 0) OVER ( -- 计算当前金额与前一个订单金额的差值
        PARTITION BY customer_id
```

奕道远信息

```

ORDER BY order_date ASC
) as AmountDifference -- 给差值结果列命名
FROM
    orders;          -- 从 orders 表查询

```

- **预期输出：**（只显示部分客户的结果）
- order_id | customer_id | order_date | amount |
PreviousOrderAmount | AmountDifference
- -----|-----|-----|-----|-----
----|-----
- 101 | 1 | 2023-01-15 | 150.00 | 0.00
| 150.00 -- 第一个订单，前一个金额为 0
- 103 | 1 | 2023-01-17 | 50.00 | 150.00
| -100.00 -- 50 - 150
- 106 | 1 | 2023-06-01 | 450.75 | 50.00
| 400.75 -- 450.75 - 50
- 111 | 1 | 2023-07-30 | 600.00 | 450.75
| 149.25 -- 600 - 450.75
- 104 | 2 | 2023-02-20 | 300.00 | 0.00
| 300.00
- 108 | 2 | 2023-06-10 | 180.00 | 300.00
| -120.00
- 113 | 2 | 2023-08-05 | 220.00 | 180.00
| 40.00
- ... | ... | ... | ... | ...
| ...
- **应用场景：**时间序列分析、比较当前数据点与前一个数据点的值、计算环比增长。LEAD() 函数类似，用于获取后一行的值。

29. 使用 COALESCE() 处理 NULL 值

- **应用案例：**显示客户的城市，如果城市信息缺失，则显示 'Unknown City'。
- **需求：**将 customers 表中 city 列的 NULL 值替换为 'Unknown City'。
- **查询代码：**

SQL

```

SELECT
    customer_id, -- 选择客户 ID
    customer_name, -- 选择客户姓名

```

奕道远信息

```

        COALESCE(city, 'Unknown City') AS DisplayCity -- 使用
COALESCE 函数，如果 city 是 NULL，则返回 'Unknown City'，否则返回
city 的值
FROM
    customers;      -- 从 customers 表查询

```

- **预期输出：**

customer_id	customer_name	DisplayCity
-----	-----	-----
1	Alice	New York
2	Bob	London
3	Charlie	New York
4	David	Paris
5	Eve	London
6	Frank	New York
7	Grace	London
8	Wendy	Unknown City

- **应用场景：**显示数据时用默认值替换 NULL、计算中将 NULL 视为 0 或其他值。不同数据库可能有类似的函数，如 IFNULL() (MySQL) 或 ISNULL() (SQL Server)。

30. 使用 NOT IN 排除特定值列表

- **应用案例：**查找既不是 'Electronics' 也不是 'Books' 类别的所有产品。
- **需求：**从 products 表中筛选 category 不在 ('Electronics', 'Books') 列表中的产品。
- **查询代码：**

SQL

```

SELECT
    product_id,    -- 选择产品 ID
    product_name,  -- 选择产品名称
    category       -- 选择类别
FROM
    products      -- 从 products 表查询
WHERE
    category NOT IN ('Electronics', 'Books'); -- 使用 NOT IN 过
滤条件，排除 category 在给定列表中的行

```

- **预期输出：**

奕道远信息

- | product_id | product_name | category |
|------------|--------------|-----------|
| 6 | Desk Chair | Furniture |
| 7 | Desk Lamp | Furniture |
- 应用场景：排除某些特定类别的产品、用户、区域等。

31. 使用 NOT LIKE 进行反向模糊匹配

- **应用案例：**查找产品名称中不包含 'Book' 且不包含 'Desk' 的所有产品。
- **需求：**从 products 表中筛选 product_name 不匹配特定模式的产品。
- **查询代码：**

SQL

```
SELECT
    product_id,    -- 选择产品 ID
    product_name,  -- 选择产品名称
    category       -- 选择类别
FROM
    products       -- 从 products 表查询
WHERE
    product_name NOT LIKE '%Book%' -- 条件一：产品名称不包含 'Book'
    AND product_name NOT LIKE '%Desk%'; -- 条件二：产品名称不包含 'Desk'（使用 AND 组合多个 NOT LIKE）
```

- **预期输出：**
- | product_id | product_name | category |
|------------|--------------|-------------|
| 1 | Laptop | Electronics |
| 2 | Keyboard | Electronics |
| 3 | Mouse | Electronics |
| 8 | Monitor | Electronics |
| 9 | Novel | Books |
| 10 | Headphones | Electronics |
- 应用场景：排除符合某种模式的记录、查找不属于特定命名规则的数据。

32. 提取日期中的年份和月份

- **应用案例：**查看每个订单发生在哪一年和哪一月。

突道远信息

- **需求：**从 orders 表中提取 order_date 的年份和月份，并与订单 ID 一起显示。
- **查询代码：**（日期函数语法可能因数据库系统而异，这里使用常用的 YEAR() 和 MONTH()）

SQL

```
SELECT
    order_id,      -- 选择订单 ID
    order_date,    -- 选择原始订单日期
    YEAR(order_date) AS OrderYear, -- 使用 YEAR() 函数提取
    order_date 的年份，并命名为 OrderYear
    MONTH(order_date) AS OrderMonth -- 使用 MONTH() 函数提取
    order_date 的月份，并命名为 OrderMonth
FROM
    orders;        -- 从 orders 表查询
```

- **预期输出：**（只显示部分结果）
- | order_id | order_date | OrderYear | OrderMonth |
|----------|------------|-----------|------------|
| 101 | 2023-01-15 | 2023 | 1 |
| 102 | 2023-01-16 | 2023 | 1 |
| 103 | 2023-01-17 | 2023 | 1 |
| 104 | 2023-02-20 | 2023 | 2 |
| 105 | 2023-05-15 | 2023 | 5 |
| ... | ... | ... | ... |
- **应用场景：**按年/月/日/周对数据进行分组或过滤、分析随时间变化的趋势。

33. 计算日期之间的差异

- **应用案例：**计算每个客户加入公司至今的天数。
- **需求：**计算当前日期与 customers 表中 join_date 之间的天数差。
- **查询代码：**（计算日期差异的函数语法因数据库系统而异，这里使用 DATEDIFF()）

SQL

```
SELECT
    customer_id,    -- 选择客户 ID
    customer_name,  -- 选择客户姓名
    join_date,      -- 选择加入日期
```

奕道远信息

```

CURDATE() AS CurrentDate, -- 获取当前日期（函数名称可能不同，
如 GETDATE() 或 NOW()）
DATEDIFF(CURDATE(), join_date) AS DaysSinceJoin -- 计算当前
日期与加入日期之间的天数差，并命名为 DaysSinceJoin
FROM
customers;      -- 从 customers 表查询

```

- **预期输出：**(CurrentDate 和 DaysSinceJoin 会根据你执行查询时的实际日期变化)
- customer_id | customer_name | join_date | CurrentDate | DaysSinceJoin
- -----|-----|-----|-----|-----
- -----
- 1 | Alice | 2022-01-10 | 2025-04-22 | 1198
- 2 | Bob | 2022-02-15 | 2025-04-22 | 1163
- 3 | Charlie | 2022-03-20 | 2025-04-22 | 1129
- 4 | David | 2022-04-01 | 2025-04-22 | 1117
- 5 | Eve | 2023-05-10 | 2025-04-22 | 713
- 6 | Frank | 2023-06-18 | 2025-04-22 | 675
- 7 | Grace | 2023-07-22 | 2025-04-22 | 631
- 8 | Wendy | 2024-01-01 | 2025-04-22 | 477
- **应用场景：**计算年龄、任职时间、逾期天数、订单处理时长等。

34. 条件聚合（使用 CASE 在聚合函数内）

- **应用案例：**统计来自 'New York' 和 'London' 的客户数量，将它们分别列在不同的列中。
- **需求：**计算 customers 表中 city 等于 'New York' 的行数，以及 city 等于 'London' 的行数。
- **查询代码：**

```

SQL

SELECT
    SUM(CASE WHEN city = 'New York' THEN 1 ELSE 0 END) AS
NewYorkCustomerCount, -- 当 city 是 'New York' 时计数为 1，否则
为 0，然后求和得到纽约客户数
    SUM(CASE WHEN city = 'London' THEN 1 ELSE 0 END) AS
LondonCustomerCount, -- 当 city 是 'London' 时计数为 1，否则
为 0，然后求和得到伦敦客户数
    SUM(CASE WHEN city IS NULL THEN 1 ELSE 0 END) AS
UnknownCityCustomerCount -- 当 city 是 NULL 时计数为 1，否则
为 0，求和得到城市未知客户数

```

```
FROM
    customers; -- 从 customers 表查询（不需要 GROUP BY，因为是对整个表进行条件计数）
```

- **预期输出：**
- NewYorkCustomerCount | LondonCustomerCount |
UnknownCityCustomerCount
- -----|-----|-----

- 3 | 3 | 1
- **应用场景：**统计符合不同条件的记录数量、计算不同类别子集的汇总值（如统计男女人数、不同产品状态的数量）、将多个条件聚合结果显示在一行中。

35. 查找具有重复值的行

- **应用案例：**找出 customers 表中城市（city）出现次数多于 1 次的城市名称，以及它们出现的次数。
- **注：**这不是找重复的行*，而是找值出现多次的列。*
- **需求：**统计 customers 表中每个城市的客户数量，并只显示那些客户数量大于 1 的城市。
- **查询代码：**

SQL

```
SELECT
    city,                -- 选择城市
    COUNT(*) AS CustomerCount -- 计算每个城市的客户数量
FROM
    customers            -- 从 customers 表查询
WHERE
    city IS NOT NULL -- 排除城市为 NULL 的情况，如果需要包含 NULL 出现多次的情况，可以移除此行
GROUP BY
    city                -- 按城市分组
HAVING
    COUNT(*) > 1; -- 过滤分组，只保留客户数量大于 1 的城市
```

- **预期输出：**
- city | CustomerCount
- -----|-----
- London | 3
- New York | 3

- **应用场景：**找出出现次数频繁的项、数据质量检查（如查找同一个 ID 出现多次的情况）。
-

36. 使用 CTE (Common Table Expression) 简化复杂查询

- **应用案例：**计算每个客户的总订单金额，然后找出总金额排名前两位的客户。
- **需求：**使用 CTE 先计算每个客户的总订单金额，然后在外层查询中利用 CTE 的结果进行排序和限制。
- **查询代码：**(CTE 使用 WITH 关键字)

SQL

```
WITH CustomerTotalOrders AS ( -- 定义一个名为
CustomerTotalOrders 的 CTE
    SELECT
        customer_id,          -- 选择客户 ID
        SUM(amount) AS TotalAmount -- 计算每个客户的订单总金额
    FROM
        orders                -- 从 orders 表查询
    GROUP BY
        customer_id           -- 按客户 ID 分组
) -- CTE 定义结束，CTE 不是永久对象，只在当前查询中有效

SELECT
    cto.customer_id,          -- 从 CTE 中选择客户 ID (使用 CTE 的
    别名 cto)
    c.customer_name,          -- 从 customers 表中选择客户姓名 (需
    要连接客户表来获取姓名)
    cto.TotalAmount           -- 从 CTE 中选择总金额
FROM
    CustomerTotalOrders cto -- 从 CTE CustomerTotalOrders 中查
    询，并使用别名 cto
INNER JOIN
    customers c ON cto.customer_id = c.customer_id -- 连接
    customers 表来获取客户姓名
ORDER BY
    cto.TotalAmount DESC -- 按总金额降序排序
LIMIT 2;                -- 限制返回前 2 行 (或其他数据库的 TOP
2)
```

- **预期输出：**

奕道远信息

- | customer_id | customer_name | TotalAmount |
|-------------|---------------|-------------|
| 1 | Alice | 1250.75 |
| 2 | Bob | 700.00 |
- **应用场景：**分解复杂查询步骤、提高查询的可读性和可维护性、在同一个查询中多次引用子结果集、递归查询。

37. 使用 RIGHT JOIN 连接两个表

- **应用案例：**获取所有订单的列表，并显示对应的客户姓名（如果客户存在）。如果某个订单的 customer_id 在客户表中找不到，订单也应该显示，但客户信息为 NULL。
- **需求：**连接 customers 表（左表）和 orders 表（右表），基于 customer_id 匹配。返回所有订单信息，并附加匹配的客户信息。
- **查询代码：**

SQL

```
SELECT
    o.order_id,      -- 选择 orders 表的 order_id (右表所有行都会保留)
    o.order_date,    -- 选择 orders 表的 order_date
    o.amount,        -- 选择 orders 表的 amount
    c.customer_name  -- 选择 customers 表的 customer_name (如果没有匹配的客户，此处显示 NULL)
FROM
    customers c      -- 从 customers 表查询 (作为左表)
RIGHT JOIN
    orders o ON c.customer_id = o.customer_id; -- 与 orders 表 (作为右表) 进行右连接
```

- **预期输出：**（与 LEFT JOIN 示例 9 类似，只是保留的是右表 orders 的所有行。在我们现有的数据中，orders 表中的 customer_id 都能在 customers 表中找到匹配，所以这里的 RIGHT JOIN 结果和 INNER JOIN 结果一样。为了看到 NULL，需要 orders 表中有 customer_id 不在 customers 中的记录，但 FOREIGN KEY 通常会阻止这种情况。）
- | order_id | order_date | amount | customer_name |
|----------|------------|--------|---------------|
| 101 | 2023-01-15 | 150.00 | Alice |
| 102 | 2023-01-16 | 250.50 | Charlie |
| 103 | 2023-01-17 | 50.00 | Alice |
| 104 | 2023-02-20 | 300.00 | Bob |

- ... | ... | ... | ...
- **应用场景：**相对于 LEFT JOIN 较少直接使用，通常用 LEFT JOIN 交换左右表顺序实现同样目的。概念上用于保留右表所有行并附加左表匹配信息。

38. 使用 CROSS JOIN 连接两个表

- **应用案例：**你想生成一个列表，包含每个客户和每个产品的所有可能组合。
- **需求：**将 customers 表中的每一行与 products 表中的每一行进行组合。
- **查询代码：**

SQL

```
SELECT
    c.customer_name,  -- 选择客户姓名
    p.product_name    -- 选择产品名称
FROM
    customers c        -- 从 customers 表查询（别名 c）
CROSS JOIN
    products p;        -- 与 products 表进行交叉连接（别名 p）
```

- **预期输出：**（结果会非常多，是客户数 * 产品数，即 $8 * 10 = 80$ 行）
- customer_name | product_name
- -----|-----
- Alice | Laptop
- Alice | Keyboard
- ... | ...
- Bob | Laptop
- Bob | Keyboard
- ... | ...
- Wendy | Novel
- Wendy | Headphones
- **应用场景：**生成所有可能的组合（如产品与颜色组合）、生成测试数据、在某些特定算法中使用。

39. 更新现有数据 (UPDATE)

- **应用案例：**将所有价格低于 50 的产品价格提高 10%。

- **需求：**修改 products 表中 price 小于 50 的记录，将其 price 增加 10%。
- **查询代码：**（⚠️ **注意：**UPDATE 语句会永久修改数据，请谨慎操作，最好先在测试环境或备份数据上运行。）

SQL

```
UPDATE products -- 指定要更新的表
SET price = price * 1.10 -- SET 子句指定要修改的列和新的值（将 price 乘以 1.10）
WHERE price < 50; -- WHERE 子句指定要更新哪些行（只更新价格小于 50 的产品）
```

- **执行结果：**执行此语句后，产品 ID 为 3 (Mouse, 25.00), 4 (SQL Book, 45.00), 7 (Desk Lamp, 35.00), 9 (Novel, 15.00) 的价格将被更新。
 - **应用场景：**批量修改数据、修正错误信息、更新状态。
-

40. 删除数据 (DELETE)

- **应用案例：**删除所有总金额小于 100 的订单。
- **需求：**从 orders 表中删除 amount 小于 100 的行。
- **查询代码：**（⚠️ **注意：**DELETE 语句会永久删除数据，请谨慎操作，最好先在测试环境或备份数据上运行。）

SQL

```
DELETE FROM orders -- 从 orders 表中删除记录
WHERE amount < 100; -- WHERE 子句指定要删除哪些行（删除金额小于 100 的订单）
```

- **执行结果：**执行此语句后，订单 ID 为 103 (50.00) 和 107 (80.00) 的记录将被删除。
- **应用场景：**清理不再需要的数据、删除符合特定条件的记录。

五、 SQL 应用模型-完整代码版

1. 应用模式：聚合与汇总 (Aggregation & Summarization)

奕道远信息

- **目的:** 计算数据集的统计量, 如总和、平均值、计数、最大值、最小值等, 以概括数据特征。
- **常用技术:** SUM(), AVG(), COUNT(), MIN(), MAX() 等聚合函数, 结合 GROUP BY 子句按维度进行分组汇总。
- **应用案例:** 分析公司产品的销售表现。需要计算每个产品类别下的:
 - 总销售金额。
 - 平均单个订单项的销售金额。
 - 售出的订单项总数量 (即有多少条销售记录)。
 - 单个订单项的最低销售金额。
 - 单个订单项的最高销售金额。
 - 售出的产品总件数 (所有订单项中数量的总和)。
- **涉及表格:** products 表 (获取产品类别信息), order_items 表 (记录具体的销售明细)。

首先, 我们需要创建并填充 order_items 表, 并确保 products 表也已创建并填充 (使用我们之前示例中的 products 表数据)。

SQL

-- 确保 orders 表如果存在先删除, 因为 order_items 引用了 orders

DROP TABLE IF EXISTS order_items;

DROP TABLE IF EXISTS orders;

DROP TABLE IF EXISTS customers;

DROP TABLE IF EXISTS products;

-- 建表和插入 products (与之前相同)

-- 1. 创建 products (产品) 表

CREATE TABLE products (

product_id INT PRIMARY KEY, -- 产品 ID, 整数, 主键

product_name VARCHAR(50), -- 产品名称, 最大 50 字符的可变字符串

category VARCHAR(50), -- 产品类别, 最大 50 字符的可变字符串

price DECIMAL(10, 2), -- 产品价格, 精确小数类型

stock INT -- 库存数量, 整数

);

-- 插入示例数据到 products 表

INSERT INTO products (product_id, product_name, category, price, stock) VALUES

(1, 'Laptop', 'Electronics', 800.00, 15),

(2, 'Keyboard', 'Electronics', 75.00, 50),

(3, 'Mouse', 'Electronics', 25.00, 100),

(4, 'SQL Book', 'Books', 45.00, 30),

(5, 'Python Book', 'Books', 60.00, 25),

(6, 'Desk Chair', 'Furniture', 150.00, 10),

(7, 'Desk Lamp', 'Furniture', 35.00, 40),

奕道远信息

```
(8, 'Monitor', 'Electronics', 300.00, 20),
(9, 'Novel', 'Books', 15.00, 80),
(10, 'Headphones', 'Electronics', 90.00, 35);
```

```
-- 建表和插入 orders (与之前相同, 虽然本例查询没直接用, 但 order_items 引用它)
```

```
-----
```

```
-- 2. 创建 orders (订单) 表
```

```
-----
```

```
CREATE TABLE orders (
  order_id INT PRIMARY KEY, -- 订单 ID, 整数, 主键
  customer_id INT,          -- 客户 ID, 整数 (外键, 将关联 customers 表)
  order_date DATE,          -- 订单日期, 日期类型
  amount DECIMAL(10, 2),    -- 订单总金额, 精确小数类型 (总共最多 10 位, 小数点后 2 位)
  -- 注意: 这里省略 FOREIGN KEY (customer_id) REFERENCES customers(customer_id) 和
  -- customers 表的建表插数据, 以简化代码, 但在实际数据库中应包含
  -- FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

```
-- 插入示例数据到 orders 表 (只需要有对应的 order_id)
```

```
INSERT INTO orders (order_id, amount) VALUES
(101, 850.00), (102, 150.00), (103, 35.00), (104, 875.00), (105, 150.00),
(106, 1100.00), (107, 75.00), (108, 150.00), (109, 45.00), (110, 90.00),
(111, 1100.00), (112, 25.00), (113, 225.00), (114, 150.00), (115, 180.00);
```

```
-- 创建 order_items 表
```

```
-----
```

```
-- 3. 创建 order_items (订单明细) 表
```

```
-----
```

```
CREATE TABLE order_items (
  item_id INT PRIMARY KEY, -- 订单明细项 ID, 主键
  order_id INT,            -- 关联的订单 ID (外键)
  product_id INT,          -- 关联的产品 ID (外键)
  quantity INT,            -- 购买数量
  item_sale_amount DECIMAL(10, 2), -- 该订单项的总销售金额 (quantity * price)
  FOREIGN KEY (order_id) REFERENCES orders(order_id), -- 引用 orders 表
  FOREIGN KEY (product_id) REFERENCES products(product_id) -- 引用 products 表
);
```

```
-----
```

```
-- 插入示例数据到 order_items 表
```

```
-----
```

```
INSERT INTO order_items (item_id, order_id, product_id, quantity, item_sale_amount)
```

```
VALUES
(1, 101, 1, 1, 800.00), -- Laptop
(2, 101, 3, 2, 50.00), -- Mouse (2 * 25.00)
(3, 102, 5, 1, 60.00), -- Python Book
(4, 102, 4, 1, 45.00), -- SQL Book
(5, 102, 9, 3, 45.00), -- Novel (3 * 15.00)
(6, 103, 7, 1, 35.00), -- Desk Lamp
(7, 104, 1, 1, 800.00), -- Laptop
(8, 104, 2, 1, 75.00), -- Keyboard
(9, 105, 6, 1, 150.00), -- Desk Chair
(10, 106, 1, 1, 800.00), -- Laptop
(11, 106, 8, 1, 300.00), -- Monitor
(12, 107, 9, 5, 75.00), -- Novel (5 * 15.00)
(13, 108, 2, 2, 150.00), -- Keyboard (2 * 75.00)
(14, 109, 4, 1, 45.00), -- SQL Book
(15, 110, 10, 1, 90.00), -- Headphones
(16, 111, 1, 1, 800.00), -- Laptop
(17, 111, 8, 1, 300.00), -- Monitor
(18, 112, 3, 1, 25.00), -- Mouse
(19, 113, 2, 3, 225.00), -- Keyboard (3 * 75.00)
(20, 114, 6, 1, 150.00), -- Desk Chair
(21, 115, 10, 2, 180.00); -- Headphones (2 * 90.00)
```

请先执行上面的 SQL 代码来设置数据库环境和数据。

查询代码:

```
SQL
SELECT
    p.category, -- 选择产品类别列，用于分组显示结果
    SUM(oi.item_sale_amount) AS TotalCategorySales, -- 计算每个类别下所有订单项销售金额的总和
    AVG(oi.item_sale_amount) AS AverageItemSales, -- 计算每个类别下所有订单项销售金额的平均值
    COUNT(oi.item_id) AS NumberOfItemsSold, -- 计算每个类别下有多少个订单项 (即有多少条明细记录)
    MIN(oi.item_sale_amount) AS MinItemSales, -- 找出每个类别下单个订单项的最低销售金额
    MAX(oi.item_sale_amount) AS MaxItemSales, -- 找出每个类别下单个订单项的最高销售金额
    SUM(oi.quantity) AS TotalQuantitySold -- 计算每个类别下售出的产品总件数 (累加数量)
FROM
    order_items oi -- 从 order_items 表查询，并使用别名 oi
INNER JOIN
    products p ON oi.product_id = p.product_id -- 与 products 表 (别名 p) 进行内连接，通
```

过 product_id 关联订单项和产品信息

GROUP BY

p.category; -- 根据产品类别 (p.category) 对数据进行分组，以便聚合函数在每个类别内独立计算

代码解释 (逐行，含函数用法):

1. SELECT p.category,
 - SELECT: 这是 SQL 查询语句的开始，用于指定要从结果集中返回哪些列。
 - p.category: 选择 products 表（我们给它起了别名 p）中的 category 列。由于我们按类别分组，SELECT 列表中非聚合列必须是分组列。
2. SUM(oi.item_sale_amount) AS TotalCategorySales,
 - SUM(列名): 聚合函数，用于计算指定列中所有数值的总和。会忽略 NULL 值。
 - oi.item_sale_amount: 指定对 order_items 表（别名 oi）中的 item_sale_amount 列进行求和。在 GROUP BY 子句的作用下，这里的求和是在每个 category 分组内独立进行的。
 - AS TotalCategorySales: 给这个计算结果列指定一个易读的别名 TotalCategorySales。
3. AVG(oi.item_sale_amount) AS AverageItemSales,
 - AVG(列名): 聚合函数，用于计算指定列中所有数值的平均值。会忽略 NULL 值。
 - oi.item_sale_amount: 指定对 item_sale_amount 列计算平均值。在每个 category 分组内计算。
 - AS AverageItemSales: 给结果列指定别名 AverageItemSales，表示每个订单项的平均销售额。
4. COUNT(oi.item_id) AS NumberOfItemsSold,
 - COUNT(列名): 聚合函数，用于计算指定列中非 NULL 值的行数。
 - oi.item_id: 指定对 item_id 列计数。item_id 是主键，通常非空且唯一，所以 COUNT(oi.item_id) 实际上是计算每个分组中有多少条订单明细记录。
 - AS NumberOfItemsSold: 给结果列指定别名 NumberOfItemsSold，表示某个类别下总共有多少个销售明细项被记录。COUNT(*) 会计算所有行，包括 NULL 值所在的行（如果整行非空），在本例中 COUNT(oi.item_id) 和 COUNT(*) 对分组行数的计数结果一致。
5. MIN(oi.item_sale_amount) AS MinItemSales,
 - MIN(列名): 聚合函数，用于找出指定列中的最小值。会忽略 NULL 值。
 - oi.item_sale_amount: 在每个 category 分组内找出 item_sale_amount 的最小值。
 - AS MinItemSales: 给结果列指定别名 MinItemSales，表示某个类别下单个订单项的最低销售金额。
6. MAX(oi.item_sale_amount) AS MaxItemSales,
 - MAX(列名): 聚合函数，用于找出指定列中的最大值。会忽略 NULL 值。
 - oi.item_sale_amount: 在每个 category 分组内找出 item_sale_amount 的最大值。
 - AS MaxItemSales: 给结果列指定别名 MaxItemSales，表示某个类别下单个订单项的最高销售金额。

7. SUM(oi.quantity) AS TotalQuantitySold
 - SUM(列名): 这里再次使用 SUM(), 但这次是对 quantity 列求和。
 - oi.quantity: 指定对 quantity 列进行求和。在每个 category 分组内计算。这代表了类别下所有订单项的购买数量的总和, 即卖出了多少件具体产品。
 - AS TotalQuantitySold: 给结果列指定别名 TotalQuantitySold。
8. FROM order_items oi
 - FROM: 指定要从中检索数据的表。
 - order_items oi: 从 order_items 表中检索, 并给该表指定别名 oi, 使后续引用列名更简洁。
9. INNER JOIN products p ON oi.product_id = p.product_id
 - INNER JOIN: 连接关键字, 用于将 order_items 表与 products 表连接。
 - products p: 指定要连接的第二个表是 products, 并指定别名 p。
 - ON oi.product_id = p.product_id: 指定连接条件。只有当 order_items 表中的 product_id 与 products 表中的 product_id 相等时, 这两张表的行才会被匹配并组合在一起。
10. GROUP BY p.category;
 - GROUP BY: 这是一个非常关键的子句, 它将 FROM 和 JOIN 子句产生的中间结果集按照指定的列 (这里是 p.category) 进行分组。
 - p.category: 指定按照产品类别进行分组。SELECT 列表中所有的聚合函数 (SUM, AVG, COUNT, MIN, MAX) 都将针对每个独立的分组进行计算。SELECT 列表中任何非聚合列都必须出现在 GROUP BY 子句中。

预期输出:

基于我们插入的示例数据, 查询执行后的预期输出如下:

category	TotalCategorySales	AverageItemSales	NumberOfItemsSold	MinItemSales	MaxItemSales	TotalQuantitySold
----------	--------------------	------------------	-------------------	--------------	--------------	-------------------

Books	270.00	54.000000	5	45.00	75.00	11
Electronics	3985.00	306.538461	13	25.00	800.00	18
Furniture	335.00	111.666666	3	35.00	150.00	3

这个详细的例子展示了如何结合使用 GROUP BY 和多种聚合函数来对数据进行分组汇总, 这是数据处理中最基本和最常用的模式之一。

2. 应用模式: 数据筛选与过滤 (Filtering Data)

本样例将沿用之前创建的 customers 表和 orders 表。

- **目的:** 根据特定条件选取数据子集, 去除不需要的噪声或无关信息。
- **常用技术:** WHERE 子句配合比较运算符 (=, >, <, >=, <=, != 或 <>, BETWEEN, LIKE, IS NULL, IS NOT NULL) 和逻辑运算符 (AND, OR, NOT, IN, NOT IN)。HAVING 子句用于过滤 GROUP BY 后的分组。
- **应用案例:** 你想分析在 **2023 年下半年** (BETWEEN) 产生的订单中, **金额大于 100** (>)

且 **不等于 150 (!=)** 的订单。同时，这些订单必须是来自城市在 **'New York'** 或 **'London'** (IN) 的客户，并且这些客户的姓名 **不是以 'D' 开头** (NOT LIKE)。最后，你只想找出那些经过上述所有过滤条件后，其 **订单总金额仍然大于 500** (HAVING) 的客户 ID。

- **涉及表格:** customers 表, orders 表。

首先，请确保您已经执行了我们之前提供的 customers 和 orders 表的建表和数据插入代码。

查询代码:

SQL

SELECT

 c.customer_id, -- 选择客户 ID

 c.customer_name, -- 选择客户姓名 (虽然 HAVING 只看总金额，但在 SELECT 里可以显示)

 SUM(o.amount) AS FilteredTotalOrderAmount -- 计算经过 WHERE 过滤后的订单金额总和

FROM

 orders o -- 从 orders 表查询，并使用别名 o

INNER JOIN

 customers c ON o.customer_id = c.customer_id -- 与 customers 表 (别名 c) 进行内连接，通过 customer_id 关联

WHERE

 o.order_date BETWEEN '2023-07-01' AND '2023-12-31' -- WHERE 条件 1: 订单日期在指定范围内 (包含边界)

 AND o.amount > 100 -- WHERE 条件 2: 订单金额大于 100

 AND o.amount != 150 -- WHERE 条件 3: 订单金额不等于 150 (!= 和 <> 都可以表示不等于)

 AND c.city IN ('New York', 'London') -- WHERE 条件 4: 客户所在城市在 'New York' 或 'London' 列表中

 AND c.customer_name NOT LIKE 'D%' -- WHERE 条件 5: 客户姓名不以 'D' 开头

 AND o.amount IS NOT NULL -- WHERE 条件 6: 订单金额不能为 NULL (虽然在本例数据中不是问题，但演示 IS NOT NULL 用法)

GROUP BY

 c.customer_id, -- 按客户 ID 分组

 c.customer_name -- 如果 SELECT 列表中包含非聚合列 (customer_name)，也需要加到 GROUP BY 中

HAVING

 SUM(o.amount) > 500; -- HAVING 条件: 过滤 GROUP BY 后的分组，只保留经过 WHERE 过滤且分组总金额大于 500 的客户

代码解释 (逐行，含技术用法):

1. SELECT c.customer_id, c.customer_name, SUM(o.amount) AS
 FilteredTotalOrderAmount

- SELECT: 指定要从查询结果中返回哪些列。
- c.customer_id: 选择 customers 表 (别名 c) 的 customer_id 列。
- c.customer_name: 选择 customers 表的 customer_name 列。

- SUM(o.amount): 使用 SUM() 聚合函数计算 orders 表 (别名 o) 中 amount 列的总和。
 - **用法解释:** SUM(列名) 用于计算指定列的数值总和。在 GROUP BY 子句存在时, 它计算的是每个分组内的总和。
- AS FilteredTotalOrderAmount: 为 SUM(o.amount) 的结果列指定一个别名 FilteredTotalOrderAmount, 以便结果更易读。
- 2. FROM orders o
 - FROM: 指定数据来源表。
 - orders o: 从 orders 表中获取数据, 并指定别名 o。
- 3. INNER JOIN customers c ON o.customer_id = c.customer_id
 - INNER JOIN: 连接关键字, 用于将 orders 表与 customers 表基于匹配条件合并。
 - customers c: 指定要连接的第二个表是 customers, 并指定别名 c。
 - ON o.customer_id = c.customer_id: 指定连接条件。只保留 orders 表和 customers 表中 customer_id 相等的行。
- 4. WHERE o.order_date BETWEEN '2023-07-01' AND '2023-12-31'
 - WHERE: 引入过滤条件, 用于筛选**原始行** (在分组和聚合发生之前)。
 - o.order_date: 指定要过滤的列是 orders 表的 order_date。
 - BETWEEN 值 1 AND 值 2: **用法解释:** 列名 BETWEEN 值 1 AND 值 2 用于筛选列值在指定范围内的行 (包含 值 1 和 值 2)。常用于数字或日期范围过滤。
- 5. AND o.amount > 100
 - AND: **用法解释:** 逻辑运算符, 用于组合多个 WHERE 条件。只有当 AND 连接的所有条件都为真时, 该行才会被保留。
 - o.amount > 100: 比较运算符 >= **用法解释:** 列名 > 值 筛选列值大于指定值的行。其他比较运算符包括 =, <, >=, <=, != (不等于), <> (不等于)。
- 6. AND o.amount != 150
 - AND: 再次使用 AND 组合条件。
 - o.amount != 150: **用法解释:** 列名 != 值 (或 <>) 筛选列值不等于指定值的行。
- 7. AND c.city IN ('New York', 'London')
 - AND: 组合条件。
 - c.city IN ('New York', 'London'): **用法解释:** 列名 IN (值 1, 值 2, ...) 筛选列值在给定的列表中的行。相当于使用 OR 连接多个等值条件 (city = 'New York' OR city = 'London')。NOT IN 则用于筛选不在列表中的行。
- 8. AND c.customer_name NOT LIKE 'D%'
 - AND: 组合条件。
 - c.customer_name NOT LIKE 'D%': **用法解释:** 列名 LIKE '模式' 用于进行模糊匹配。% 代表任意零个或多个字符, _ 代表任意单个字符。NOT LIKE 则用于筛选不匹配指定模式的行。这里 'D%' 表示以 'D' 开头的任意字符串。
- 9. AND o.amount IS NOT NULL
 - AND: 组合条件。
 - o.amount IS NOT NULL: **用法解释:** 列名 IS NULL 用于检查列值是否为 NULL。IS NOT NULL 用于检查列值是否不为 NULL。**注意:** 不能使用 = 或 != 来判断 NULL。

10. GROUP BY c.customer_id, c.customer_name

- GROUP BY: 将经过 WHERE 子句过滤后的行, 按照 customer_id 和 customer_name 进行分组。聚合函数 (SUM 在此例中) 将针对每个分组进行计算。SELECT 列表中所有非聚合的列都必须出现在 GROUP BY 子句中。

11. HAVING SUM(o.amount) > 500;

- HAVING: 引入过滤条件, 用于筛选经过 GROUP BY 分组并且聚合函数计算完成后的分组。
- SUM(o.amount) > 500: 这是一个基于聚合结果的条件。它会检查每个分组的 SUM(o.amount) 是否大于 500。只有满足条件的分组才会被包含在最终结果中。
- 用法解释: HAVING 条件 用于过滤分组, 条件中通常包含聚合函数。与 WHERE 的区别在于, WHERE 在分组前过滤原始行, HAVING 在分组后过滤分组。

预期输出:

根据我们之前提供的 customers 和 orders 数据, 并应用上述所有过滤条件:

1. 首先, WHERE 子句会过滤出 2023 年下半年, 金额 > 100 且 != 150, 来自纽约或伦敦且姓名不以 D 开头的客户的订单。
 - 原始订单 (H2 2023): 110(Grace, London, 210), 111(Alice, NY, 600), 112(Charlie, NY, 110), 113(Bob, London, 220), 114(Eve, London, 150), 115(Grace, London, 350).
 - 应用 amount > 100: 所有都符合。
 - 应用 amount != 150: 排除 114 (Eve, 150)。剩余: 110, 111, 112, 113, 115.
 - 应用 city IN ('New York', 'London'): 剩余订单的客户 Grace(London), Alice(NY), Charlie(NY), Bob(London) 的城市都在列表中, 符合。
 - 应用 customer_name NOT LIKE 'D%': 剩余客户 Grace, Alice, Charlie, Bob 的姓名都不以 D 开头, 符合。
 - 应用 amount IS NOT NULL: 所有剩余订单金额非空, 符合。
 - WHERE 过滤后的订单及客户:
 - (7, Grace), 110, 2023-07-25, 210.00
 - (1, Alice), 111, 2023-07-30, 600.00
 - (3, Charlie), 112, 2023-08-01, 110.00
 - (2, Bob), 113, 2023-08-05, 220.00
 - (7, Grace), 115, 2023-08-15, 350.00
2. 然后, GROUP BY c.customer_id, c.customer_name 对这些过滤后的订单进行分组。
 - 客户 1 (Alice): 订单 111 (金额 600.00)。总金额 = 600.00。
 - 客户 2 (Bob): 订单 113 (金额 220.00)。总金额 = 220.00。
 - 客户 3 (Charlie): 订单 112 (金额 110.00)。总金额 = 110.00。
 - 客户 7 (Grace): 订单 110 (金额 210.00), 订单 115 (金额 350.00)。总金额 = 210.00 + 350.00 = 560.00。
3. 最后, HAVING SUM(o.amount) > 500 过滤分组。
 - 客户 1 (Alice): 总金额 600.00 > 500。保留。
 - 客户 2 (Bob): 总金额 220.00 <= 500。排除。
 - 客户 3 (Charlie): 总金额 110.00 <= 500。排除。
 - 客户 7 (Grace): 总金额 560.00 > 500。保留。

最终结果只包含客户 ID 1 和客户 ID 7 的信息。

customer_id	customer_name	FilteredTotalOrderAmount
1	Alice	600.00
7	Grace	560.00

这个例子综合运用了多种 WHERE 条件和 HAVING 子句，展示了如何在不同阶段（行过滤和分组过滤）应用筛选规则，这是 SQL 数据处理中非常常见和强大的能力。

3. 数据关联与整合

数据关联与整合是 SQL 中进行数据处理的核心环节。它允许我们将分散在不同表中的相关信息整合到一起，形成一个更完整的数据视图进行分析。

我们将详细演示 各种 JOIN 类型。为了更好地展示不同 JOIN 类型的行为，我们将在之前使用的表基础上，稍微修改一下 products 表（增加一个从未售出的产品），并新增两个用于演示 FULL OUTER JOIN 的简单列表表。

首先，是更新后的表结构和数据（包含之前的数据和新增用于演示的数据）：

SQL

-- 删除旧表以重新创建

DROP TABLE IF EXISTS order_items;

DROP TABLE IF EXISTS orders;

DROP TABLE IF EXISTS customers;

DROP TABLE IF EXISTS products;

DROP TABLE IF EXISTS ListA; -- 新增的演示表

DROP TABLE IF EXISTS ListB; -- 新增的演示表

-- 建表和插入 products (新增产品 ID 11)

-- 1. 创建 products (产品) 表

```
CREATE TABLE products (  
  product_id INT PRIMARY KEY,  
  product_name VARCHAR(50),  
  category VARCHAR(50),  
  price DECIMAL(10, 2),  
  stock INT
```

);

-- 插入示例数据到 products 表 (新增产品 ID 11)

INSERT INTO products (product_id, product_name, category, price, stock) VALUES

```
(1, 'Laptop', 'Electronics', 800.00, 15),
(2, 'Keyboard', 'Electronics', 75.00, 50),
(3, 'Mouse', 'Electronics', 25.00, 100),
(4, 'SQL Book', 'Books', 45.00, 30),
(5, 'Python Book', 'Books', 60.00, 25),
(6, 'Desk Chair', 'Furniture', 150.00, 10),
(7, 'Desk Lamp', 'Furniture', 35.00, 40),
(8, 'Monitor', 'Electronics', 300.00, 20),
(9, 'Novel', 'Books', 15.00, 80),
(10, 'Headphones', 'Electronics', 90.00, 35),
(11, 'Mystery Item', 'Misc', 999.99, 5); -- 新增一个产品，它不会出现在 order_items 表中
```

```
-- 建表和插入 orders (与之前相同，仅为保证外键引用存在)
```

```
-- 2. 创建 orders (订单) 表
```

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    amount DECIMAL(10, 2)
);
```

```
-- 插入示例数据到 orders 表 (仅 order_id 和 amount)
```

```
INSERT INTO orders (order_id, amount) VALUES
(101, 850.00), (102, 150.00), (103, 35.00), (104, 875.00), (105, 150.00),
(106, 1100.00), (107, 75.00), (108, 150.00), (109, 45.00), (110, 90.00),
(111, 1100.00), (112, 25.00), (113, 225.00), (114, 150.00), (115, 180.00);
```

```
-- 建表和插入 order_items (与之前相同)
```

```
-- 3. 创建 order_items (订单明细) 表
```

```
CREATE TABLE order_items (
    item_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT,
    item_sale_amount DECIMAL(10, 2),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

```
-----
-- 插入示例数据到 order_items 表 (与之前相同)
-----
INSERT INTO order_items (item_id, order_id, product_id, quantity, item_sale_amount)
VALUES
(1, 101, 1, 1, 800.00), (2, 101, 3, 2, 50.00), (3, 102, 5, 1, 60.00), (4, 102, 4, 1, 45.00), (5, 102, 9,
3, 45.00),
(6, 103, 7, 1, 35.00), (7, 104, 1, 1, 800.00), (8, 104, 2, 1, 75.00), (9, 105, 6, 1, 150.00), (10, 106,
1, 1, 800.00),
(11, 106, 8, 1, 300.00), (12, 107, 9, 5, 75.00), (13, 108, 2, 2, 150.00), (14, 109, 4, 1, 45.00), (15,
110, 10, 1, 90.00),
(16, 111, 1, 1, 800.00), (17, 111, 8, 1, 300.00), (18, 112, 3, 1, 25.00), (19, 113, 2, 3, 225.00), (20,
114, 6, 1, 150.00),
(21, 115, 10, 2, 180.00);
```

```
-- 新增用于 FULL OUTER JOIN 演示的简单表
```

```
-- 4. 创建 ListA 表
```

```
-----
CREATE TABLE ListA (
    ID INT PRIMARY KEY,
    ValueA VARCHAR(10)
);
```

```
-- 插入示例数据到 ListA
```

```
-----
INSERT INTO ListA VALUES
(1, 'A_one'),
(2, 'A_two'),
(3, 'A_three');
```

```
-- 5. 创建 ListB 表
```

```
-----
CREATE TABLE ListB (
    ID INT PRIMARY KEY,
    ValueB VARCHAR(10)
);
```

```
-- 插入示例数据到 ListB
```

```
-----  
INSERT INTO ListB VALUES  
(2, 'B_two'),  
(3, 'B_three'),  
(4, 'B_four');
```

请先执行上面的 SQL 代码来设置数据库环境和数据。

SQL 应用案例：不同 JOIN 类型演示

我们将使用 products 表和 order_items 表来演示 INNER JOIN, LEFT JOIN, RIGHT JOIN。使用 ListA 和 ListB 表来演示 FULL OUTER JOIN。使用 customers 和 products 表来演示 CROSS JOIN。

关于 ON 子句：

- **ON 子句：** **用法解释：** ON 子句紧跟在 JOIN 关键字（或其类型如 INNER JOIN）之后，用于指定连接两个表时，用来匹配行的条件。通常是 表 1.列名 = 表 2.列名，这些列通常是一个表的主键和另一个表的外键。

3.1 INNER JOIN (内连接)

- **目的/行为：** 返回两个表中都满足连接条件的行。如果某个表中的行在另一个表中没有匹配项，则该行不会包含在结果中。它是最常见的连接类型，默认的 JOIN 类型通常就是 INNER JOIN。
- **应用案例：** 查看每个订单明细项对应的产品名称和类别。
- **涉及表格：** order_items, products.
- **查询代码：**

SQL

SELECT

```
oi.item_id,    -- 选择订单明细项 ID  
oi.order_id,   -- 选择关联的订单 ID  
p.product_name, -- 选择关联的产品名称 (来自 products 表)  
p.category,    -- 选择关联的产品类别 (来自 products 表)  
oi.quantity,   -- 选择购买数量  
oi.item_sale_amount -- 选择该项的销售金额
```

FROM

```
order_items oi -- 从 order_items 表查询，别名 oi
```

INNER JOIN

```
products p ON oi.product_id = p.product_id; -- 与 products 表 (别名 p) 进行内连接，条件是 product_id 相等
```

- **代码解释：**
 - SELECT ...: 指定要返回的列，从两个表中选取需要的信息。
 - FROM order_items oi: 从左表 order_items 查询，并起别名 oi。
 - INNER JOIN products p: 与右表 products 进行内连接，并起别名 p。
 - ON oi.product_id = p.product_id: 指定连接条件。SQL 会查找 order_items 表中 product_id 的值与 products 表中 product_id 的值相等的行，并将它们

组合起来。

- **预期输出:** (所有 order_items 中的 product_id 都能在 products 中找到, 所以会返回 order_items 的所有行, 并附加对应的产品信息。产品 ID 11 'Mystery Item' 不会出现在结果中, 因为它没有匹配的 order_items 记录)
- item_id | order_id | product_name | category | quantity | item_sale_amount
- -----|-----|-----|-----|-----|-----

- 1 | 101 | Laptop | Electronics | 1 | 800.00
- 2 | 101 | Mouse | Electronics | 2 | 50.00
- 3 | 102 | Python Book | Books | 1 | 60.00
- ... | ... | ... | ... | ... | ...
- 21 | 115 | Headphones | Electronics | 2 | 180.00

3.2 LEFT JOIN (左连接)

- **目的/行为:** 返回左表 (FROM 后面的表) 的所有行, 以及右表 (JOIN 后面的表) 中满足连接条件的匹配行。如果左表的某一行在右表中没有匹配项, 则结果中右表的列将显示 NULL。
- **应用案例:** 查看所有产品的信息, 并显示它们被销售的订单明细项 (如果曾被销售过)。这可以用来找出从未被销售过的产品。
- **涉及表格:** products (左表), order_items (右表)。

- **查询代码:**

SQL

SELECT

```
p.product_id, -- 选择产品 ID (左表的所有行都保留)
p.product_name, -- 选择产品名称
p.category, -- 选择产品类别
oi.item_id, -- 选择订单明细项 ID (如果没有匹配的订单明细项, 此处显示 NULL)
oi.order_id, -- 选择关联的订单 ID (如果没有匹配的订单明细项, 此处显示 NULL)
oi.quantity -- 选择购买数量 (如果没有匹配的订单明细项, 此处显示 NULL)
```

FROM

```
products p -- 从 products 表查询 (作为左表)
```

LEFT JOIN

```
order_items oi ON p.product_id = oi.product_id; -- 与 order_items 表 (作为右表) 进行左连接, 条件是 product_id 相等
```

- **代码解释:**
 - FROM products p: 从左表 products 查询。
 - LEFT JOIN order_items oi: 与右表 order_items 进行左连接。
 - ON p.product_id = oi.product_id: 连接条件。SQL 会保留 products 表中的所有行。对于 products 中的每一行, 如果在 order_items 中找到了 product_id 相等的匹配项, 则组合这些行的列; 如果没有找到匹配项 (即该产品从未出现在 order_items 中), 则结果中来自 order_items 的列 (item_id, order_id, quantity 等) 将为 NULL。
- **预期输出:** (会包含所有产品, 包括产品 ID 11 'Mystery Item', 因为它是左表

products 的一部分)

- product_id | product_name | category | item_id | order_id | quantity
- -----|-----|-----|-----|-----|-----
- 1 | Laptop | Electronics | 1 | 101 | 1
- 1 | Laptop | Electronics | 7 | 104 | 1
- 1 | Laptop | Electronics | 10 | 106 | 1
- 1 | Laptop | Electronics | 16 | 111 | 1
- 2 | Keyboard | Electronics | 8 | 104 | 1
- 2 | Keyboard | Electronics | 13 | 108 | 2
- 2 | Keyboard | Electronics | 19 | 113 | 3
- ... | ... | ... | ... | ... | ...
- 11 | Mystery Item | Misc | NULL | NULL | NULL -- 这个产品从未被销售过

3.3 RIGHT JOIN (右连接)

- **目的/行为:** 返回右表 (JOIN 后面的表) 的所有行, 以及左表 (FROM 后面的表) 中满足连接条件的匹配行。如果右表的某一行在左表中没有匹配项, 则结果中左表的列将显示 NULL。概念上与 LEFT JOIN 相反。
- **应用案例:** 查看所有订单明细项, 并显示它们关联的产品信息 (如果产品存在)。这可以用来找出订单明细中引用了不存在的产品 ID 的“孤儿”记录 (在有外键约束时通常不可能出现)。
- **涉及表格:** order_items (左表), products (右表)。

• 查询代码:

SQL

SELECT

```
p.product_id, -- 选择产品 ID (来自 products 表, 右表所有行都保留)
p.product_name, -- 选择产品名称
p.category, -- 选择产品类别
oi.item_id, -- 选择订单明细项 ID (如果没有匹配的订单明细项, 此处显示 NULL)
oi.order_id, -- 选择关联的订单 ID (如果没有匹配的订单明细项, 此处显示 NULL)
oi.quantity -- 选择购买数量 (如果没有匹配的订单明细项, 此处显示 NULL)
```

FROM

```
order_items oi -- 从 order_items 表查询 (作为左表)
```

RIGHT JOIN

```
products p ON oi.product_id = p.product_id; -- 与 products 表 (作为右表) 进行右连接,
条件是 product_id 相等
```

• 代码解释:

- FROM order_items oi: 从左表 order_items 查询。
- RIGHT JOIN products p: 与右表 products 进行右连接。
- ON oi.product_id = p.product_id: 连接条件。SQL 会保留 products 表中的所有行。对于 products 中的每一行, 如果在 order_items 中找到了 product_id 相等的匹配项, 则组合这些行的列; 如果没有找到匹配项, 则结果中来自 order_items 的列将为 NULL。

- **预期输出:** (结果与上面的 LEFT JOIN 示例非常相似, 因为它是保留右表所有行, 而 products 是右表。通常, RIGHT JOIN 的场景可以用 LEFT JOIN 交换 FROM 和 JOIN 后面的表来实现, 所以 LEFT JOIN 更常用。)
- product_id | product_name | category | item_id | order_id | quantity
- -----|-----|-----|-----|-----|-----
- 1 | Laptop | Electronics | 1 | 101 | 1
- 1 | Laptop | Electronics | 7 | 104 | 1
- 1 | Laptop | Electronics | 10 | 106 | 1
- 1 | Laptop | Electronics | 16 | 111 | 1
- 2 | Keyboard | Electronics | 8 | 104 | 1
- 2 | Keyboard | Electronics | 13 | 108 | 2
- 2 | Keyboard | Electronics | 19 | 113 | 3
- ... | ... | ... | ... | ... | ...
- 11 | Mystery Item | Misc | NULL | NULL | NULL -- 这个产品没有匹配的 order_items 记录

3.4 FULL OUTER JOIN (全外连接)

- **目的/行为:** 返回当使用 LEFT JOIN 和 RIGHT JOIN 时, 所有未被匹配的行。也就是说, 返回两个表中的所有行, 无论它们在另一个表中是否有匹配项。如果某行的匹配项只存在于一个表中, 则另一表对应的列显示 NULL。
- **应用案例:** 比较 ListA 和 ListB 中的 ID, 查看哪些 ID 只存在于 ListA, 哪些只存在于 ListB, 哪些同时存在于两者。
- **涉及表格:** ListA, ListB.
- **查询代码:** (并非所有数据库系统都支持 FULL OUTER JOIN, 例如 MySQL 8.0 以前的版本不支持, 但可以用 LEFT JOIN UNION ALL RIGHT JOIN 的方式模拟实现)

SQL

SELECT

```
A.ID AS A_ID,    -- 选择 ListA 的 ID
A.ValueA,        -- 选择 ListA 的 ValueA (如果在 ListB 中没有匹配, 此处显示 NULL)
B.ID AS B_ID,    -- 选择 ListB 的 ID (如果在 ListA 中没有匹配, 此处显示 NULL)
B.ValueB         -- 选择 ListB 的 ValueB (如果在 ListA 中没有匹配, 此处显示 NULL)
```

FROM

```
ListA A          -- 从 ListA 表查询, 别名 A
```

FULL OUTER JOIN

```
ListB B ON A.ID = B.ID; -- 与 ListB 表 (别名 B) 进行全外连接, 条件是 ID 相等
```

- **代码解释:**
 - FROM ListA A FULL OUTER JOIN ListB B ON A.ID = B.ID: 对 ListA 和 ListB 进行全外连接。SQL 会保留所有在 A.ID = B.ID 条件下匹配的行, 以及所有只在 ListA 中存在 (且在 ListB 中无匹配) 的行, 以及所有只在 ListB 中存在 (且在 ListA 中无匹配) 的行。
 - 在结果中, 如果某行只来自 ListA, 则 B.ID 和 B.ValueB 为 NULL。如果某行只来自 ListB, 则 A.ID 和 A.ValueA 为 NULL。如果两边都有匹配, 则所有列都显示对应的值。

突迨远信息

- **预期输出:**
- A_ID | ValueA | B_ID | ValueB
- -----|-----|-----|-----
- 1 | A_one | NULL | NULL -- 仅在 ListA 中存在
- 2 | A_two | 2 | B_two -- 在两表中都存在
- 3 | A_three | 3 | B_three -- 在两表中都存在
- NULL | NULL | 4 | B_four -- 仅在 ListB 中存在
- **应用场景:** 比较两个数据集的异同、合并可能存在不完整匹配的数据。

3.5 CROSS JOIN (交叉连接/笛卡尔积)

- **目的/行为:** 返回左表中的每一行与右表中的每一行的所有可能组合。结果集的行数等于左表行数乘以右表行数。它通常不用于通过键关联数据，而是用于生成所有可能的组合。
- **应用案例:** 生成所有客户与所有产品的可能组合列表。
- **涉及表格:** customers, products.

• 查询代码:

SQL

SELECT

c.customer_name, -- 选择客户姓名

p.product_name -- 选择产品名称

FROM

customers c -- 从 customers 表查询，别名 c

CROSS JOIN

products p; -- 与 products 表进行交叉连接，别名 p

- **代码解释:**
 - FROM customers c CROSS JOIN products p: 对 customers 表和 products 表进行交叉连接。SQL 会将 customers 表的每一行与 products 表的每一行进行组合。
 - **注意:** 交叉连接没有 ON 子句，因为它不基于任何条件进行匹配，只是简单地将所有行进行组合。
- **预期输出:** (客户数 (8) * 产品数 (11) = 88 行，只显示部分结果)
- customer_name | product_name
- -----|-----
- Alice | Laptop
- Alice | Keyboard
- ... | ...
- Alice | Headphones
- Alice | Mystery Item
- Bob | Laptop
- Bob | Keyboard
- ... | ...
- Wendy | Novel
- Wendy | Headphones

奕道远信息

- Wendy | Mystery Item
 - **应用场景:** 生成所有组合列表 (如产品 SKU 变体组合)、生成测试数据、某些特定的数学计算或统计分析。
-

4. 数据转换与衍生 (Data Transformation & Derivation)

- 数据转换与衍生是 SQL 中非常灵活和实用的能力, 它允许我们根据业务需求, 对现有数据进行计算、格式化或逻辑判断, 生成新的信息列。
- **应用模式:** 数据转换与衍生 (Data Transformation & Derivation)
- **目的:** 根据现有数据计算新的字段, 或改变数据的表现形式, 以便更直接地进行分析或报告。
- **常用技术:**
 - CASE WHEN 语句: 实现条件判断和分支赋值。
 - 算术运算符 (+, -, *, /): 进行基本的数学计算。
 - 字符串函数 (SUBSTRING, CONCAT, LENGTH 等): 处理文本数据。
 - 日期函数 (YEAR, MONTH, DAY, DATEDIFF 等): 处理日期和时间数据。
- **应用案例:** 对客户信息进行多方面的转换和衍生, 以便更好地理解客户画像和生命周期。例如:
 - 将客户姓名进行格式化或提取部分信息。
 - 计算客户的会员时长。
 - 根据加入年份对客户进行分段。
 - 基于某些属性计算一个简单的得分。
- **涉及表格:** customers 表 (包含 customer_name 和 join_date)。我们将使用 CURDATE() 或类似的函数代表当前日期进行计算。
- 首先, 请确保您已经执行了我们之前提供的 customers 表的建表和数据插入代码 (包括城市为 NULL 的 Wendy)。

• 查询代码:

- (假设当前日期是 '2025-04-24' 进行 DATEDIFF 计算)
- SQL
- SELECT
 - customer_id, -- 选择客户 ID
 - customer_name, -- 选择原始客户姓名
 - - -- 字符串处理示例: 生成一个格式化的姓名显示
 - CONCAT(-- CONCAT 函数用法: CONCAT(string1, string2, ...) 将多个字符串连接起来
 - SUBSTRING(customer_name, 1, 3), -- SUBSTRING 函数用法: SUBSTRING(string, start, length) 提取字符串的一部分, 从 start 位置开始, 提取 length 个字符
 - '... (Length: ', -- 文字字符串连接
 - LENGTH(customer_name), -- LENGTH 函数用法: LENGTH(string) 计算字符串的长度 (字符数)

- ' ' -- 文字字符串连接
-) AS FormattedName, -- 结果列别名, 例如 'Ali... (Length: 5)'
-
- -- 日期函数示例: 提取年份和月份, 计算日期差异
- YEAR(join_date) AS JoinYear, -- YEAR 函数用法: YEAR(date) 提取日期的年份
- MONTH(join_date) AS JoinMonth, -- MONTH 函数用法: MONTH(date) 提取日期的月份
- DATEDIFF(CURDATE(), join_date) AS MembershipDaysTotal, -- DATEDIFF 函数用法: DATEDIFF(date1, date2) 计算两个日期之间的天数差 (date1 - date2)
- DATEDIFF(CURDATE(), join_date) / 365.25 AS MembershipYearsApprox, -- 算术运算符 / 用法: 进行除法计算。这里用天数除以 365.25 (考虑闰年) 近似计算年数
-
- -- 算术运算符示例: 基于日期差异和姓名长度进行简单计算
- DATEDIFF(CURDATE(), join_date) - 30 AS DaysMinus30, -- 算术运算符 - 用法: 进行减法计算。计算入会天数减去 30
- LENGTH(customer_name) * 10 AS NameBasedScore, -- 算术运算符 * 用法: 进行乘法计算。计算姓名长度乘以 10 作为得分
-
- -- CASE WHEN 示例: 根据年份对客户进行分类
- CASE -- CASE WHEN 用法: 基于条件返回不同的值, 像编程语言的 if/else if/else
- WHEN YEAR(join_date) = 2022 THEN 'Joined 2022' -- WHEN condition THEN result: 当 YEAR(join_date) 等于 2022 时, 返回 'Joined 2022'
- WHEN YEAR(join_date) = 2023 THEN 'Joined 2023' -- 另一个 WHEN 条件
- ELSE 'Joined Before 2022 or After 2023' -- ELSE result: 当所有 WHEN 条件都不满足时, 返回此值
- END AS JoinPeriodCategory -- 结果列别名
- FROM
- customers; -- 从 customers 表查询
- 🔍 代码逐句解析与技术说明:
- SELECT customer_id, customer_name, ...:
- SELECT: 指定要返回的列。这里选择原始列和多个通过计算衍生的新列。
- CONCAT(SUBSTRING(customer_name, 1, 3), '... (Length: ', LENGTH(customer_name), ')') AS FormattedName:
- CONCAT(string1, string2, ...): 用途: 将多个字符串、列值、计算结果连接成一个字符串。非常常用。
- SUBSTRING(string, start, length): 用途: 从一个字符串中提取指定长度的子串, 从指定位置开始。位置通常从 1 开始计数。在此例中提取姓名的前 3 个字符。
- LENGTH(string): 用途: 计算字符串的长度 (字符数)。不同数据库可能有 LEN() 函数实现相同功能。
- 这些函数组合起来, 创建了一个新的列 FormattedName, 格式为“姓名前 3 字符... (Length: 姓名总长度)”。AS FormattedName 指定列别名。
- YEAR(join_date) AS JoinYear, MONTH(join_date) AS JoinMonth:
- YEAR(date): 用途: 从日期或时间戳中提取年份。
- MONTH(date): 用途: 从日期或时间戳中提取月份。

- 类似函数还有 DAY(), HOUR(), MINUTE(), SECOND() 等, 用于提取日期/时间的不同部分。AS JoinYear, AS JoinMonth 指定列别名。
- DATEDIFF(CURDATE(), join_date) AS MembershipDaysTotal:
- DATEDIFF(date1, date2): **用途:** 计算 date1 和 date2 之间的天数差异 (date1 - date2)。
注意: 函数名和参数顺序在不同数据库中可能有差异 (例如 MySQL 是 DATEDIFF(date1, date2), SQL Server 是 DATEDIFF(day, date2, date1))。CURDATE() 函数用于获取当前日期 (函数名也可能不同, 如 GETDATE(), NOW())。
- 在此例中, 计算当前日期与客户加入日期之间的总天数, 代表会员的总天数。AS MembershipDaysTotal 指定别名。
- DATEDIFF(CURDATE(), join_date) / 365.25 AS MembershipYearsApprox:
- 算术运算符 /: **用途:** 执行除法计算。
- 将总天数除以 365.25 (近似一年天数) 来估算会员年限。AS MembershipYearsApprox 指定别名。
- DATEDIFF(CURDATE(), join_date) - 30 AS DaysMinus30:
- 算术运算符 -: **用途:** 执行减法计算。
- 计算会员总天数减去 30。AS DaysMinus30 指定别名。
- LENGTH(customer_name) * 10 AS NameBasedScore:
- 算术运算符 *: **用途:** 执行乘法计算。
- 将姓名长度乘以 10 作为一个简单的“得分”。AS NameBasedScore 指定别名。
- CASE WHEN YEAR(join_date) = 2022 THEN 'Joined 2022' ... END AS JoinPeriodCategory:
- CASE WHEN condition THEN result: **用途:** 根据设定的条件进行判断。如果 condition 为真, 则返回 THEN 后面的 result。可以有多个 WHEN...THEN 子句。
- ELSE result: **用途:** 如果之前所有的 WHEN 条件都不为真, 则返回 ELSE 后面的 result。ELSE 子句是可选的, 如果省略且所有 WHEN 都不满足, 结果将是 NULL。
- 在此例中, 根据客户加入的年份 (YEAR(join_date)) 将客户分为不同的加入时期类别。AS JoinPeriodCategory 指定别名。
- FROM customers::
- FROM: 指定数据来源表是 customers。

🔍 **预期输出:**

• (假设当前日期 CURDATE() 是 '2025-04-24')

• customer_id | customer_name | FormattedName | JoinYear | JoinMonth | MembershipDaysTotal | MembershipYearsApprox | DaysMinus30 | NameBasedScore | JoinPeriodCategory

• -----	-----	-----	-----	-----
---|-----

• 1 | Alice | Ali... (Length: 5) | 2022 | 1 | 1199 | 3.2827...
| 1169 | 50 | Joined 2022

• 2 | Bob | Bob... (Length: 3) | 2022 | 2 | 1164 | 3.1855...
| 1134 | 30 | Joined 2022

• 3 | Charlie | Cha... (Length: 7) | 2022 | 3 | 1130 | 3.0924...
| 1100 | 70 | Joined 2022

• 4 | David | Dav... (Length: 5) | 2022 | 4 | 1118 | 3.0605...

- | | | | | | | | |
|-----|-----------|--------------------|-------------|----------------------------------|-----|-----------|--|
| | 1088 | 50 | Joined 2022 | | | | |
| • 5 | Eve | Eve... (Length: 3) | 2023 | 5 | 715 | 1.9575... | |
| | 685 | 30 | Joined 2023 | | | | |
| • 6 | Frank | Fra... (Length: 5) | 2023 | 6 | 677 | 1.8535... | |
| | 647 | 50 | Joined 2023 | | | | |
| • 7 | Grace | Gra... (Length: 5) | 2023 | 7 | 633 | 1.7330... | |
| | 603 | 50 | Joined 2023 | | | | |
| • 8 | Wendy | Wen... (Length: 5) | 2024 | 1 | 479 | | |
| | 1.3115... | 449 | 50 | Joined Before 2022 or After 2023 | | | |
- (注: *MembershipDaysTotal* 和 *MembershipYearsApprox* 的具体数值会根据您执行查询时的实际当前日期变化。)
 - 这个详细的例子展示了如何在一个查询中综合运用 CASE WHEN、算术运算符、字符串函数和日期函数来创建多个衍生列, 对原始数据进行转换和丰富, 使其更适合后续分析和报告。

5. 窗口函数应用 (Window Function Applications)

窗口函数是 SQL 中进行复杂分析计算的强大工具。与常规聚合函数 (如 SUM(), AVG()) 会将多行数据折叠成一行不同, 窗口函数在计算时会考虑与当前行相关的“一个窗口”内的数据, 但结果仍然为原数据集中的每一行返回一个值。

应用模式: 窗口函数应用 (Window Function Applications)

- **目的:** 在不减少结果集行数的前提下, 执行排名、连续求和/平均、比较当前行与相邻行等分析计算。
- **常用技术:** OVER() 子句定义窗口, PARTITION BY 分组, ORDER BY 排序。配合不同的函数类型:
 - **排名函数:** ROW_NUMBER(), RANK(), DENSE_RANK() 等。
 - **聚合窗口函数:** SUM() OVER(), AVG() OVER(), COUNT() OVER(), MIN() OVER(), MAX() OVER() 等 (可以在窗口内进行聚合计算)。
 - **偏移函数:** LAG(), LEAD() 等 (获取窗口内前/后行的值)。
- **应用案例:** 深入分析客户的订单历史。对于每个订单, 我们想知道它在**该客户所有订单**中的排名 (按金额)、序列号 (按日期)、以及该客户截至当前订单日期的累计消费和平均消费, 同时比较当前订单与该客户前一个和后一个订单的金额。
- **涉及表格:** orders 表 (包含 order_id, customer_id, order_date, amount)。

首先, 请确保您已经执行了我们之前提供的 orders 表的建表和数据插入代码。

查询代码:

```
SQL
SELECT
  order_id, -- 选择订单 ID
  customer_id, -- 选择客户 ID
  order_date, -- 选择订单日期
  amount, -- 选择订单金额
```



```

-- === 排名函数示例 (通常 ORDER BY 度量值) ===
RANK() OVER (
    PARTITION BY customer_id -- PARTITION BY 用法: 将数据按 customer_id 分割成独立的
    分区, 窗口函数在每个分区内独立计算
    ORDER BY amount DESC -- ORDER BY 用法: 定义分区内行的顺序。排名函数依赖
    此顺序。这里按金额降序排。
) AS RankByAmount, -- 计算每个客户订单金额的排名 (并列跳过后续名次)

ROW_NUMBER() OVER (
    PARTITION BY customer_id -- 按客户 ID 分区
    ORDER BY order_date ASC -- 按订单日期升序排序
) AS OrderSequence, -- 计算每个客户订单按日期排序的序列号 (连续不跳号)

-- === 聚合窗口函数示例 (通常 ORDER BY 时间/序列) ===
SUM(amount) OVER (
    PARTITION BY customer_id -- 按客户 ID 分区
    ORDER BY order_date ASC -- 按订单日期升序排序
    -- ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW -- 窗口范围: 从
    分区开始到当前行 (累计求和/平均的默认范围, 可省略)
) AS CumulativeSum, -- 计算每个客户截至当前订单日期的累计订单总金额

AVG(amount) OVER (
    PARTITION BY customer_id -- 按客户 ID 分区
    ORDER BY order_date ASC -- 按订单日期升序排序
    -- ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW -- 窗口范围: 从
    分区开始到当前行
) AS CumulativeAvg, -- 计算每个客户截至当前订单日期的累计订单平均金额

-- === 偏移函数示例 (需要 ORDER BY) ===
LAG(amount, 1, 0) OVER ( -- LAG 函数用法: LAG(column, offset, default_value) 获取当前
    行上方 (offset) 行的 column 值, 若不存在返回 default_value
    PARTITION BY customer_id -- 按客户 ID 分区
    ORDER BY order_date ASC -- 按订单日期升序排序 (决定“上方”是哪一行)
) AS PreviousOrderAmount, -- 获取当前订单日期的前一个订单的金额 (同客户内)

LEAD(amount, 1, 0) OVER ( -- LEAD 函数用法: LEAD(column, offset, default_value) 获取
    当前行下方 (offset) 行的 column 值, 若不存在返回 default_value
    PARTITION BY customer_id -- 按客户 ID 分区
    ORDER BY order_date ASC -- 按订单日期升序排序 (决定“下方”是哪一行)
) AS NextOrderAmount -- 获取当前订单日期的后一个订单的金额 (同客户内)

FROM
    orders -- 从 orders 表查询

```

ORDER BY -- 最后对整个结果集进行排序，使输出更清晰

customer_id ASC,

order_date ASC;

🔍 代码逐句解析与技术说明：

1. SELECT order_id, customer_id, order_date, amount, ...:
 - SELECT: 指定要返回的列。这里既包含原始订单信息，也包含后面通过窗口函数计算出的新列。
2. RANK() OVER (...) AS RankByAmount:
 - RANK(): **用途:** 计算当前行在其分区内的排名。如果存在并列值，它们会得到相同的排名，然后下一个排名会跳过相应的数字。
 - OVER(...): **用途:** 标记一个函数是窗口函数。括号内的内容定义了“窗口”（即计算范围和顺序）。
 - PARTITION BY customer_id: **用途:** 将数据集分割成多个分区，窗口函数在每个分区（这里是每个客户）内独立进行计算，互不影响。
 - ORDER BY amount DESC: **用途:** 定义分区内行的逻辑顺序。对于 RANK(), 这个顺序决定了排名是按什么指标计算的（这里按 amount 降序）。
 - AS RankByAmount: 为排名结果列指定别名。
3. ROW_NUMBER() OVER (...) AS OrderSequence:
 - ROW_NUMBER(): **用途:** 为分区内的每一行分配一个唯一的、连续的整数序列号，从 1 开始。即使存在并列值，行号也不会跳跃。
 - OVER(...): 同上，定义窗口。
 - PARTITION BY customer_id: 按客户 ID 分区。
 - ORDER BY order_date ASC: 按订单日期升序排序。行号将按照订单发生的先后顺序分配。
 - AS OrderSequence: 别名。
4. SUM(amount) OVER (...) AS CumulativeSum:
 - SUM(列名) OVER(...): **用途:** 将聚合函数 SUM 用作窗口函数。它可以计算分区内所有行的总和，或者根据窗口框架计算一个**移动总和**或**累计总和**。
 - PARTITION BY customer_id: 按客户 ID 分区。
 - ORDER BY order_date ASC: 按订单日期升序排序。默认的窗口框架是从分区开始到当前行 (ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)。因此，这里计算的是每个客户截至当前订单日期的**累计总金额**。
 - AS CumulativeSum: 别名。
5. AVG(amount) OVER (...) AS CumulativeAvg:
 - AVG(列名) OVER(...): **用途:** 将聚合函数 AVG 用作窗口函数。计算分区内行的平均值。
 - PARTITION BY customer_id: 按客户 ID 分区。
 - ORDER BY order_date ASC: 按订单日期升序排序。默认窗口框架下，计算的是每个客户截至当前订单日期的**累计平均金额**。
 - AS CumulativeAvg: 别名。
6. LAG(amount, 1, 0) OVER (...) AS PreviousOrderAmount:
 - LAG(列名, offset, default_value): **用途:** 访问分区内当前行上方的行（由 offset 指定偏移量，1 表示紧邻上方一行）的指定列的值。如果上方不存在

- 相应的行，返回 default_value。
- PARTITION BY customer_id: 按客户 ID 分区。
 - ORDER BY order_date ASC: 决定了“上方”是指按订单日期排序后，比当前行更早的订单。
 - AS PreviousOrderAmount: 别名。
7. LEAD(amount, 1, 0) OVER (...) AS NextOrderAmount:
- LEAD(列名, offset, default_value): **用途:** 访问分区内当前行下方的行（由 offset 指定偏移量，1 表示紧邻下方一行）的指定列的值。如果下方不存在相应的行，返回 default_value。
 - PARTITION BY customer_id: 按客户 ID 分区。
 - ORDER BY order_date ASC: 决定了“下方”是指按订单日期排序后，比当前行更晚的订单。
 - AS NextOrderAmount: 别名。
8. FROM orders: 从 orders 表中获取数据。
9. ORDER BY customer_id ASC, order_date ASC: 最后对整个结果集进行排序，先按客户 ID 升序，再按订单日期升序，使输出结果的顺序逻辑清晰。这个 ORDER BY 作用于最终结果，与 OVER() 子句中的 ORDER BY 不同。

📊 预期输出:

(输出将按 customer_id 和 order_date 排序显示所有订单的明细行，并附加计算出的窗口函数结果)

order_id	customer_id	order_date	amount	RankByAmount	OrderSequence	CumulativeSum	CumulativeAvg	PreviousOrderAmount	NextOrderAmount
101	1	2023-01-15	150.00	3	1	150.00	150.000000		
			50.00					-- Alice 第 1 个订单	
103	1	2023-01-17	50.00	4	2	200.00	100.000000		
			450.75					-- Alice 第 2 个订单 (금액 기준 4 등)	
106	1	2023-06-01	450.75	2	3	650.75	216.916666		
			600.00					-- Alice 第 3 个订单	
111	1	2023-07-30	600.00	1	4	1250.75	312.687500		
			450.75					-- Alice 第 4 个订单 (금액 기준 1 등)	
104	2	2023-02-20	300.00	1	1	300.00	300.000000		
			180.00					-- Bob 第 1 个订单	
108	2	2023-06-10	180.00	3	2	480.00	240.000000		
			300.00					-- Bob 第 2 个订单 (금액 기준 3 등)	
113	2	2023-08-05	220.00	2	3	700.00	233.333333		
			180.00					-- Bob 第 3 个订单 (금액 기준 2 등)	
...	...	...	...	...	...	...	...	...	...

这个详细的例子展示了如何利用窗口函数在数据处理中执行复杂的分析任务，如分组内排名、累计计算和相邻行比较，这在没有窗口函数时是很难高效实现的。掌握窗口函数是提升 SQL 分析能力的关键一步。

6. 处理缺失数据 (Handling Missing Data)

- **目的:** 识别并处理数据集中的 NULL 值, 确保计算的准确性和结果的可读性。NULL 表示未知或缺失的数据。
- **常用技术:**
 - IS NULL, IS NOT NULL: 用于在 WHERE 子句中筛选 NULL 或非 NULL 的记录。
 - COALESCE(expr1, expr2, ..., exprN) 或 IFNULL(expr1, expr2) (部分数据库): 用于在 SELECT 列表中将 NULL 值替换为指定的默认值。
 - COUNT(列名) 与 COUNT(*) 的区别: COUNT(列名) 只计算指定列中非 NULL 值的行数, 而 COUNT(*) 计算所有行数, 包括含 NULL 的行。
- **应用案例:** 你的客户表中有些客户的城市信息是缺失的 (NULL)。你需要:

1. 找出城市信息缺失的客户。
2. 找出城市信息完整的客户。
3. 在显示客户列表时, 将缺失的城市信息显示为“未知城市”。
4. 了解总客户数和城市信息完整的客户数之间的差异。

- **涉及表格:** customers 表 (包含 customer_id, customer_name, city, 我们之前插入了城市为 NULL 的客户 ID 8)。

首先, 请确保您已经执行了我们之前包含客户 ID 8 的 customers 表的建表和数据插入代码。

查询代码 6.1: 查找缺失/完整的城市信息

SQL

-- 查找城市信息缺失的客户

SELECT

customer_id, -- 选择客户 ID
customer_name, -- 选择客户姓名
city -- 选择城市 (此处会是 NULL)

FROM

customers -- 从 customers 表查询

WHERE

city IS NULL; -- 使用 IS NULL 过滤条件, 只保留 city 列值为 NULL 的行

- **代码解释:**
 - WHERE city IS NULL: **用途:** IS NULL 用于检查指定列的值是否为 NULL。这是过滤 NULL 值的标准方法。
- **预期输出 6.1:**
- customer_id | customer_name | city
- -----|-----|-----
- 8 | Wendy | NULL

查询代码 6.2: 查找城市信息完整的客户

SQL

-- 查找城市信息完整的客户

SELECT

customer_id, -- 选择客户 ID

奕道远信息

```

customer_name, -- 选择客户姓名
city          -- 选择城市 (此处不会是 NULL)
FROM
customers     -- 从 customers 表查询
WHERE
city IS NOT NULL; -- 使用 IS NOT NULL 过滤条件，只保留 city 列值不为 NULL 的行

```

- **代码解释:**
 - WHERE city IS NOT NULL: **用途:** IS NOT NULL 用于检查指定列的值是否不为 NULL。
- **预期输出 6.2:**

customer_id	customer_name	city
1	Alice	New York
2	Bob	London
3	Charlie	New York
4	David	Paris
5	Eve	London
6	Frank	New York
7	Grace	London

查询代码 6.3: 显示时替换 NULL 值

```

SQL
-- 显示客户列表时，将缺失的城市信息显示为“未知城市”
SELECT
customer_id, -- 选择客户 ID
customer_name, -- 选择客户姓名
city AS OriginalCity, -- 选择原始城市列，命名为 OriginalCity
COALESCE(city, '未知城市') AS DisplayCity -- 使用 COALESCE 将 city 列的 NULL 值替换为 '未知城市'
FROM
customers; -- 从 customers 表查询

```

- **代码解释:**
 - COALESCE(expr1, expr2, ..., exprN): **用途:** 返回参数列表中第一个非 NULL 的表达式值。如果所有表达式都为 NULL，则返回 NULL。
 - COALESCE(city, '未知城市'): 如果 city 的值不是 NULL，就返回 city 的值；如果 city 的值是 NULL，就返回 '未知城市'。
 - IFNULL(expr1, expr2): 部分数据库（如 MySQL）使用的函数，用途与 COALESCE(expr1, expr2) 相同。

- **预期输出 6.3:**

customer_id	customer_name	OriginalCity	DisplayCity
1	Alice	New York	New York
2	Bob	London	London

- | | | | |
|---|---------|----------|----------|
| 3 | Charlie | New York | New York |
| 4 | David | Paris | Paris |
| 5 | Eve | London | London |
| 6 | Frank | New York | New York |
| 7 | Grace | London | London |
| 8 | Wendy | NULL | 未知城市 |

查询代码 6.4: 统计包含/不包含 NULL 的数量

SQL

-- 统计总客户数和城市信息完整的客户数

SELECT

COUNT(*) AS TotalCustomerCount, -- 统计所有行数

COUNT(city) AS CustomersWithCity -- 统计 city 列非 NULL 的行数

FROM

customers; -- 从 customers 表查询

- **代码解释:**
 - COUNT(*): **用途:** 计算结果集中的总行数, 包括任何列为 NULL 的行。
 - COUNT(列名): **用途:** 计算指定列中非 **NULL 值** 的行数。
 - 通过对比这两个计数, 可以知道有多少行的 city 列是 NULL。
- **预期输出 6.4:**
- TotalCustomerCount | CustomersWithCity
- -----|-----
- 8 | 7
- **处理缺失数据总结:** 识别 NULL 值是数据清洗的第一步。IS NULL/IS NOT NULL 用于过滤, COALESCE/IFNULL 用于显示或计算时的替换。同时要注意聚合函数在处理 NULL 时的行为差异。

7. 数据去重 (Deduplication)

- **目的:** 识别并移除数据集中的重复记录, 确保数据的唯一性。重复的定义取决于业务场景, 可能是所有列都相同, 也可能是部分关键列相同。
- **常用技术:**
 - DISTINCT: 用于 SELECT 语句, 返回指定列的唯一组合。
 - GROUP BY 结合 COUNT(*)>1: 用于找出哪些组合出现了多次 (即存在重复)。
 - 窗口函数 (ROW_NUMBER()) 结合子查询/CTE 和 DELETE: 一种常用的删除重复行的方法, 保留每组重复中的第一条 (或根据其他标准)。
 - 自连接结合 DELETE: 另一种删除重复的方法。
- **应用案例:** 你有一个包含客户注册信息的表, 由于系统或录入错误, 导致有些客户信息被重复记录。你需要找出重复的客户记录, 并删除重复项, 只保留每组重复中的一条。
- **涉及表格:** 我们创建一个新的简单表 Customers_Duplicate 来演示。

首先，创建并填充一个包含重复数据的示例表：

SQL

-- 删除旧的重复演示表（如果存在）

DROP TABLE IF EXISTS Customers_Duplicate;

-- 创建重复客户表

```
CREATE TABLE Customers_Duplicate (  
    cust_dup_id INT PRIMARY KEY, -- 唯一 ID，用于区分重复行  
    customer_name VARCHAR(50),  
    city VARCHAR(50),  
    registration_date DATE  
);
```

-- 插入包含重复的示例数据

```
INSERT INTO Customers_Duplicate (cust_dup_id, customer_name, city, registration_date)  
VALUES  
(1, 'Alice', 'New York', '2022-01-10'),  
(2, 'Bob', 'London', '2022-02-15'),  
(3, 'Alice', 'New York', '2022-01-10'), -- 与 ID 1 重复  
(4, 'Charlie', 'Paris', '2022-03-20'),  
(5, 'Bob', 'London', '2022-02-15'), -- 与 ID 2 重复  
(6, 'Alice', 'New York', '2022-01-10'), -- 与 ID 1, 3 重复  
(7, 'David', 'New York', '2022-04-01'); -- 不重复
```

查询代码 7.1：查看去重后的唯一组合

SQL

-- 查看去重后的客户信息列表

SELECT DISTINCT -- 使用 DISTINCT 关键字

customer_name, -- 指定要考虑去重的列

city,

registration_date

FROM

Customers_Duplicate; -- 从包含重复数据的表中查询

- **代码解释：**

- DISTINCT 列 1, 列 2, ...: **用途：** DISTINCT 关键字放在 SELECT 后面，应用于 SELECT 后面的所有列的组合。它会返回指定列组合的唯一行，即如果多行的这些列的值完全相同，只保留一行。

- **预期输出 7.1：**

- customer_name | city | registration_date

- -----|-----|-----

- Alice | New York | 2022-01-10 -- 原表中有多条 Alice/New York/2022-01-10

- Bob | London | 2022-02-15 -- 原表中有多条 Bob/London/2022-02-15

- Charlie | Paris | 2022-03-20
- David | New York | 2022-04-01

查询代码 7.2: 找出存在重复的记录组合及其次数

SQL

-- 找出 customer_name, city, registration_date 组合出现次数多于 1 的记录

SELECT

customer_name, -- 选择重复组合的列

city,

registration_date,

COUNT(*) AS DuplicateCount -- 计算该组合出现的次数

FROM

Customers_Duplicate -- 从表中查询

GROUP BY

customer_name, -- 按这三列组合进行分组

city,

registration_date

HAVING

COUNT(*) > 1; -- 过滤分组, 只保留出现次数大于 1 的分组

- **代码解释:**
 - GROUP BY 列 1, 列 2, ...: 按指定列的组合进行分组。
 - COUNT(*): 计算每个分组中的行数。
 - HAVING COUNT(*) > 1: **用途:** HAVING 用于过滤 GROUP BY 后的分组。这里筛选出那些行数大于 1 的分组, 即存在重复的组合。

- **预期输出 7.2:**

- customer_name | city | registration_date | DuplicateCount

- -----|-----|-----|-----

- Alice | New York | 2022-01-10 | 3

- Bob | London | 2022-02-15 | 2

查询代码 7.3: 删除重复行 (保留每组重复中的一条)

- **目的:** 从 Customers_Duplicate 表中删除重复行, 只保留 cust_dup_id 最小 (或者其他规则, 如 ROW_NUMBER = 1) 的那一条。
- **常用方法之一 (使用 ROW_NUMBER):** 给每一组重复的行按某个规则 (如 ID) 排序并分配行号, 然后删除行号大于 1 的行。

SQL

-- 使用 CTE 和 ROW_NUMBER 删除重复行 (保留 cust_dup_id 最小的那条)

-- 注意: DELETE 语句会永久修改数据, 请在测试环境或备份数据上运行!

WITH RankedDuplicates AS (-- 定义一个 CTE 名为 RankedDuplicates

SELECT

cust_dup_id, -- 选择唯一 ID

奕道远信息


```

customer_name, -- 选择用于识别重复的列
city,
registration_date,
ROW_NUMBER() OVER ( -- 使用 ROW_NUMBER 窗口函数
    PARTITION BY customer_name, city, registration_date -- 按这三列组合分区 (重复的
行会在同一个分区)
    ORDER BY cust_dup_id ASC -- 在每个分区内按 cust_dup_id 升序排序 (ID 最小的排
第 1)
) as rn -- 分配行号, rn = 1 表示每组重复中的第一行 (ID 最小的)
FROM
Customers_Duplicate -- 从原始表查询
) -- CTE 定义结束

```

```

-- 方法 1: DELETE 使用 CTE (并非所有数据库直接支持 DELETE FROM CTE)
-- DELETE FROM RankedDuplicates WHERE rn > 1;

```

```

-- 方法 2: DELETE 使用子查询 (更通用)
DELETE FROM Customers_Duplicate -- 从原始表中删除
WHERE cust_dup_id IN ( -- 删除 cust_dup_id 在以下子查询结果中的行
    SELECT cust_dup_id -- 子查询选择需要删除的 ID
    FROM RankedDuplicates -- 从 CTE RankedDuplicates 中选择
    WHERE rn > 1 -- 过滤条件: 行号大于 1 的都是重复行 (非第一条)
);

```

```

-- 执行 DELETE 后, 可以再次运行 SELECT * FROM Customers_Duplicate; 来验证结果
-- 预期 Customers_Duplicate 表中的数据将变为 (顺序可能不同):
-- cust_dup_id | customer_name | city | registration_date

```

```

-- -----|-----|-----|-----
-- 1      | Alice      | New York | 2022-01-10    -- 原 ID 1, 3, 6 中 ID 最小的
-- 2      | Bob        | London  | 2022-02-15    -- 原 ID 2, 5 中 ID 最小的
-- 4      | Charlie    | Paris   | 2022-03-20
-- 7      | David      | New York | 2022-04-01

```

- **代码解释:**

- WITH RankedDuplicates AS (...): 定义一个临时结果集 RankedDuplicates。
- ROW_NUMBER() OVER (PARTITION BY customer_name, city, registration_date ORDER BY cust_dup_id ASC): 在 RankedDuplicates CTE 中计算。它按 customer_name, city, registration_date 的组合对行进行分组, 并在每个组内按 cust_dup_id 升序分配一个从 1 开始的连续行号 (rn)。这样, 每组重复的行中, rn=1 的就是 cust_dup_id 最小的那条。
- DELETE FROM Customers_Duplicate WHERE cust_dup_id IN (...): 这是实际执行删除的语句。它从原始表中删除记录。
- SELECT cust_dup_id FROM RankedDuplicates WHERE rn > 1: 子查询从 CTE 中选择所有 rn 大于 1 的行的 cust_dup_id。这些 ID 对应的行就是每组重复中除了第一条以外的其他行, 也就是需要删除的重复项。

- **其他删除方法:** 也可以使用 GROUP BY 和自连接来删除, 例如找到每组重复的最小 ID, 然后删除那些 ID 不在最小 ID 列表中的行。使用 ROW_NUMBER() 是相对通用和易于理解的一种。
 - **数据去重总结:** DISTINCT 快速查看唯一组合; GROUP BY + HAVING 识别重复组合; 删除重复需要更复杂的操作, 常用方法是利用窗口函数 ROW_NUMBER 或基于聚合的自连接/子查询来找出要删除的行的唯一标识符。
-

8. 数据透视 (Pivoting Data)

- **目的:** 将数据的行转换为列。通常用于将某个分类维度的唯一值变成结果集中的新列, 并显示这些新列与另一个维度的交叉聚合值。
- **常用技术:**
 - 使用 CASE WHEN 语句结合聚合函数 (SUM, COUNT 等) 和 GROUP BY 子句。这种方法是标准 SQL, 兼容性好。
 - 数据库特定的 PIVOT 或 CROSSTAB 语法。语法简洁但仅限于特定数据库 (如 SQL Server 的 PIVOT, PostgreSQL 的 CROSSTAB 扩展)。
- **应用案例:** 按月份统计每个产品类别 (如 'Electronics', 'Books', 'Furniture', 'Misc') 的总销售金额。我们希望结果集的每一行代表一个月份, 每一列代表一个产品类别, 交叉点是该月份该类别的总销售金额。
- **涉及表格:** orders 表 (获取订单日期), order_items 表 (获取销售金额), products 表 (获取产品类别)。我们将使用基于 CASE WHEN 的标准 SQL 方法。

首先, 请确保您已经执行了我们之前包含产品 ID 11 ('Misc' 类别) 的 products 表, 以及 orders 和 order_items 表的建表和数据插入代码。

查询代码 8.1: 使用 CASE WHEN 进行数据透视

SQL

SELECT

YEAR(o.order_date) AS OrderYear, -- 选取订单年份作为结果行的第一列

MONTH(o.order_date) AS OrderMonth, -- 选取订单月份作为结果行的第二列

-- 使用 CASE WHEN 和 SUM 实现透视: 将 Electronics 类别的销售金额聚合到一列

SUM(CASE WHEN p.category = 'Electronics' THEN oi.item_sale_amount ELSE 0 END) AS ElectronicsSales,

-- CASE WHEN 用法: 当 p.category 等于 'Electronics' 时, 取 oi.item_sale_amount 的值, 否则取 0

-- SUM() 用法: 在 GROUP BY 后的每个分组 (每个月份) 内, 对 CASE WHEN 返回的值求和。这样就只累加了 Electronics 类别的销售金额。

-- 对 Books 类别进行透视

SUM(CASE WHEN p.category = 'Books' THEN oi.item_sale_amount ELSE 0 END) AS BooksSales,

-- 对 Furniture 类别进行透视

奕道远信息

```

SUM(CASE WHEN p.category = 'Furniture' THEN oi.item_sale_amount ELSE 0 END) AS
FurnitureSales,
-- 对 Misc 类别进行透视 (如果数据中存在)
SUM(CASE WHEN p.category = 'Misc' THEN oi.item_sale_amount ELSE 0 END) AS
MiscSales

FROM
    order_items oi -- 从订单明细表查询
INNER JOIN
    products p ON oi.product_id = p.product_id -- 与产品表连接, 获取产品类别
INNER JOIN
    orders o ON oi.order_id = o.order_id -- 与订单表连接, 获取订单日期
GROUP BY
    YEAR(o.order_date), -- 按订单年份分组
    MONTH(o.order_date) -- 再按订单月份分组, 结果的每一行代表一个年月
ORDER BY -- 最后按年份和月份排序结果
    OrderYear ASC,
    OrderMonth ASC;

```

- **代码解释:**

- FROM ... JOIN ... JOIN ...: 连接三个表, 获取订单明细、产品类别和订单日期。
- GROUP BY YEAR(o.order_date), MONTH(o.order_date): 将数据按订单的年份和月份进行分组。最终结果的每一行对应一个年月。
- SUM(CASE WHEN p.category = '...' THEN oi.item_sale_amount ELSE 0 END) AS ...: 这是透视的核心。在每个年月分组内:
 - CASE WHEN p.category = 'Electronics' THEN oi.item_sale_amount ELSE 0 END: 对于该分组内的每一行订单明细, 如果其产品类别是 'Electronics', CASE 返回该明细项的销售金额; 否则返回 0。
 - SUM(...): 对 CASE 语句为该分组 (该年月) 返回的所有值求和。由于非 'Electronics' 的项返回 0, 这个求和结果就是该年月内所有 'Electronics' 类别产品的总销售额。
 - 重复这个模式, 为需要透视的每个类别创建一个 SUM(CASE WHEN ...) 列。
- AS ...Sales: 为透视出的新列指定别名。
- SELECT YEAR(...) AS OrderYear, MONTH(...) AS OrderMonth, ...: 选择年份和月份作为行标识, 以及透视出的销售额列。

- **预期输出 8.1:** (基于我们插入的数据)

- 一月 2023 (Orders 101, 102, 103): 101(Laptop, Mouse), 102(Python Book, SQL Book, Novel), 103(Desk Lamp).
 - Electronics: $800.00 + 50.00 = 850.00$
 - Books: $60.00 + 45.00 + 45.00 = 150.00$
 - Furniture: 35.00
 - Misc: 0
- 二月 2023 (Order 104): 104(Laptop, Keyboard)
 - Electronics: $800.00 + 75.00 = 875.00$

- Books: 0
- Furniture: 0
- Misc: 0
- 五月 2023 (Order 105): 105(Desk Chair)
 - Electronics: 0
 - Books: 0
 - Furniture: 150.00
 - Misc: 0
- 六月 2023 (Orders 106, 107, 108, 109): 106(Laptop, Monitor), 107(Novel), 108(Keyboard), 109(SQL Book)
 - Electronics: $800.00 + 300.00 + 150.00 = 1250.00$
 - Books: $75.00 + 45.00 = 120.00$
 - Furniture: 0
 - Misc: 0
- 七月 2023 (Orders 110, 111): 110(Headphones), 111(Laptop, Monitor)
 - Electronics: $90.00 + 800.00 + 300.00 = 1190.00$
 - Books: 0
 - Furniture: 0
 - Misc: 0
- 八月 2023 (Orders 112, 113, 114, 115): 112(Mouse), 113(Keyboard), 114(Desk Chair), 115(Headphones)
 - Electronics: $25.00 + 225.00 + 180.00 = 430.00$
 - Books: 0
 - Furniture: 150.00
 - Misc: 0

• OrderYear | OrderMonth | ElectronicsSales | BooksSales | FurnitureSales | MiscSales

• -----|-----|-----|-----|-----|-----|

• 2023	1	850.00	150.00	35.00	0.00
• 2023	2	875.00	0.00	0.00	0.00
• 2023	5	0.00	0.00	150.00	0.00
• 2023	6	1250.00	120.00	0.00	0.00
• 2023	7	1190.00	0.00	0.00	0.00
• 2023	8	430.00	0.00	150.00	0.00

(注: 由于产品 ID 11 ('Misc') 没有销售记录, MiscSales 列全为 0。如果 Misc 有销售记录, 该列会显示对应月份的销售额。)

• **数据库特定 PIVOT 语法 (简述):**

- 不同数据库有自己的 PIVOT 语法, 通常更简洁, 但也牺牲了兼容性。
- **SQL Server 示例:**

SQL

SELECT OrderYear, OrderMonth, [Electronics], [Books], [Furniture], [Misc] -- 列出透视后的列名

FROM (

-- 准备基础数据: 年份, 月份, 类别, 销售额

奕道远信息

```

SELECT YEAR(o.order_date) AS OrderYear, MONTH(o.order_date) AS OrderMonth,
       p.category, oi.item_sale_amount
FROM order_items oi JOIN products p ON oi.product_id = p.product_id JOIN orders o
ON oi.order_id = o.order_id
) AS SourceData -- 源数据需要有别名
PIVOT (
    SUM(item_sale_amount) -- 指定聚合函数和聚合的列
    FOR category IN ([Electronics], [Books], [Furniture], [Misc]) -- 指定要透视的列 (category)
    及其可能的取值, 这些取值成为新列名
) AS PivotTable; -- 透视结果也需要别名
    
```

- 这种语法更直接, 但需要熟悉具体数据库的实现。
- **数据透视总结:** 数据透视是改变数据结构的常用手段, 将分类值转化为列。CASE WHEN 结合聚合是标准 SQL 方法, 数据库特定语法更简洁。

9. 逆透视 (Unpivoting Data)

- **目的:** 将数据的列转换为行。当你有多个列存储的是同一种属性 (如不同季度的销售额、不同年份的收入), 但作为独立列存在时, 可以使用逆透视将其转换成更规范的“长”格式, 即一列表示属性类型 (如“季度”), 另一列表示对应的值 (如“销售额”)。这通常是为了方便后续的过滤、分组或可视化。
- **常用技术:**
 - UNION ALL: 标准 SQL 方法, 通过多次 SELECT 并用 UNION ALL 连接, 将不同列的数据堆叠起来。
 - 数据库特定的 UNPIVOT 语法: 语法更简洁, 但兼容性较差。
- **应用案例:** 你有一个简单的报表, 显示了不同产品类别的季度销售额, 每个季度是一个独立的列。你希望将它转换成每行包含“类别”、“季度”和“销售额”三列的明细格式。
- **涉及表格:** 我们创建一个新的简单表 QuarterlySales 来演示。

首先, 创建并填充一个包含季度销售额列的示例表:

SQL

```
-- 删除旧的季度销售演示表 (如果存在)
```

```
DROP TABLE IF EXISTS QuarterlySales;
```

```
-- 创建季度销售表 (已透视格式)
```

```
CREATE TABLE QuarterlySales (
    SaleYear INT,
    ProductCategory VARCHAR(50),
    Q1_Sales DECIMAL(10, 2),
    Q2_Sales DECIMAL(10, 2),
    Q3_Sales DECIMAL(10, 2),
    Q4_Sales DECIMAL(10, 2)
);
```

-- 插入示例数据

```
INSERT INTO QuarterlySales (SaleYear, ProductCategory, Q1_Sales, Q2_Sales, Q3_Sales,
Q4_Sales) VALUES
(2023, 'Electronics', 15000.00, 18000.00, 22000.00, 25000.00),
(2023, 'Books', 3000.00, 2500.00, 3200.00, 2800.00),
(2024, 'Electronics', 16000.00, 19000.00, NULL, 26000.00), -- 假设 Q3 数据缺失
(2024, 'Furniture', 5000.00, 5500.00, 6000.00, 6200.00);
```

查询代码 9.1: 使用 UNION ALL 进行逆透视

SQL

SELECT

```
SaleYear,      -- 选择年份
ProductCategory, -- 选择产品类别
'Q1' AS Quarter, -- 创建季度列, 值为 'Q1'
Q1_Sales AS Sales -- 选择 Q1_Sales 列, 命名为 Sales
```

FROM

```
QuarterlySales -- 从表中查询 Q1 数据
```

WHERE

```
Q1_Sales IS NOT NULL -- 排除 NULL 值 (如果需要包含 NULL 值行, 可以移除此 WHERE 子句)
```

UNION ALL -- 使用 UNION ALL 连接结果集, 保留所有行

SELECT

```
SaleYear,
ProductCategory,
'Q2' AS Quarter, -- 创建季度列, 值为 'Q2'
Q2_Sales AS Sales -- 选择 Q2_Sales 列, 命名为 Sales
```

FROM

```
QuarterlySales -- 从表中查询 Q2 数据
```

WHERE

```
Q2_Sales IS NOT NULL -- 排除 NULL 值
```

UNION ALL -- 连接结果集

SELECT

```
SaleYear,
ProductCategory,
'Q3' AS Quarter, -- 创建季度列, 值为 'Q3'
Q3_Sales AS Sales -- 选择 Q3_Sales 列, 命名为 Sales
```

FROM

```
QuarterlySales -- 从表中查询 Q3 数据
```

奕道远信息

WHERE

Q3_Sales IS NOT NULL -- 排除 NULL 值

UNION ALL -- 连接结果集

SELECT

SaleYear,

ProductCategory,

'Q4' AS Quarter, -- 创建季度列, 值为 'Q4'

Q4_Sales AS Sales -- 选择 Q4_Sales 列, 命名为 Sales

FROM

QuarterlySales -- 从表中查询 Q4 数据

WHERE

Q4_Sales IS NOT NULL; -- 排除 NULL 值

- **代码解释:**

- UNION ALL: **用途:** 用于合并两个或多个 SELECT 语句的结果集。UNION ALL 会简单地堆叠所有结果, 包括重复行。如果使用 UNION (不带 ALL), 则会自动去除重复行。
- 每个 SELECT 语句都从原始表 (QuarterlySales) 中选取 SaleYear 和 ProductCategory 列, 以及一个特定的季度销售额列 (如 Q1_Sales)。
- 'Q1' AS Quarter: 在每个 SELECT 中, 硬编码一个字符串 (如 'Q1') 作为新的 Quarter 列的值, 以此标识原始数据来自哪个季度列。
- Q1_Sales AS Sales: 将原始季度销售额列重命名为统一的 Sales 列。
- WHERE Q1_Sales IS NOT NULL: **用途:** 在每个 SELECT 中过滤掉对应季度销售额为 NULL 的行。如果你需要包含 NULL 值 (即表示该季度没有销售数据), 可以移除这些 WHERE 子句。
- 通过多次 SELECT 和 UNION ALL, 将原本分散在 Q1_Sales, Q2_Sales, Q3_Sales, Q4_Sales 四列中的数据, 堆叠到统一的 Sales 列下, 并用 Quarter 列进行区分。

- **预期输出 9.1:** (行的顺序可能因数据库实现而异, 但内容应包含以下非 NULL 数据)

- SaleYear | ProductCategory | Quarter | Sales

- -----|-----|-----|-----

- 2023 | Electronics | Q1 | 15000.00

- 2023 | Books | Q1 | 3000.00

- 2024 | Electronics | Q1 | 16000.00

- 2024 | Furniture | Q1 | 5000.00

- 2023 | Electronics | Q2 | 18000.00

- 2023 | Books | Q2 | 2500.00

- 2024 | Electronics | Q2 | 19000.00

- 2024 | Furniture | Q2 | 5500.00

- 2023 | Electronics | Q3 | 22000.00

- 2023 | Books | Q3 | 3200.00

- 2024 | Furniture | Q3 | 6000.00 -- 2024 Electronics Q3 是 NULL, 被 WHERE IS NOT NULL 移除了

尖道迅信息

- 2023 | Electronics | Q4 | 25000.00
- 2023 | Books | Q4 | 2800.00
- 2024 | Electronics | Q4 | 26000.00
- 2024 | Furniture | Q4 | 6200.00
- **数据库特定 UNPIVOT 语法 (简述):**
 - 某些数据库提供 UNPIVOT 语法，它通常与 PIVOT 相反。语法 varies。
 - **SQL Server 示例:**

SQL

SELECT SaleYear, ProductCategory, Quarter, Sales

FROM QuarterlySales

UNPIVOT (

Sales FOR Quarter IN (Q1_Sales, Q2_Sales, Q3_Sales, Q4_Sales) -- 将 Q1_Sales, Q2_Sales... 的值放入 Sales 列，列名放入 Quarter 列

) AS UnpivotedSales; -- 逆透视结果的别名

- 这种方法通常更简洁，但不如 UNION ALL 通用。
- **逆透视总结:** 逆透视是数据重塑的重要技术，将“宽”格式数据转换为“长”格式，方便按属性类型进行分组分析。UNION ALL 是最通用的实现方法。

10. 分解复杂逻辑 (Breaking Down Complex Logic)

- **目的:** 将一个复杂的查询任务分解为多个更小、更易于理解、测试和管理的步骤。这有助于提高查询的可读性、可维护性，有时还能提高性能。
- **常用技术:**
 - 子查询 (Subqueries): 在 WHERE, SELECT, FROM 子句中嵌套另一个 SELECT 查询。
 - CTE (Common Table Expressions): 使用 WITH 子句定义临时的、命名的结果集，可以在主查询或其他 CTE 中引用。CTE 通常比子查询更具可读性，特别是在多层嵌套或需要多次引用的情况下。
- **应用案例:** 找出平均订单金额高于所有客户平均订单金额的客户。这需要至少两个计算步骤：1. 计算所有客户的平均订单金额；2. 计算每个客户的平均订单金额；3. 比较每个客户的平均金额与所有客户的平均金额。
- **涉及表格:** orders 表。我们将使用 CTE 来分解这个逻辑。

首先，请确保您已经执行了我们之前包含所有订单数据的 orders 表的建表和数据插入代码。

查询代码 10.1: 使用 CTE 分解复杂逻辑

SQL

WITH OverallOrderAverage AS (-- 定义第一个 CTE，计算所有订单的平均金额

SELECT

AVG(amount) AS AverageAmount -- 计算 amount 列的平均值

FROM

orders -- 从 orders 表查询

), -- CTE 定义结束，用逗号分隔多个 CTE

奕道远信息


```

CustomerOrderAverages AS ( -- 定义第二个 CTE，计算每个客户的平均订单金额
    SELECT
        customer_id, -- 选择客户 ID
        AVG(amount) AS CustomerAverageAmount -- 计算每个客户的平均订单金额
    FROM
        orders -- 从 orders 表查询
    GROUP BY
        customer_id -- 按客户 ID 分组
) -- 第二个 CTE 定义结束

-- 主查询：结合两个 CTE 的结果进行过滤
SELECT
    coa.customer_id, -- 从 CustomerOrderAverages CTE 中选择客户 ID (使用别名 coa)
    coa.CustomerAverageAmount -- 选择客户的平均订单金额
FROM
    CustomerOrderAverages coa -- 从 CustomerOrderAverages CTE 查询，并使用别名 coa
CROSS JOIN
    OverallOrderAverage ooa -- 与 OverallOrderAverage CTE 进行交叉连接 (因为 ooa 只返回一行，交叉连接相当于附加这一行到 coa 的每一行)
WHERE
    coa.CustomerAverageAmount > ooa.AverageAmount; -- 过滤条件：客户平均金额大于总平均金额

```

- **代码解释:**

- WITH CTE_Name AS (...): **用途:** 定义一个 CTE。CTE 是一个临时的、命名的结果集，其生命周期仅限于紧随其后的 SQL 语句（通常是 SELECT, INSERT, UPDATE, DELETE）。
- OverallOrderAverage AS (...): 第一个 CTE，计算出整个 orders 表的平均金额，结果只有一行一列。
- CustomerOrderAverages AS (...): 第二个 CTE，计算出每个 customer_id 对应的平均订单金额，结果是每个客户一行。
- SELECT ... FROM CustomerOrderAverages coa CROSS JOIN OverallOrderAverage ooa: 主查询从这两个 CTE 中获取数据。因为 OverallOrderAverage 只有一个结果 (AverageAmount)，这里使用 CROSS JOIN 简单地将这个总平均值附加到 CustomerOrderAverages 的每一行旁边，方便比较。
- WHERE coa.CustomerAverageAmount > ooa.AverageAmount: **用途:** 在主查询中使用 WHERE 子句，过滤出客户平均订单金额 (coa.CustomerAverageAmount) 大于所有订单平均金额 (ooa.AverageAmount) 的客户记录。
- 子查询用法示例 (等效于上述 CTE，但可读性可能稍差):

```

SQL
SELECT
    customer_id, -- 选择客户 ID

```

```

AVG(amount) AS CustomerAverageAmount -- 计算每个客户的平均金额
FROM
  orders -- 从 orders 表查询
GROUP BY
  customer_id -- 按客户 ID 分组
HAVING -- 使用 HAVING 过滤分组
  AVG(amount) > ( -- HAVING 条件: 客户平均金额大于以下子查询的结果
    SELECT AVG(amount) -- 子查询: 计算所有订单的平均金额
    FROM orders -- 从 orders 表查询
  ); -- 子查询结束

```

- 这里的子查询用在 HAVING 中，计算并返回一个标量值（单个数值），用于与 AVG(amount) 的结果进行比较。
- 预期输出 10.1:**
 - 首先计算所有订单的平均金额: $SUM(amount) = 2931.25$, $COUNT(*) = 15$ 。总平均 $= 2931.25 / 15 \approx 195.4167$ 。
 - 计算每个客户平均金额 (来自之前案例 6):
 - 客户 1: $1250.75 / 4 = 312.6875 (> 195.4167)$
 - 客户 2: $700.00 / 3 = 233.3333 (> 195.4167)$
 - 客户 3: $440.50 / 3 = 146.8333 (< 195.4167)$
 - 客户 5: $360.00 / 3 = 120.0000 (< 195.4167)$
 - 客户 7: $560.00 / 2 = 280.0000 (> 195.4167)$
- customer_id | CustomerAverageAmount
- |-----
- 1 | 312.687500
- 2 | 233.333333
- 7 | 280.000000
- 分解复杂逻辑总结:** 子查询和 CTE 是分解复杂 SQL 逻辑的重要工具。CTE 通常提供更好的可读性和结构，尤其是在多步计算或需要多次引用中间结果时。

11. 查找序列中的“缺失”或“间隔” (Identifying Gaps)

- 目的:** 在具有序列性（如日期、ID）的数据中，找出缺失的记录或不连续的点。
- 常用技术:**
 - 窗口函数 (LAG(), LEAD()) 结合日期/ID 差异检查: 检查当前记录与前一条记录（按序列排序后）之间的间隔是否大于预期。
 - 生成完整的序列（使用递归 CTE 或数据库特定函数）与原始数据进行 LEFT JOIN: 查找在完整序列中存在但在原始数据中不存在的点。这是查找所有缺失点的有效方法。
 - 自连接结合序列检查: 另一种方法，但实现可能较复杂。
- 应用案例:** 你有一个记录了用户每天签到日志的表，只包含签到的日期。你需要找出用户在某个日期范围内没有签到的日期。
- 涉及表格:** 我们创建一个新的简单表 UserSignins 来演示。

首先，创建并填充一个包含用户签到日期，但存在间隔的示例表：

SQL

-- 删除旧的签到演示表（如果存在）

DROP TABLE IF EXISTS UserSignins;

-- 创建用户签到表

CREATE TABLE UserSignins (

 user_id INT,

 signin_date DATE PRIMARY KEY -- 假设每天最多签到一次，日期为主键

);

-- 插入示例签到数据 (存在间隔)

INSERT INTO UserSignins (user_id, signin_date) VALUES

(101, '2023-01-01'),

(101, '2023-01-02'),

(101, '2023-01-04'), -- 缺少 2023-01-03

(101, '2023-01-05'),

(101, '2023-01-07'), -- 缺少 2023-01-06

(101, '2023-01-08'),

(101, '2023-01-10'); -- 缺少 2023-01-09

-- (假设我们关注的日期范围是 2023-01-01 到 2023-01-10)

查询代码 11.1：使用 LAG() 查找间隔的开始日期

SQL

-- 查找用户签到记录中，日期与前一天不连续的记录 (标记间隔的开始日期)

SELECT

 user_id, -- 选择用户 ID

 signin_date, -- 选择当前签到日期

 LAG(signin_date, 1, signin_date - INTERVAL '1 day') OVER (-- LAG 用法: 获取按日期排序的前一条记录的日期

 PARTITION BY user_id -- 按用户 ID 分区

 ORDER BY signin_date ASC -- 按签到日期升序排序

) AS PreviousSignInDate -- 前一个签到日期 (如果不存在前一个，例如第一个日期，则返回当前日期减去 1 天)

FROM

 UserSignins -- 从签到表查询

QUALIFY signin_date - PreviousSignInDate > INTERVAL '1 day'; -- QUALIFY 用法 (部分数据库支持，如 Teradata, Snowflake): 过滤窗口函数结果，这里筛选当前日期与前一个签到日期的差大于 1 天的记录

-- 或者使用子查询 (更通用):

/*

SELECT user_id, signin_date, PreviousSignInDate

FROM (

奕道远信息

```

SELECT
    user_id, signin_date,
    LAG(signin_date, 1, signin_date - INTERVAL '1 day') OVER (PARTITION BY user_id
ORDER BY signin_date ASC) AS PreviousSignInDate
FROM UserSignins
) AS T
WHERE T.signin_date - T.PreviousSignInDate > INTERVAL '1 day';
*/

```

- **代码解释:**

- LAG(signin_date, 1, signin_date - INTERVAL '1 day') OVER (...): 对于每一行签到记录，获取按日期排序的其前一条记录的签到日期。如果当前是该用户的第一条记录 (LAG 没有前一条)，则返回 signin_date - INTERVAL '1 day' (例如，如果第一条是 2023-01-01，这个默认值就是 2022-12-31)。INTERVAL '1 day' 是数据库中表示时间间隔的语法。
- signin_date - PreviousSignInDate > INTERVAL '1 day': 计算当前日期与前一个签到日期的差。如果差大于 1 天，说明在前一个签到日期之后存在间隔。
- QUALIFY 或子查询 + WHERE: 用于过滤窗口函数的结果。QUALIFY 语法比子查询简洁，但并非所有数据库支持。

- **预期输出 11.1:** (显示间隔“开始”的日期，即间隔前的最后一天)

- user_id | signin_date | PreviousSignInDate

- -----|-----|-----

- 101 | 2023-01-04 | 2023-01-02 -- 2023-01-04 比 2023-01-02 晚超过 1 天，说明 2023-01-03 缺失

- 101 | 2023-01-07 | 2023-01-05 -- 2023-01-07 比 2023-01-05 晚超过 1 天，说明 2023-01-06 缺失

- 101 | 2023-01-10 | 2023-01-08 -- 2023-01-10 比 2023-01-08 晚超过 1 天，说明 2023-01-09 缺失

查询代码 11.2: 生成完整日期序列并 LEFT JOIN 查找缺失日期

- **目的:** 找出在指定日期范围（例如 2023-01-01 到 2023-01-10）内，用户 101 所有缺失的签到日期。
- **方法:** 生成该日期范围内的所有日期，然后与用户的签到记录进行 LEFT JOIN，查找没有匹配项的日期。
- **涉及表格:** UserSignins，以及生成日期序列的方法。

首先，我们需要一个方法来生成日期序列。这里使用**递归 CTE**，它是一种标准 SQL 方法（语法可能略有差异）。

SQL

-- 查找在指定日期范围内用户缺失的签到日期

-- 假设要查找的日期范围是 '2023-01-01' 到 '2023-01-10'

WITH RECURSIVE DateSequence AS (-- 定义一个递归 CTE 名为 DateSequence

-- 递归的起始成员 (Anchor Member)

SELECT DATE '2023-01-01' AS MissingDate -- 选择范围的起始日期，命名为

交道远信息

MissingDate

UNION ALL -- 将起始成员与递归成员连接

-- 递归成员 (Recursive Member)

SELECT MissingDate + INTERVAL '1 day' -- 选择前一次结果日期的后一天

FROM DateSequence -- 从 CTE 本身引用，实现递归

WHERE MissingDate + INTERVAL '1 day' <= DATE '2023-01-10' -- 递归终止条件：日期不能超过范围的结束日期

) -- CTE 定义结束

-- 主查询：将完整的日期序列与用户的签到记录进行 LEFT JOIN

SELECT

ds.MissingDate -- 选择日期序列中的日期

FROM

DateSequence ds -- 从完整的日期序列 CTE 中查询，别名 ds

LEFT JOIN

UserSignins us ON ds.MissingDate = us.signin_date AND us.user_id = 101 -- 与用户 101 的签到记录进行左连接，连接条件是日期和用户 ID 都匹配

WHERE

us.signin_date IS NULL; -- 过滤条件：只保留在日期序列中存在，但在用户 101 的签到记录中不存在匹配项的日期 (即缺失的日期)

- **代码解释：**

- WITH RECURSIVE DateSequence AS (...): **用途：**定义一个递归 CTE。递归 CTE 包含一个非递归部分（起始成员）和一个递归部分，通过 UNION ALL 连接，递归部分会反复执行，直到满足终止条件。
- SELECT DATE '2023-01-01' AS MissingDate: 起始成员。生成日期序列的第一个日期。
- UNION ALL: 连接起始成员和递归成员。
- SELECT MissingDate + INTERVAL '1 day' FROM DateSequence WHERE MissingDate + INTERVAL '1 day' <= DATE '2023-01-10': 递归成员。它从 DateSequence 上一次产生的结果中取 MissingDate，加上 1 天，生成下一个日期。WHERE 子句是终止条件，防止无限循环。
- FROM DateSequence ds: 主查询从生成的完整日期序列 DateSequence CTE 中查询。
- LEFT JOIN UserSignins us ON ds.MissingDate = us.signin_date AND us.user_id = 101: 与 UserSignins 表进行左连接，只关联用户 ID 101 的记录。左连接保留 DateSequence 中的所有日期。
- WHERE us.signin_date IS NULL: **用途：**这是查找缺失项的核心模式。在 LEFT JOIN 结果中，如果 DateSequence 中的某个日期在 UserSignins 中没有匹配（即用户 101 在那天没有签到），那么 us.signin_date 就会是 NULL。过滤出这些 NULL 的行，就是缺失的日期。

- **预期输出 11.2：**

- MissingDate

- -----

- 2023-01-03
 - 2023-01-06
 - 2023-01-09
 - **查找序列间隔总结:** LAG/LEAD 适合找出不连续的“点”（间隔的开始/结束），而生成完整序列与 LEFT JOIN 结合是找出“所有缺失点”的标准且强大的方法。递归 CTE 是生成序列的通用方法。
-

12. 分组内 Top N 查询 (Top N per Group Query)

- **目的:** 在每个分组（如每个部门、每个产品类别）中，找出某个指标（如销售额、分数）最高或最低的 N 条记录。这是一个非常常见的数据分析需求。
- **常用技术:** 通常结合使用 **窗口函数**（特别是 ROW_NUMBER(), RANK(), DENSE_RANK()）和**子查询**或 **CTE (Common Table Expression)**。PARTITION BY 用于定义分组，ORDER BY 用于指定排序依据，然后在外部查询中根据窗口函数分配的排名结果进行过滤（例如 WHERE rn <= N）。
- **案例:** 找出每个产品类别中**价格最高的前 2 个产品**。
- **涉及表格:** products 表。

首先，请确保您已经执行了我们之前提供的 products 表的建表和数据插入代码。

查询代码 12.1: 使用窗口函数分配排名并筛选 Top N

SQL

```
WITH RankedProducts AS ( -- 定义一个 CTE 名为 RankedProducts
    SELECT
        product_id, -- 选择产品 ID
        product_name, -- 选择产品名称
        category, -- 选择产品类别
        price, -- 选择产品价格

        -- === 排名函数示例 ===
        -- ROW_NUMBER() 用法: 在每个分区内按顺序生成一个唯一连续的行号
        ROW_NUMBER() OVER (
            PARTITION BY category -- PARTITION BY 用法: 按产品类别对产品进行分组，排名
            在每个类别内独立计算
            ORDER BY price DESC -- ORDER BY 用法: 在每个类别内，按价格降序排序 (价格
            最高的排前面)
        ) AS rn, -- 分配行号，rn=1 表示该类别中最贵的产品

        -- RANK() 用法: 计算排名，并列值排名相同，后续排名会跳跃
        RANK() OVER (
            PARTITION BY category -- 按产品类别分区
            ORDER BY price DESC -- 按价格降序排序
        ) AS rnk, -- 计算每个类别产品金额的排名
```

奕道远信息

```

-- DENSE_RANK() 用法: 计算排名, 并列值排名相同, 后续排名不跳跃
DENSE_RANK() OVER (
    PARTITION BY category -- 按产品类别分区
    ORDER BY price DESC -- 按价格降序排序
) AS dns_rnk -- 计算每个类别产品金额的紧密排名

FROM
    products -- 从 products 表查询
) -- CTE 定义结束

-- 主查询: 从 RankedProducts CTE 中选择排名在前 N 的记录
SELECT
    product_id,
    product_name,
    category,
    price,
    rn, -- 显示行号
    rnk, -- 显示排名
    dns_rnk -- 显示紧密排名
FROM
    RankedProducts -- 从 RankedProducts CTE 查询
WHERE
    rn <= 2 -- 过滤条件: 只保留每个类别中按行号排在前 2 的产品 (通常用 ROW_NUMBER
进行 Top N 过滤)
ORDER BY -- 最后排序结果, 按类别和行号
    category ASC,
    rn ASC;

```

- **代码解释:**

- WITH RankedProducts AS (...): 定义一个 CTE 来计算窗口函数结果。这样做使得后面的过滤步骤更清晰。
- ROW_NUMBER() OVER (PARTITION BY category ORDER BY price DESC): **用途:** 为每个 category 分区内的行按 price 降序分配一个唯一的行号 rn (1, 2, 3...)。这个行号决定了在每个类别中最贵的、次贵的等。
- RANK() OVER (...) AS rnk: **用途:** 计算排名。如果两个产品价格相同, 它们会有相同的 rnk 值, 并且下一个 rnk 值会跳过相应的数字。
- DENSE_RANK() OVER (...) AS dns_rnk: **用途:** 计算排名。与 RANK() 类似, 但并列值后不会跳过排名数字。
- PARTITION BY category: **用途:** 将 products 表的数据按 category 分割成独立的组 (分区)。窗口函数在每个类别内独立计算排名。
- ORDER BY price DESC: **用途:** 定义分区内的排序顺序, 这决定了 ROW_NUMBER(), RANK(), DENSE_RANK() 如何分配。这里是按价格从高到低排序。
- SELECT ... FROM RankedProducts WHERE rn <= 2: 在 CTE 外的主查询中,

从带有排名结果的 RankedProducts 中选取数据，并使用 WHERE rn <= 2 过滤，只保留每个类别中 rn 小于或等于 2 的行，即每个类别中价格最高的 2 个产品（即使有并列，ROW_NUMBER 也会给出唯一的行号，从而保证只取 2 个）。

- ORDER BY category ASC, rn ASC: 最后对整个结果集按类别和行号进行排序，使输出更整齐。

- **预期输出 12.1:**

- product_id | product_name | category | price | rn | rnk | dns_rnk
 - -----|-----|-----|-----|----|----|-----
 - Books | Python Book | Books | 60.00 | 1 | 1 | 1
 - Books | SQL Book | Books | 45.00 | 2 | 2 | 2
 - Electronics| Laptop | Electronics | 800.00 | 1 | 1 | 1
 - Electronics| Monitor | Electronics | 300.00 | 2 | 2 | 2
 - Furniture | Desk Chair | Furniture | 150.00 | 1 | 1 | 1
 - Furniture | Desk Lamp | Furniture | 35.00 | 2 | 2 | 2
 - Misc | Mystery Item | Misc | 999.99 | 1 | 1 | 1
 - -- Misc 类别只有一个产品，所以只显示 1 个 Top N
 - **分组内 Top N 总结:** 窗口函数（尤其是 ROW_NUMBER）配合 PARTITION BY 和 ORDER BY 是实现分组内 Top N 查询的效率高且标准的方法。理解 ROW_NUMBER, RANK, DENSE_RANK 在处理并列时的区别很重要。
-

13. 识别连续序列或“岛” (Identifying Consecutive Sequences or “Islands”)

- **目的:** 找出数据中连续出现某种特定状态或属性的记录块，例如用户连续登录的天数、产品连续有销量的日期。
- **常用技术:** 这是一种相对复杂的模式，常用方法之一是利用窗口函数 ROW_NUMBER() 的特性。通过计算行的“自然”序列号和基于某种偏移的序列号之间的差异，来识别连续的块。
- **案例:** 找出用户签到记录中的连续签到天数块。
- **涉及表格:** 我们使用之前创建的 UserSignins 表，它包含用户签到日期。

首先，请确保您已经执行了我们之前包含用户 101 签到日期的 UserSignins 表的建表和数

查询代码 13.1: 使用 ROW_NUMBER() 和日期差异识别连续序列

SQL

WITH DateWithSequence AS (-- 定义 CTE 1: 给每个用户的签到日期按顺序分配行号

SELECT

user_id, -- 用户 ID

signin_date, -- 签到日期

ROW_NUMBER() OVER (-- ROW_NUMBER 用法: 在每个用户分区内按日期分配序列号

奕道远信息


```

        PARTITION BY user_id -- 按用户 ID 分区
        ORDER BY signin_date ASC -- 按签到日期升序排序
    ) as rn -- 分配的序列号
FROM
    UserSignins -- 从签到表查询
), -- CTE 1 定义结束，用逗号分隔

IslandHelper AS ( -- 定义 CTE 2: 计算一个“辅助日期”来标识连续块
    SELECT
        user_id,
        signin_date,
        rn,
        -- 计算辅助日期: 签到日期 减去 (该日期在用户签到历史中的序列号 减去 1) 天
        -- 如果日期是连续的 (如 Day1, Day2, Day3), 它们的 rn 是 1, 2, 3.
        -- Day1 - (1-1)天 = Day1
        -- Day2 - (2-1)天 = Day1
        -- Day3 - (3-1)天 = Day1
        -- 连续日期计算出的辅助日期会相同，不连续日期计算出的辅助日期会不同
        signin_date - (rn * INTERVAL '1 day') AS IslandIdDate -- 使用 signin_date - rn*1day 也
        可以达到效果 (部分数据库需要 CAST rn 为 INTERVAL), signin_date - (rn-1)*1day 更直观表
        示连续块的开始日期。这里用 signin_date - rn*1day 因为语法兼容性好且能区分块。
        -- (请根据您的数据库语法调整 INTERVAL 的用法，如 num * INTERVAL 'DAY')
        -- 例如 MySQL: signin_date - INTERVAL rn DAY
        -- 例如 PostgreSQL/Oracle: signin_date - (rn * INTERVAL '1 day')
        -- 例如 SQL Server: DATEADD(day, -rn, signin_date)
    FROM
        DateWithSequence -- 从第一个 CTE 查询
    ) -- CTE 2 定义结束

-- 主查询：按辅助日期分组，找出每个连续块的起始和结束日期
SELECT
    user_id, -- 用户 ID
    MIN(signin_date) AS ConsecutiveStartDate, -- 连续块的起始日期 (该块中日期最小的)
    MAX(signin_date) AS ConsecutiveEndDate, -- 连续块的结束日期 (该块中日期最大的)
    COUNT(*) AS ConsecutiveDaysCount -- 连续天数 (该块中的记录数)
FROM
    IslandHelper -- 从第二个 CTE 查询
GROUP BY
    user_id, -- 按用户 ID 分组
    IslandIdDate -- 按计算出的辅助日期分组 (相同辅助日期的行属于同一个连续块)
ORDER BY
    user_id ASC,
    ConsecutiveStartDate ASC;

```

- 代码解释:

- WITH RECURSIVE DateWithSequence AS (...): 第一个 CTE, 给每个用户的所有签到日期按顺序分配一个从 1 开始的序列号 rn。这是后续计算的基础。
- IslandHelper AS (...): 第二个 CTE, 计算关键的 IslandIdDate。对于按日期排序的连续序列, $\text{signin_date} - (\text{rn} * \text{INTERVAL '1 day'})$ 计算出的日期会是同一个固定日期 (或者 $\text{signin_date} - (\text{rn}-1) * \text{INTERVAL '1 day'}$ 会等于该连续块的第一个日期)。当序列中断时, 这个计算出的日期就会变化。
- GROUP BY user_id, IslandIdDate: **用途:** 按用户 ID 和计算出的 IslandIdDate 对数据进行分组。具有相同 user_id 和 IslandIdDate 的行被分到同一组, 而这个组就代表了一个连续签到块。
- MIN(signin_date) AS ConsecutiveStartDate, MAX(signin_date) AS ConsecutiveEndDate, COUNT(*) AS ConsecutiveDaysCount: 对每个连续块 (每个分组) 计算其最小日期 (开始日期)、最大日期 (结束日期) 和总记录数 (连续天数)。
- ORDER BY user_id ASC, ConsecutiveStartDate ASC: 最后按用户 ID 和连续块的开始日期排序结果。
- **预期输出 13.1:**
- user_id | ConsecutiveStartDate | ConsecutiveEndDate | ConsecutiveDaysCount
- -----|-----|-----|-----
- -
- 101 | 2023-01-01 | 2023-01-03 | 3
- 101 | 2023-01-05 | 2023-01-06 | 2
- 101 | 2023-01-09 | 2023-01-11 | 3
- **识别连续序列总结:** 识别连续序列是一个相对复杂的模式, 利用 ROW_NUMBER() 和基于序列号/日期的计算来生成一个能标识连续块的辅助列是常用的技术。配合 GROUP BY 或其他窗口函数可以进一步分析这些连续块。

14. 计算期环比/同比变化 (Calculating Period-over-Period Change)

- **目的:** 比较当前时期 (如本月、本年) 的指标与上一个时期 (上月、去年同期) 的指标, 计算增长率或差值, 用于分析业务随时间的变化趋势。
 - **常用技术:**
 - 使用窗口函数 LAG() (获取前一个周期的值)。
 - 结合日期函数 (如 YEAR, MONTH) 进行分组或提取时间维度。
 - 使用聚合函数 (SUM, COUNT 等) 计算每个周期的汇总值。
 - 使用算术运算符 (-, /) 计算差值或增长率。
 - 通常使用 CTE 或子查询先按时间维度进行聚合, 再在聚合结果上应用 LAG() 和计算变化。
 - **案例:** 计算 'Electronics' 产品类别月总销售额的环比变化。
 - **涉及表格:** orders, order_items, products。
- 首先, 请确保您已经执行了我们之前包含 Electronics 产品类别销售数据的 orders,

order_items, products 表的建表和数据插入代码。

查询代码 14.1：计算月总销售额环比变化

SQL

```
WITH MonthlySales AS ( -- CTE 1: 计算每个月 'Electronics' 类别的总销售额
    SELECT
        YEAR(o.order_date) AS SaleYear, -- 提取订单年份
        MONTH(o.order_date) AS SaleMonth, -- 提取订单月份
        SUM(oi.item_sale_amount) AS MonthlyTotalSales -- 计算月总销售额
    FROM
        order_items oi -- 从订单明细表查询
    INNER JOIN
        products p ON oi.product_id = p.product_id -- 连接产品表获取类别
    INNER JOIN
        orders o ON oi.order_id = o.order_id -- 连接订单表获取日期
    WHERE
        p.category = 'Electronics' -- 过滤只保留 'Electronics' 类别
    GROUP BY
        YEAR(o.order_date), -- 按年份分组
        MONTH(o.order_date) -- 按月份分组
), -- CTE 1 定义结束

MonthlySalesWithLag AS ( -- CTE 2: 在月销售额结果上使用 LAG 获取上月销售额
    SELECT
        SaleYear, -- 年份
        SaleMonth, -- 月份
        MonthlyTotalSales, -- 当前月销售额
        LAG(MonthlyTotalSales, 1, 0) OVER ( -- LAG 用法: 获取按年、月排序的前一个月销售额
            ORDER BY SaleYear ASC, SaleMonth ASC -- 按年份和月份升序排序, 决定“前一个月”
        ) AS PreviousMonthSales -- 上个月销售额 (如果当前月是第一个月, 则为 0)
    FROM
        MonthlySales -- 从第一个 CTE 查询 (月销售额结果)
), -- CTE 2 定义结束

-- 主查询: 计算环比差值和增长率
SELECT
    SaleYear, -- 年份
    SaleMonth, -- 月份
    MonthlyTotalSales, -- 当前月总销售额
    PreviousMonthSales, -- 上个月总销售额
    MonthlyTotalSales - PreviousMonthSales AS MonthOverMonthChange, -- 环比差值 (当前月 - 上个月)
```

```

CASE -- 计算环比增长率，需要处理上个月销售额为 0 的情况
  WHEN PreviousMonthSales = 0 THEN NULL -- 如果上个月销售额为 0，增长率无法计
算，显示 NULL 或其他标记
  ELSE ((MonthlyTotalSales - PreviousMonthSales) / PreviousMonthSales) * 100 -- 算术
运算符: (差值 / 上个月销售额) * 100 得到百分比增长率
END AS MonthOverMonthGrowthRate -- 环比增长率
FROM
  MonthlySalesWithLag -- 从第二个 CTE 查询
ORDER BY
  SaleYear ASC,
  SaleMonth ASC; -- 最后按年月排序结果

```

• **代码解释:**

- MonthlySales AS (...): 第一个 CTE。它执行标准的聚合操作，计算出每个年月 ('2023-01', '2023-02', 等) 的 'Electronics' 类别的总销售额。这是环比计算的基础数据。
- MonthlySalesWithLag AS (...): 第二个 CTE。它在前一个 CTE 的月销售额结果集上应用 LAG() 窗口函数。LAG(MonthlyTotalSales, 1, 0) 按年份和月份排序，获取当前行（当前月）上方第 1 行（上个月）的 MonthlyTotalSales 值。如果当前是第一个月，没有前一个值，LAG 返回 0。
- SELECT ... FROM MonthlySalesWithLag: 主查询从第二个 CTE 获取数据。
- MonthlyTotalSales - PreviousMonthSales: **用途:** 算术运算符 - 用于计算当前月销售额与上个月销售额的差值，即环比差值。
- CASE WHEN PreviousMonthSales = 0 THEN NULL ELSE ((MonthlyTotalSales - PreviousMonthSales) / PreviousMonthSales) * 100 END: 使用 CASE 语句处理环比增长率的计算。如果上个月销售额 PreviousMonthSales 为 0，直接计算除法会导致错误或结果无意义，所以判断为 0 时返回 NULL。否则，使用算术运算符 -, /, * 计算增长率百分比。
- ORDER BY SaleYear ASC, SaleMonth ASC: 最后按年月排序结果。

- **预期输出 14.1:** (基于 Electronics 类别销售数据: Jan: 850, Feb: 875, Mar: 0, Apr: 0, May: 0, Jun: 1250, Jul: 1190, Aug: 430)

• SaleYear | SaleMonth | MonthlyTotalSales | PreviousMonthSales |
MonthOverMonthChange | MonthOverMonthGrowthRate

• -----|-----|-----|-----|-----
-----|-----

• 2023 | 1 | 850.00 | 0.00 | 850.00 | NULL
-- 第一个月，上月销售额为 0

• 2023 | 2 | 875.00 | 850.00 | 25.00 | 2.9411...
-- (875-850)/850 * 100

• 2023 | 3 | 0.00 | 875.00 | -875.00 | -100.0000...
-- (0-875)/875 * 100

• 2023 | 4 | 0.00 | 0.00 | 0.00 | NULL --
上月为 0

• 2023 | 5 | 0.00 | 0.00 | 0.00 | NULL --
上月为 0

奕道远信息

- 2023 | 6 | 1250.00 | 0.00 | 1250.00 | NULL
-- 上月为 0
- 2023 | 7 | 1190.00 | 1250.00 | -60.00 | -4.8000...
-- (1190-1250)/1250 * 100
- 2023 | 8 | 430.00 | 1190.00 | -760.00 | -63.8655...
-- (430-1190)/1190 * 100

(注意: 数据中 3, 4, 5 月份 Electronics 类别没有销售, 因此 MonthlyTotalSales 是 0。)

- **计算期变化总结:** LAG() 窗口函数结合按时间维度聚合是计算环比/同比变化的标准模式。通过 CTE 或子查询分步进行, 先聚合再计算变化, 可以提高查询的可读性。处理除以零的情况是计算增长率时需要注意的。同比计算类似, 只需调整 LAG() 的偏移量 (如 LAG(..., 12, ...) 计算去年同月)。

15. 数据分箱/离散化 (Data Binning / Discretization)

- **目的:** 将连续的数值数据划分到预定义的区间或类别中。这样做可以简化数据、减少噪声, 或将数据转换为适合分类分析的形式。
- **常用技术:** CASE WHEN 语句是实现自定义分箱最常用和灵活的方法, 根据数值范围设置不同的条件。对于等宽或等频分箱, 可能需要先计算分位数或最大最小值来确定边界, 再应用 CASE WHEN 或特定函数。
- **案例:** 根据产品价格将产品划分为不同的价格区间等级 (例如: 低、中、高、特级)。
- **涉及表格:** products 表 (包含 product_name 和 price)。

首先, 请确保您已经执行了我们之前提供的 products 表的建表和数据插入代码。

查询代码 15.1: 使用 CASE WHEN 进行价格分箱

SQL

SELECT

product_id, -- 选择产品 ID

product_name, -- 选择产品名称

price, -- 选择产品价格

CASE -- CASE WHEN 用法: 基于数值范围条件进行判断和赋值

WHEN price < 50 THEN 'Low' -- 当价格小于 50 时, 分到 'Low' 箱

WHEN price >= 50 AND price < 200 THEN 'Medium' -- 当价格大于等于 50 且小于 200 时, 分到 'Medium' 箱

WHEN price >= 200 AND price < 500 THEN 'High' -- 当价格大于等于 200 且小于 500 时, 分到 'High' 箱

ELSE 'Premium' -- 否则 (价格大于等于 500) 分到 'Premium' 箱

END AS PriceSegment -- 将分箱结果命名为 PriceSegment 列

FROM

products; -- 从 products 表查询

- **代码解释:**

- CASE WHEN condition THEN result ... ELSE result END: **用途:** 根据 WHEN 后面的条件, 返回对应的 THEN 后面的值。如果没有 WHEN 条件满足, 返回

ELSE 后面的值。

- 每个 WHEN price ... THEN '...' 子句定义了一个价格区间和对应的分类标签。
- price < 50: 比较运算符，检查价格是否小于 50。
- price >= 50 AND price < 200: 比较运算符和逻辑运算符 AND 组合，检查价格是否在 50 到 200 之间（不含 200）。
- ELSE 'Premium': **用途:** 覆盖所有不满足前述 WHEN 条件的情况。这里表示价格 >= 500。
- AS PriceSegment: 为新的分箱结果列指定别名。

- **预期输出 15.1:**

- product_id | product_name | price | PriceSegment
- -----|-----|-----|-----
- 1 | Laptop | 800.00 | Premium
- 2 | Keyboard | 75.00 | Medium
- 3 | Mouse | 25.00 | Low
- 4 | SQL Book | 45.00 | Low
- 5 | Python Book | 60.00 | Medium
- 6 | Desk Chair | 150.00 | Medium
- 7 | Desk Lamp | 35.00 | Low
- 8 | Monitor | 300.00 | High
- 9 | Novel | 15.00 | Low
- 10 | Headphones | 90.00 | Medium
- 11 | Mystery Item | 999.99 | Premium

- **数据分箱总结:** CASE WHEN 提供了一种灵活的方式来创建自定义的分箱规则，是离散化连续数据、创建分类维度的常用技术。
-

16. 计算百分位数或分位数 (Calculating Percentiles or Quantiles)

- **目的:** 找出数据集中位于特定百分位（如第 25、50、75、90 百分位）的值，或将数据等分为 N 份（如分成 4 份即四分位数），以了解数据的分布特征，特别是在数据分布不均匀时。
- **常用技术:** 数据库通常提供特定的窗口函数：
 - NTILE(n): 将有序数据分为 n 个等级，并为每一行分配其所属的等级编号（1 到 n）。
 - PERCENT_RANK(): 计算一行在其分区内的百分比排名（值越大，排名百分比越接近 1）。公式为 (rank - 1) / (total_rows_in_partition - 1)。
 - CUME_DIST(): 计算行的累计分布（小于等于当前值的行所占的比例）。
 - PERCENTILE_CONT(p) 或 PERCENTILE_DISC(p): 更直接计算特定百分位 p 的值（Cont 表示插值计算，Disc 表示取实际数据点）。这些函数语法 varies 且并非所有数据库支持。
- **应用案例:** 分析订单金额的分布。对于每个订单，计算其在所有订单中的百分比排

名, 累计分布百分比, 并将其划分到四分位 (1-4)。

- **涉及表格:** orders 表 (包含 amount)。

首先, 请确保您已经执行了我们之前提供的 orders 表的建表和数据插入代码。

查询代码 16.1: 使用窗口函数计算百分位数和分位数

SQL

SELECT

order_id, -- 选择订单 ID

amount, -- 选择订单金额

-- === 分位数 (Quantile) 函数 ===

NTILE(4) OVER (-- NTILE 用法: NTILE(n) 将有序数据分为 n 个等份

ORDER BY amount ASC -- ORDER BY 用法: NTILE 依赖此顺序进行划分

) AS Quartile, -- 将所有订单按金额分为 4 个等级 (四分位)

-- === 百分位排名函数 ===

PERCENT_RANK() OVER (-- PERCENT_RANK 用法: 计算行的百分比排名 (值越大越接近 1)

ORDER BY amount ASC -- ORDER BY 用法: 计算百分比排名依赖此顺序

) AS PercentRank, -- 计算当前订单金额在所有订单中的百分比排名

-- === 累计分布函数 ===

CUME_DIST() OVER (-- CUME_DIST 用法: 计算行的累计分布 (小于等于当前值的比例)

ORDER BY amount ASC -- ORDER BY 用法: 计算累计分布依赖此顺序

) AS CumulativeDistribution -- 计算小于等于当前订单金额的订单所占的比例

FROM

orders -- 从 orders 表查询

ORDER BY -- 最后按订单金额排序结果, 方便查看百分位

amount ASC;

- **代码解释:**

- OVER (ORDER BY amount ASC): **用途:** 为窗口函数定义一个窗口。由于没有 PARTITION BY, 窗口是整个结果集。ORDER BY amount ASC 是必须的, 它定义了数据在窗口内的排序顺序, 这是计算排名、分位数、累计分布等的基础。
- NTILE(4) ...: **用途:** 将查询结果集 (按金额排序后) 尽量均匀地分成 4 份, 并给每行分配其所属的组号 (1, 2, 3, 4)。常用于将数据划分为四分位、十分位等。
- PERCENT_RANK() ...: **用途:** 计算当前行值在其分区内的百分比排名。结果范围在 0 到 1 之间。公式为 $(rank - 1) / (总行数 - 1)$ 。第一行的 PercentRank 总是 0。
- CUME_DIST() ...: **用途:** 计算当前行值在其分区内的累计分布。结果表示小于等于当前值的所有行所占的比例。结果范围大于 0 且小于等于 1。

- **预期输出 16.1:** (基于 orders 表数据, 按 amount 排序)

- order_id | amount | Quartile | PercentRank | CumulativeDistribution

```

• -----|-----|-----|-----|-----
• 103 | 50.00 | 1 | 0.000000 | 0.066666... -- 1/15
• 107 | 80.00 | 1 | 0.066666... | 0.133333... -- 2/15
• 110 | 90.00 | 1 | 0.133333... | 0.200000... -- 3/15
• 112 | 110.00 | 1 | 0.200000... | 0.266666... -- 4/15
• 105 | 120.00 | 2 | 0.266666... | 0.333333... -- 5/15
• 101 | 150.00 | 2 | 0.333333... | 0.466666... -- 7/15 (150 有两个并列)
• 114 | 150.00 | 2 | 0.333333... | 0.466666...
• 108 | 180.00 | 2 | 0.533333... | 0.533333... -- 8/15
• 115 | 210.00 | 3 | 0.600000... | 0.600000... -- 9/15
• 113 | 220.00 | 3 | 0.666666... | 0.666666... -- 10/15
• 102 | 250.50 | 3 | 0.733333... | 0.733333... -- 11/15
• 104 | 300.00 | 4 | 0.800000... | 0.866666... -- 13/15 (300 有两个并列)
• 111 | 600.00 | 4 | 0.933333... | 0.933333... -- 14/15

```

(注: 精确的百分位和累计分布数值可能因数据库实现略有差异, 特别是处理并列值时。
NTILE 的划分也可能略有偏差, 尽量使每组数量接近, 但不保证绝对相等)

- **计算百分位数总结:** NTILE(), PERCENT_RANK(), CUME_DIST() 等窗口函数是分析数据分布、进行分位或百分位计算的有效工具, 它们依赖于 ORDER BY 子句来确定数据的顺序。

17. 连接多个 (多于两个) 表 (Joining Multiple Tables)

- **目的:** 从三个或更多相关的表中整合数据, 以获取关于某个实体或事件的更全面的信息视图。
- **常用技术:** 通过连续使用 JOIN 子句来实现。连接的顺序通常遵循表之间的逻辑关系 (一个表的外键关联另一个表的主键)。可以连续使用 INNER JOIN, LEFT JOIN 等。
- **案例:** 获取每个订单明细项的详细信息, 包括明细项本身的数量和金额、所属订单的日期和总金额、下单客户的姓名和城市、以及销售产品的名称和类别。
- **涉及表格:** customers, orders, order_items, products。我们将使用 INNER JOIN 将它们全部连接起来。

首先, 请确保您已经执行了我们之前包含所有必要数据的 customers, orders, order_items, products 表的建表和数据插入代码。

查询代码 17.1: 连接四个表获取订单明细完整信息

SQL

SELECT

```

oi.item_id,      -- 订单明细项 ID (来自 order_items)
oi.quantity,     -- 购买数量 (来自 order_items)
oi.item_sale_amount, -- 单个明细项销售金额 (来自 order_items)

o.order_id,      -- 关联的订单 ID (来自 orders)
o.order_date,    -- 订单日期 (来自 orders)

```

奕道远信息

o.amount AS OrderTotalAmount, -- 订单总金额 (来自 orders)

c.customer_id, -- 关联的客户 ID (来自 customers)

c.customer_name, -- 客户姓名 (来自 customers)

c.city AS CustomerCity, -- 客户城市 (来自 customers)

p.product_id, -- 关联的产品 ID (来自 products)

p.product_name, -- 产品名称 (来自 products)

p.category AS ProductCategory, -- 产品类别 (来自 products)

p.price AS ProductUnitPrice -- 产品单价 (来自 products)

FROM

order_items oi -- 从 order_items 表开始查询 (别名 oi)

INNER JOIN

orders o ON oi.order_id = o.order_id -- 第 1 次 JOIN: 将 order_items 与 orders 连接, 条件是 order_id 相等

INNER JOIN

customers c ON o.customer_id = c.customer_id -- 第 2 次 JOIN: 将 JOIN 前的结果 (包含 oi 和 o 的信息) 与 customers 连接, 条件是 customer_id 相等

INNER JOIN

products p ON oi.product_id = p.product_id; -- 第 3 次 JOIN: 将 JOIN 前的结果 (包含 oi, o, c 的信息) 与 products 连接, 条件是 product_id 相等

- **代码解释:**

- FROM table1 JOIN table2 ON condition1 JOIN table3 ON condition2 JOIN table4 ON condition3 ...: **用途:** 连续使用 JOIN 子句来连接多个表。SQL 会从 FROM 子句的第一个表开始, 依次与后面的表进行连接。每次 JOIN 都将前一步的结果集与新表进行合并。

- 连接的顺序是关键。通常从最“中心”或最详细的表开始, 或者从关系最清晰的表开始, 然后依次向外连接相关的表。本例从 order_items 开始, 因为它记录了具体的销售事件, 然后连接到 orders 获取订单整体信息, 连接到 customers 获取客户信息, 连接到 products 获取产品信息。

- 每个 ON 子句都指定了前一步的结果集与当前要连接的表之间的匹配条件, 通常是主键和外键的关系。

- **预期输出 17.1:** (将显示 order_items 表中的每一行, 并附加来自 orders, customers, products 表中匹配行的信息)

- item_id | quantity | item_sale_amount | order_id | order_date | OrderTotalAmount | customer_id | customer_name | CustomerCity | product_id | product_name | ProductCategory | ProductUnitPrice

- -----|-----|-----|-----|-----|-----
-----|-----|-----|-----|-----|-----
-----|-----|-----|-----|-----|-----

- 1 | 1 | 800.00 | 101 | 2023-01-15 | 850.00 | 1 | Alice
| New York | 1 | Laptop | Electronics | 800.00
- 2 | 2 | 50.00 | 101 | 2023-01-15 | 850.00 | 1 | Alice

- | | | | | | | | | | | |
|---|-----|-----|----------|----|-----|-------------|-------------|-------|-----|---------|
| | | | New York | 3 | | Mouse | Electronics | 25.00 | | |
| • | 3 | 1 | 60.00 | | 102 | 2023-01-16 | 150.00 | | 3 | Charlie |
| | | | New York | 5 | | Python Book | Books | 60.00 | | |
| • | ... | ... | ... | | ... | ... | ... | | ... | ... |
| | 21 | 2 | 180.00 | | 115 | 2023-08-15 | 180.00 | | 7 | Grace |
| | | | London | 10 | | Headphones | Electronics | 90.00 | | |
- **连接多个表总结:** 通过连续使用 JOIN 子句, 可以从多个表中整合数据。连接的顺序和 ON 条件的准确性是确保结果正确性的关键。理解表之间的主键-外键关系有助于确定正确的连接路径。

18. 数据验证规则检查 (Data Validation Rules Check)

- **目的:** 编写查询来识别数据中不符合业务规则、逻辑不一致或违反完整性约束的记录。这是数据清洗和数据质量管理的关键步骤。
- **常用技术:**
 - WHERE 子句配合各种条件判断 (=, >, <, !=, IS NULL, IS NOT NULL, BETWEEN, LIKE 等): 检查单个字段的值是否有效或符合预期格式/范围。
 - **聚合函数** (COUNT, SUM 等) 结合 HAVING: 检查分组后的数据是否符合规则, 如检查是否存在重复项 (COUNT(*) > 1)。
 - **连接** (LEFT JOIN) 结合 IS NULL: 检查记录是否存在关联的表中应该存在的匹配项 (如订单是否关联了有效的客户)。
 - **子查询** 或 **CTE**: 用于检查更复杂的跨记录或跨表逻辑。
- **应用案例:** 在客户、订单和产品数据中查找以下潜在的数据质量问题:
 1. 产品库存或价格为负数。
 2. 客户注册日期晚于其第一笔订单日期。
 3. 存在同名客户 (姓名完全相同, 但可能是不同客户)。
 4. 产品价格包含不规范的 '.99' 结尾 (假设业务规则要求价格为整数或 '.00')。
- **涉及表格:** customers, orders, products.

为了演示这些验证场景, 我们需要在之前的数据基础上, 添加一些符合验证规则 (即“错误”数据) 的示例记录。

SQL

-- 在 products 表中添加负库存/负价格和 .99 价格的产品

INSERT INTO products (product_id, product_name, category, price, stock) VALUES

(12, 'Defective Item', 'Misc', -5.00, -10), -- 负价格, 负库存

(13, 'Shiny Gadget', 'Electronics', 199.99, 25); -- .99 价格

-- 在 customers 表中添加同名客户

INSERT INTO customers (customer_id, customer_name, city, join_date) VALUES

(9, 'Alice', 'Los Angeles', '2024-02-01'); -- 与 customer_id 1 的 Alice 同名

-- 在 orders 表中添加一个订单, 使其日期早于客户 ID 9 的 join_date

奕道远信息

```
INSERT INTO orders (order_id, customer_id, order_date, amount) VALUES
(116, 9, '2024-01-15', 50.00); -- 订单日期早于客户 9 的 join_date '2024-02-01'
请先执行上述 INSERT 语句，在之前的数据基础上添加这些记录。
```

查询代码 18.1: 查找数据验证问题

SQL

-- 验证规则 1: 查找产品库存或价格为负数的产品

SELECT

'Negative Stock/Price' AS IssueType, -- 标记问题类型

product_id, -- 产品 ID

product_name, -- 产品名称

price, -- 价格

stock -- 库存

FROM

products -- 从 products 表查询

WHERE

stock < 0 OR price < 0 -- WHERE 用法: 过滤 stock 或 price 小于 0 的行

-- OR stock IS NULL OR price IS NULL -- 也可以检查是否为 NULL

UNION ALL -- UNION ALL 用法: 合并不同验证查询的结果集

-- 验证规则 2: 查找客户注册日期晚于其第一笔订单日期的异常记录

SELECT

'Join Date After First Order' AS IssueType, -- 标记问题类型

c.customer_id, -- 客户 ID

c.customer_name, -- 客户姓名

c.join_date, -- 加入日期

MIN(o.order_date) AS FirstOrderDate -- 计算客户的第一笔订单日期

FROM

customers c -- 从 customers 表查询

INNER JOIN

orders o ON c.customer_id = o.customer_id -- 连接 orders 表获取订单日期

GROUP BY -- 聚合函数 MIN(o.order_date) 需要 GROUP BY

c.customer_id, c.customer_name, c.join_date -- 按客户信息分组，才能计算每个客户的

MIN 订单日期

HAVING

c.join_date > MIN(o.order_date) -- HAVING 用法: 过滤 GROUP BY 后的分组，检查加入日期是否晚于第一笔订单日期

UNION ALL -- 合并结果集

-- 验证规则 3: 查找存在同名客户 (姓名完全相同，不考虑是否是同一个人)

SELECT

尖道远信息

```

'Duplicate Customer Name' AS IssueType, -- 标记问题类型
customer_name,      -- 客户姓名 (重复的姓名)
COUNT(*) AS NameCount -- 该姓名出现的次数
FROM
    customers      -- 从 customers 表查询
GROUP BY
    customer_name   -- 按姓名分组
HAVING
    COUNT(*) > 1    -- 过滤分组, 只保留出现次数大于 1 的姓名

UNION ALL -- 合并结果集

```

-- 验证规则 4: 查找产品价格包含 .99 结尾 (假设为不规范数据)

```

SELECT
    'Price Ends With .99' AS IssueType, -- 标记问题类型
    product_id,      -- 产品 ID
    product_name,    -- 产品名称
    price            -- 价格
FROM
    products      -- 从 products 表查询
WHERE
    CAST(price AS VARCHAR) LIKE '%.99'; -- LIKE 用法: 模糊匹配。需要先将价格转换为字符串才能使用 LIKE 进行模式匹配。CAST(column AS type) 是转换函数。

```

- **代码解释:**

- UNION ALL: **用途:** 用于将多个独立的 SELECT 语句的结果合并成一个结果集。这里用于将不同验证规则发现的“问题记录”统一列出。每个 SELECT 语句都增加一个 IssueType 列来标识问题类型。
- 每个 SELECT 块实现了不同的验证规则, 使用了各自适用的技术:
 - 规则 1 使用 WHERE 和比较运算符 (<) 及逻辑运算符 (OR)。
 - 规则 2 使用 INNER JOIN 连接 customers 和 orders 表, 按客户 GROUP BY 计算每个客户的 MIN(o.order_date) (第一笔订单日期), 然后使用 HAVING 过滤出 c.join_date > MIN(o.order_date) 的客户分组。
 - 规则 3 使用 GROUP BY customer_name 计算每个姓名的出现次数 (COUNT(*)), 然后使用 HAVING COUNT(*) > 1 过滤出出现多次的姓名分组。
 - 规则 4 使用 WHERE 和 LIKE '%.99' 进行模式匹配。CAST(price AS VARCHAR) 用于将数值类型的 price 临时转换为字符串, 因为 LIKE 操作符用于字符串。

- **预期输出 18.1:** (合并了所有验证规则发现的问题记录)

```

IssueType      | product_id | product_name | price | stock | customer_id |
customer_name | join_date  | FirstOrderDate | NameCount
-----|-----|-----|-----|-----|-----|
-----|-----|-----|-----|-----|-----|

```

- Negative Stock/Price | 12 | Defective Item | -5.00 | -10 | NULL | NULL
| NULL | NULL | NULL -- 产品问题
- Negative Stock/Price | NULL | NULL | NULL | NULL | NULL | NULL
| NULL | NULL | NULL -- 负库存产品 (如果 stock < 0 是单独条件)
- Join Date After First Order | NULL | NULL | NULL | NULL | 9 | Alice
| 2024-02-01 | 2024-01-15 | NULL -- 客户日期问题
- Duplicate Customer Name | NULL | NULL | NULL | NULL | NULL |
Alice | NULL | NULL | 2 -- 同名客户问题
- Price Ends With .99 | 13 | Shiny Gadget | 199.99 | NULL | NULL | NULL
| NULL | NULL | NULL -- 产品价格问题

(注意: `UNION ALL` 要求合并的 `SELECT` 语句有相同数量和兼容类型的列。这里通过在不需要的列位置选择 `NULL` 来匹配列的数量和类型。)

- **数据验证规则检查总结:** 数据验证通过组合 `WHERE`, `HAVING`, `JOIN` 以及字符串/日期函数和聚合函数来实现。通常针对不同的数据质量问题编写独立的查询, 然后可以使用 `UNION ALL` 将结果合并。

19. 数据抽样 (Sampling Data)

- **目的:** 从大型数据集中随机选取一部分数据进行分析、测试或原型开发, 以提高处理效率或获取代表性子集。
- **常用技术:**
 - 数据库特定的抽样语法: 如 SQL Server/PostgreSQL 的 `TABLESAMPLE`。
 - 使用随机函数结合排序和限制行数: 如 `ORDER BY RAND() LIMIT N` (MySQL) 或 `ORDER BY RANDOM() LIMIT N` (PostgreSQL, SQLite)。这是跨数据库比较通用的思路。
- **应用案例:** 从 `orders` 表中随机抽取 5 个订单进行人工核查。
- **涉及表格:** `orders` 表。

首先, 请确保您已经执行了我们之前包含所有订单数据的 `orders` 表的建表和数据插入代码。

查询代码 19.1: 使用随机排序和 `LIMIT` 进行抽样

SQL

-- 从 `orders` 表中随机抽取 5 个订单

SELECT

order_id, -- 选择订单 ID

customer_id, -- 选择客户 ID

order_date, -- 选择订单日期

amount -- 选择订单金额

FROM

orders -- 从 `orders` 表查询

ORDER BY

RAND() -- 或 RANDOM() -- `ORDER BY` 用法: 使用随机函数对所有行进行随机排序

LIMIT 5; -- `LIMIT` 用法: 限制结果只返回前 5 行 (排序后取前 5 行, 达到随机抽样效果)

-- 注意: RAND() 或 RANDOM() 函数名称在不同数据库中可能不同。MySQL 使用 RAND(), PostgreSQL 和 SQLite 使用 RANDOM()。SQL Server 可用 NEWID() 配合 ORDER BY。
-- 对于非常大的表, 这种方法效率可能不高, 因为需要对所有行进行排序。
-- 数据库特定的抽样语法 TABLESAMPLE 通常效率更高。
-- 例如 SQL Server: SELECT ... FROM orders TABLESAMPLE (5 ROWS); 或 TABLESAMPLE (10 PERCENT);
-- 例如 PostgreSQL: SELECT ... FROM orders TABLESAMPLE SYSTEM (5); 或 TABLESAMPLE BERNOULLI (10);

- **代码解释:**

- ORDER BY RAND(): **用途:** RAND() 或 RANDOM() 函数为每一行生成一个随机数。ORDER BY 子句根据这些随机数对所有行进行排序, 打乱了原始顺序。
- LIMIT 5: **用途:** LIMIT N (或 SQL Server 的 TOP N) 限制结果集只返回排序后的前 N 行。
- 将随机排序与 LIMIT 结合, 就实现了随机抽取 N 行。
- **预期输出 19.1:** (结果会随着每次执行而不同, 但总是返回 5 行 orders 表中的记录)
- order_id | customer_id | order_date | amount
- -----|-----|-----|-----
- ??? | ??? | ???-??-?? | ????.?? -- 每次随机不同的订单
- ... (共 5 行)
- **数据抽样总结:** 随机抽样是处理大数据集的实用技巧。使用随机函数排序后 LIMIT 是比较通用的方法, 但针对大数据, 数据库特定的抽样语法 (TABLESAMPLE) 通常更高效。

20. 分组内的首/末事件查询 (Finding First/Last Event per Group)

- **目的:** 在每个分组 (如每个客户、每个产品) 中, 找出时间上 (或根据其他顺序) 最早发生或最晚发生的事件记录。这个模式在之前的分组内 Top N 案例 (特别是使用 ROW_NUMBER()) 中已经有所体现, 因为它本质上是找出排序后的第一行或最后一行。这里我们更聚焦于“事件”的概念, 并可以使用 FIRST_VALUE()/LAST_VALUE() 函数来直接获取首/末事件的某个值。
- **常用技术:**
 - 窗口函数 ROW_NUMBER() 结合 PARTITION BY 和 ORDER BY, 然后在外部查询过滤 rn = 1 或 rn = count_of_group (后者需要另一个窗口函数或聚合)。这是获取首/末**整行记录**的常用方法。
 - 窗口函数 FIRST_VALUE(列名) OVER(...) 和 LAST_VALUE(列名) OVER(...): 直接获取窗口内第一行或最后一行的**指定列的值**。
 - **注意:** LAST_VALUE 默认窗口范围是当前行到分区结束, 但 FIRST_VALUE 默认是分区开始到当前行。为了让 LAST_VALUE 总是看到整个分区, 可能需要显式指定窗口框架为 ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING。
- **应用案例:** 对于每个客户, 找出其**第一笔订单**的日期和金额, 以及**最后一笔订单**的

日期和金额。

- **涉及表格:** orders 表 (包含 order_id, customer_id, order_date, amount)。

首先, 请确保您已经执行了我们之前包含所有订单数据的 orders 表的建表和数据插入代码。

查询代码 20.1: 使用 ROW_NUMBER() 查找首/末订单记录

SQL

WITH CustomerOrderSequence AS (-- 定义 CTE 1: 给每个客户的订单按日期分配升序和降序行号

SELECT

order_id, -- 订单 ID

customer_id, -- 客户 ID

order_date, -- 订单日期

amount, -- 订单金额

-- 分配升序行号, 用于找到第一笔订单 (rn_asc = 1)

ROW_NUMBER() OVER (

PARTITION BY customer_id -- 按客户 ID 分区

ORDER BY order_date ASC -- 按订单日期升序排序

) AS rn_asc,

-- 分配降序行号, 用于找到最后一笔订单 (rn_desc = 1)

ROW_NUMBER() OVER (

PARTITION BY customer_id -- 按客户 ID 分区

ORDER BY order_date DESC -- 按订单日期降序排序

) AS rn_desc

FROM

orders -- 从 orders 表查询

) -- CTE 定义结束

-- 主查询: 筛选出 rn_asc = 1 (第一笔订单) 或 rn_desc = 1 (最后一笔订单) 的记录

SELECT

order_id,

customer_id,

order_date,

amount,

-- 可以添加一列标识是首单还是末单

CASE

WHEN rn_asc = 1 THEN 'First Order'

WHEN rn_desc = 1 THEN 'Last Order'

ELSE 'Other Order' -- 实际情况中, 一个订单可能同时是首单和末单 (如果客户只有一个订单)

END AS OrderType

FROM

CustomerOrderSequence -- 从 CTE 查询

WHERE

rn_asc = 1 OR rn_desc = 1 -- 过滤条件: 保留每个客户 rn_asc 为 1 (首单) 或 rn_desc 为

1 (末单) 的行

ORDER BY -- 最后排序结果

customer_id ASC,

order_date ASC; -- 按客户 ID 和订单日期排序

- **代码解释:**

- WITH CustomerOrderSequence AS (...): 定义一个 CTE 来计算行号。
- ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date ASC) AS rn_asc: 在每个客户内, 按日期升序分配行号。rn_asc = 1 的行就是该客户的第一笔订单。
- ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) AS rn_desc: 在每个客户内, 按日期降序分配行号。rn_desc = 1 的行就是该客户的最后一笔订单。
- SELECT ... FROM CustomerOrderSequence WHERE rn_asc = 1 OR rn_desc = 1: 在主查询中, 过滤出那些同时满足 rn_asc=1 (首单) 或 rn_desc=1 (末单) 的行。如果一个客户只有一个订单, 那么该订单的 rn_asc 和 rn_desc 都会是 1, 会在结果中出现一次。

- **预期输出 20.1:** (每个客户显示其第一笔和最后一笔订单的记录)

- order_id | customer_id | order_date | amount | OrderType

- -----|-----|-----|-----|-----

- 101 | 1 | 2023-01-15 | 150.00 | First Order -- Alice 第一笔订单
- 111 | 1 | 2023-07-30 | 600.00 | Last Order -- Alice 最后一笔订单
- 104 | 2 | 2023-02-20 | 300.00 | First Order -- Bob 第一笔订单
- 113 | 2 | 2023-08-05 | 220.00 | Last Order -- Bob 最后一笔订单
- 102 | 3 | 2023-01-16 | 250.50 | First Order -- Charlie 第一笔也是最后一笔
- 112 | 3 | 2023-08-01 | 110.00 | Last Order -- Charlie 最后一笔
- 105 | 5 | 2023-05-15 | 120.00 | First Order -- Eve 第一笔也是最后一笔
- 114 | 5 | 2023-08-10 | 150.00 | Last Order -- Eve 最后一笔
- 110 | 7 | 2023-07-25 | 210.00 | First Order -- Grace 第一笔也是最后一笔
- 115 | 7 | 2023-08-15 | 350.00 | Last Order -- Grace 最后一笔
- 116 | 9 | 2024-01-15 | 50.00 | First Order -- Alice (LA) 第一笔也是最后一笔

(注: 客户 4, 6, 8 没有订单, 不会出现在结果中。)

查询代码 20.2: 使用 FIRST_VALUE() 和 LAST_VALUE() 获取首/末订单金额 (显示在所有行上)

SQL

SELECT

order_id, -- 订单 ID

customer_id, -- 客户 ID

order_date, -- 订单日期

amount, -- 订单金额

-- FIRST_VALUE 用法: 获取窗口内第一行 (按日期排序) 的 amount 值

变道远信息


```

FIRST_VALUE(amount) OVER (
  PARTITION BY customer_id -- 按客户 ID 分区
  ORDER BY order_date ASC -- 按订单日期升序排序
) AS FirstOrderAmount, -- 获取每个客户的第一笔订单金额 (显示在所有行上)

-- LAST_VALUE 用法: 获取窗口内最后一行 (按日期排序) 的 amount 值
LAST_VALUE(amount) OVER (
  PARTITION BY customer_id -- 按客户 ID 分区
  ORDER BY order_date ASC -- 按订单日期升序排序
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING -- 窗口
范围: 明确指定从分区开始到分区结束
) AS LastOrderAmount -- 获取每个客户的最后一笔订单金额 (显示在所有行上)

```

```

FROM
  orders -- 从 orders 表查询
ORDER BY -- 最后排序结果
  customer_id ASC,
  order_date ASC;

```

- **代码解释:**

- FIRST_VALUE(列名) OVER (...): **用途:** 返回窗口内第一行 (由 ORDER BY 定义顺序) 的指定列的值。
- LAST_VALUE(列名) OVER (...): **用途:** 返回窗口内最后一行 (由 ORDER BY 定义顺序) 的指定列的值。
- ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING: **用途:** 这是一个窗口框架子句。UNBOUNDED PRECEDING 表示从分区的第一行开始, UNBOUNDED FOLLOWING 表示到分区的最后一行结束。这是 LAST_VALUE 函数通常需要的窗口框架, 以确保它能看到整个分区内的最后一行 (默认的 LAST_VALUE 窗口通常只到当前行为止)。
- 在此查询中, FIRST_VALUE 和 LAST_VALUE 计算出的首末订单金额会显示在**每个客户的每一行订单记录上**。

- **预期输出 20.2:** (每个客户的每行订单都显示其首单和末单金额)

```

order_id | customer_id | order_date | amount | FirstOrderAmount | LastOrderAmount
-----|-----|-----|-----|-----|-----

```

```

101  | 1      | 2023-01-15 | 150.00 | 150.00      | 600.00      -- Alice 的所有
      |      |      |      |      |      | 订单都显示首单 150, 末单 600
103  | 1      | 2023-01-17 | 50.00  | 150.00      | 600.00
106  | 1      | 2023-06-01 | 450.75 | 150.00      | 600.00
111  | 1      | 2023-07-30 | 600.00 | 150.00      | 600.00
104  | 2      | 2023-02-20 | 300.00 | 300.00      | 220.00      -- Bob 的所有
      |      |      |      |      |      | 订单都显示首单 300, 末单 220
108  | 2      | 2023-06-10 | 180.00 | 300.00      | 220.00
113  | 2      | 2023-08-05 | 220.00 | 300.00      | 220.00
...   | ...    | ...        | ...    | ...         | ...

```

- 116 | 9 | 2024-01-15 | 50.00 | 50.00 | 50.00 -- Alice (LA) 只有一个订单，首单末单金额相同
 - **查找分组内的首/末事件总结:** 这是一个非常普遍的分析需求。使用 ROW_NUMBER() 配合 CTE 过滤是一种获取首/末行的标准方法。使用 FIRST_VALUE() 和 LAST_VALUE() 函数可以直接获取首/末行的**指定列的值**，并且可以将这些值显示在分组内的所有行上，这对于比较或进一步计算非常有用。
-

21. 计算移动平均/总和 (Calculating Moving Average/Sum)

- **目的:** 计算一个数据点及其前后指定范围内的点的平均值或总和。这用于平滑数据中的短期波动，揭示更长期的趋势，常用于时间序列分析。
- **常用技术: 聚合窗口函数** AVG() OVER() 或 SUM() OVER() 结合指定的**窗口框架 (Window Frame)**。窗口框架定义了当前计算中应包含哪些行，最常用的通过 ROWS BETWEEN ... AND ... 或 RANGE BETWEEN ... AND ... 来定义。
- **案例:** 计算订单金额的**3 个订单的移动平均值**，按照订单日期排序。
- **涉及表格:** orders 表 (包含 order_id, order_date, amount)。

首先，请确保您已经执行了我们之前提供的 orders 表的建表和数据插入代码。

查询代码 21.1: 计算 3 个订单的移动平均金额

SQL

SELECT

```
order_id, -- 订单 ID
order_date, -- 订单日期
amount, -- 订单金额
```

```
-- 计算当前行和前面 2 行的金额平均值 (共 3 行)
```

```
AVG(amount) OVER ( -- AVG() OVER(): 将 AVG 作为窗口函数使用
```

```
ORDER BY order_date ASC -- ORDER BY 用法: 定义窗口内行的顺序，这对于移动计算至关重要
```

```
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW -- 窗口框架: 定义窗口包含当前行和它前面的 2 行 (共 3 行)
```

```
) AS MovingAvg3Orders, -- 结果列别名
```

```
-- 计算当前行和前面 2 行的金额总和 (共 3 行)
```

```
SUM(amount) OVER ( -- SUM() OVER(): 将 SUM 作为窗口函数使用
```

```
ORDER BY order_date ASC -- 按订单日期升序排序
```

```
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW -- 窗口框架: 包含当前行和前面 2 行
```

```
) AS MovingSum3Orders -- 结果列别名
```

FROM

```
orders -- 从 orders 表查询
```

奕道远信息

ORDER BY -- 最后按订单日期排序结果

order_date ASC;

• **代码解释:**

- AVG(amount) OVER (...): **用途:** 将 AVG 函数作为窗口函数使用。它计算 amount 列在其 OVER() 子句定义的“窗口”内的平均值。
- SUM(amount) OVER (...): **用途:** 将 SUM 函数作为窗口函数使用。它计算 amount 列在其定义的窗口内的总和。
- OVER (ORDER BY order_date ASC ROWS BETWEEN 2 PRECEDING AND CURRENT ROW): 这是核心部分。
 - ORDER BY order_date ASC: **用途:** 定义窗口内的行的处理顺序, 对于移动平均/总和至关重要。
 - ROWS BETWEEN 2 PRECEDING AND CURRENT ROW: **用途:** 定义窗口框架。ROWS 表示基于物理行数来定义窗口。2 PRECEDING 表示包含当前行之前的 2 行。CURRENT ROW 表示包含当前行。所以 ROWS BETWEEN 2 PRECEDING AND CURRENT ROW 定义了一个包含当前行及其前面 2 行的窗口, 总共 3 行。
- 对于第一行, 窗口只包含它自己 (前面没有行)。对于第二行, 窗口包含它自己和前面 1 行。从第三行开始, 窗口包含当前行和前面完整的 2 行。

• **预期输出 21.1:** (按订单日期排序显示所有订单, 并附加移动平均/总和)

• order_id | order_date | amount | MovingAvg3Orders | MovingSum3Orders

• -----|-----|-----|-----|-----

- 101 | 2023-01-15 | 150.00 | 150.000000 | 150.00 -- 第 1 行: 窗口 [101]
- 103 | 2023-01-17 | 50.00 | 100.000000 | 200.00 -- 第 2 行: 窗口 [101, 103] (Avg(150, 50), Sum(150, 50))
- 104 | 2023-02-20 | 300.00 | 166.666666 | 500.00 -- 第 3 行: 窗口 [101, 103, 104] (Avg(150, 50, 300), Sum(150, 50, 300))
- 105 | 2023-05-15 | 120.00 | 156.666666 | 470.00 -- 第 4 行: 窗口 [103, 104, 105] (Avg(50, 300, 120), Sum(50, 300, 120))
- ... | ... | ... | ... | ...
- 116 | 2024-01-15 | 50.00 | ??? | ??? -- 最后一行, 窗口包含它自己和它前面的 2 行

(注意: RANGE BETWEEN ... AND ... 是另一种窗口框架定义方式, 基于值或日期范围而非行数, 用于处理具有相同值的行或在日期序列中有跳跃的情况。)

- **计算移动平均/总和总结:** 利用聚合窗口函数结合 ORDER BY 和窗口框架, 可以在不折叠行的情况下计算滚动指标, 是时间序列分析中的重要技术。

22. 填充数据中的间隔或缺失值 (Filling Gaps or Imputing Missing Data)

- **目的:** 当数据序列中存在 NULL 值时, 根据相邻的非 NULL 值或其他逻辑进行填充, 以创建更完整的数据集用于分析。

- **常用技术:**
 - **窗口函数** (LAG(), LEAD()) 结合 COALESCE(): 获取前/后行的值进行填充。对于填充“上一个非 NULL 值”这种模式, 需要更复杂的窗口函数用法 (LAST_VALUE 结合 IGNORE NULLS 或复杂的框架) 或递归 CTE。
 - **递归 CTE:** 可以按顺序遍历数据, 并在遇到 NULL 时引用前一行的计算结果进行填充。这是填充“上一个非 NULL 值”且兼容性较好的标准方法之一。
 - **生成完整序列 + LEFT JOIN + 填充:** 如前述案例 11 所示, 先生成序列, 然后 LEFT JOIN 原始数据, 再对 JOIN 结果中的 NULL 值进行填充 (可能再次用到 LAG/LEAD/递归 CTE)。
- **应用案例:** 你有一系列按时间记录的传感器读数, 某些时间点的数据缺失 (NULL)。你需要用最近的前一个有效的 (非 NULL) 读数来填充这些缺失值。
- **涉及表格:** 我们创建一个新的简单表 SensorReadings 来演示。

首先, 创建并填充一个包含按时间顺序排列且带 NULL 值的示例表:

SQL

```
-- 删除旧的传感器读数演示表 (如果存在)
```

```
DROP TABLE IF EXISTS SensorReadings;
```

```
-- 创建传感器读数表
```

```
CREATE TABLE SensorReadings (
    reading_time TIMESTAMP PRIMARY KEY, -- 读数时间, 主键
    value DECIMAL(10, 2)               -- 读数值, 可能为 NULL
);
```

```
-- 插入示例数据 (包含缺失值)
```

```
INSERT INTO SensorReadings (reading_time, value) VALUES
('2023-01-01 10:00:00', 10.5),
('2023-01-01 10:01:00', 12.1),
('2023-01-01 10:02:00', NULL), -- 缺失
('2023-01-01 10:03:00', 15.0),
('2023-01-01 10:04:00', NULL), -- 缺失
('2023-01-01 10:05:00', NULL), -- 连续缺失
('2023-01-01 10:06:00', 18.5),
('2023-01-01 10:07:00', NULL), -- 缺失
('2023-01-01 10:08:00', 20.2);
```

查询代码 22.1: 使用递归 CTE 填充缺失值 (向前传播上一个非 NULL 值)

SQL

```
WITH RECURSIVE FilledReadings AS ( -- 定义一个递归 CTE
    -- 递归的起始成员 (Anchor Member): 选择第一个非 NULL 的读数或序列中的第一行
    SELECT
        reading_time, -- 读数时间
        value AS imputed_value, -- 将原始值作为初始填充值
        1 AS rn -- 辅助列, 标识序列号
```

```

FROM
    SensorReadings -- 从原始表查询
ORDER BY
    reading_time ASC -- 按时间排序
LIMIT 1 -- 取第一行作为起始

UNION ALL -- 连接起始成员与递归成员

-- 递归成员 (Recursive Member): 处理后续行
SELECT
    sr.reading_time, -- 当前行的读数时间
    -- 如果当前行的值是 NULL，则使用前一次递归结果 (前一行) 的填充值；否则使用当前行的原始值
    COALESCE(sr.value, fr.imputed_value) AS imputed_value,
    fr.rn + 1 AS rn -- 序列号递增
FROM
    SensorReadings sr -- 从原始表查询当前行
INNER JOIN
    FilledReadings fr ON sr.reading_time = fr.reading_time + INTERVAL '1 minute' -- 连接到前一次递归的结果 (前一行)，条件是时间连续 (这里假设间隔是 1 分钟)
    -- WHERE -- 递归终止条件隐式包含在 JOIN 和没有匹配时递归停止中，也可以加一个显式终止条件
) -- CTE 定义结束

-- 主查询：选择填充后的结果
SELECT
    reading_time, -- 读数时间
    imputed_value -- 填充后的值
FROM
    FilledReadings -- 从递归 CTE 查询
ORDER BY
    reading_time ASC; -- 按时间排序结果

```

- **代码解释:**

- **WITH RECURSIVE FilledReadings AS (...): 用途:** 定义递归 CTE。这种结构的递归部分可以引用自身的结果 (fr) 来计算当前行 (sr) 的值，非常适合需要依赖前一行计算的场景（如填充缺失值）。
- **起始成员:** 选取按时间排序后的第一行作为递归的起点。它的 imputed_value 就是它自己的原始 value。
- **递归成员:** 关键在于 INNER JOIN SensorReadings sr ... ON sr.reading_time = fr.reading_time + INTERVAL '1 minute'。这表示在递归的每一步，都尝试找到原始表中时间比前一次递归结果(fr)晚 1 分钟的行(sr)。
- **COALESCE(sr.value, fr.imputed_value) AS imputed_value: 用途:** 如果当前行 (sr) 的原始 value 是 NULL，则使用前一次递归结果(fr)中的 imputed_value（即前一个非 NULL 值或已填充的值）来作为当前行的 imputed_value。如

果当前行的原始 value 不是 NULL，则直接使用它自己的原始值。

- 递归会沿着时间序列进行，每次处理一个新时间点，直到范围内没有更多连续的记录。
- **注意:** 这种递归 CTE 填充方法假设数据是按固定间隔 (INTERVAL '1 minute') 连续的。如果间隔不固定，或者需要跳过大的时间间隔，需要更复杂的 JOIN 条件或预处理步骤（如生成完整的日期序列）。

- **预期输出 22.1:** (填充后的传感器读数)

- reading_time | imputed_value

- -----|-----

- 2023-01-01 10:00:00 | 10.50

- 2023-01-01 10:01:00 | 12.10

- 2023-01-01 10:02:00 | 12.10 -- 填充前一分钟的值

- 2023-01-01 10:03:00 | 15.00

- 2023-01-01 10:04:00 | 15.00 -- 填充前一分钟的值

- 2023-01-01 10:05:00 | 15.00 -- 填充前一分钟的值

- 2023-01-01 10:06:00 | 18.50

- 2023-01-01 10:07:00 | 18.50 -- 填充前一分钟的值

- 2023-01-01 10:08:00 | 20.20

(注意：如果数据库支持 `LAST_VALUE(value IGNORE NULLS) OVER (ORDER BY reading_time)`，则可以更简洁地实现向前填充。)

- **填充缺失值总结:** 填充缺失值是数据预处理的重要环节。对于简单的向前/向后填充，递归 CTE 或特定窗口函数（如 `LAST_VALUE IGNORE NULLS`）是强大的工具。处理非固定间隔的填充或生成完整序列填充缺失日期是更复杂的场景。

23. 处理层级结构数据 (Handling Hierarchical Data)

- **目的:** 查询或分析具有父子关系的层级数据，例如公司组织架构（员工及其上下级）、物料清单（产品及其组件）、网络拓扑等。
- **常用技术: 递归 CTE (Recursive CTE)** 是处理层级数据最常用和强大的标准 SQL 方法。通过定义一个 CTE，其中一部分引用自身，可以沿着父子关系（向上或向下）遍历层级结构。
- **案例:** 给定一个员工表，其中包含员工 ID 和其经理 ID。找出向特定经理（包括该经理自己）汇报的所有员工（包括直接和间接下属）。
- **涉及表格:** 我们创建一个新的简单表 `Employees` 来演示。

首先，创建并填充一个包含经理 ID 的员工表示例：

SQL

-- 删除旧的员工演示表（如果存在）

DROP TABLE IF EXISTS Employees;

-- 创建员工表 (包含经理 ID)

CREATE TABLE Employees (
 emp_id INT PRIMARY KEY, -- 员工 ID

奕道远信息

```

emp_name VARCHAR(50), -- 员工姓名
manager_id INT NULL,   -- 经理 ID (外键, 引用 Employees.emp_id)。CEO 的
manager_id 为 NULL。
-- FOREIGN KEY (manager_id) REFERENCES Employees(emp_id) -- 实际应用中通常有自
引用外键
);

-- 插入示例层级数据
INSERT INTO Employees (emp_id, emp_name, manager_id) VALUES
(1, 'CEO', NULL),      -- CEO 没有经理
(2, 'Manager A', 1),   -- Manager A 直接向 CEO 汇报
(3, 'Emp A1', 2),      -- Emp A1 直接向 Manager A 汇报
(4, 'Emp A2', 2),      -- Emp A2 直接向 Manager A 汇报
(5, 'Manager B', 1),   -- Manager B 直接向 CEO 汇报
(6, 'Emp B1', 5),      -- Emp B1 直接向 Manager B 汇报
(7, 'Emp B1a', 6),     -- Emp B1a 直接向 Emp B1 汇报 (多层次)
(8, 'Emp C1', 5);      -- Emp C1 直接向 Manager B 汇报

```

查询代码 23.1: 使用递归 CTE 查询所有下属

SQL

```

-- 查找向 Manager B (emp_id = 5) 汇报的所有员工 (包括 Manager B 自己)
WITH RECURSIVE SubordinateHierarchy AS ( -- 定义一个递归 CTE
    -- 递归的起始成员 (Anchor Member): 从指定的经理开始
    SELECT
        emp_id,      -- 员工 ID
        emp_name,    -- 员工姓名
        manager_id,  -- 经理 ID
        0 AS level   -- 层级, 起始经理是第 0 层
    FROM
        Employees -- 从员工表查询
    WHERE
        emp_id = 5 -- 指定要查询的起始经理 ID (这里是 Manager B)

    UNION ALL -- 连接起始成员与递归成员

    -- 递归成员 (Recursive Member): 查找直接向 CTE 中员工汇报的员工
    SELECT
        e.emp_id,    -- 下属员工 ID
        e.emp_name,  -- 下属员工姓名
        e.manager_id, -- 下属员工的经理 ID
        sh.level + 1 AS level -- 层级加 1
    FROM
        Employees e -- 从员工表查询所有员工

```

INNER JOIN

SubordinateHierarchy sh ON e.manager_id = sh.emp_id -- 连接到前一次递归的结果
(SubordinateHierarchy), 条件是当前员工的经理 ID 等于 前一次结果中的员工 ID
) -- CTE 定义结束

-- 主查询: 从递归 CTE 中选择所有结果

SELECT

emp_id, -- 员工 ID
emp_name, -- 员工姓名
manager_id, -- 经理 ID
level -- 层级

FROM

SubordinateHierarchy -- 从递归 CTE 查询

ORDER BY

level ASC, -- 按层级排序
emp_id ASC; -- 按员工 ID 排序

- **代码解释:**

- WITH RECURSIVE SubordinateHierarchy AS (...): **用途:** 定义一个递归 CTE, 用于遍历层级结构。
- **起始成员:** SELECT ... FROM Employees WHERE emp_id = 5: 非递归部分。它选择层级遍历的起点, 即指定的经理 (emp_id = 5)。这是递归的“根”。
- UNION ALL: 连接起始成员和递归成员。
- **递归成员:** SELECT e.emp_id, ... FROM Employees e INNER JOIN SubordinateHierarchy sh ON e.manager_id = sh.emp_id: 这是核心递归逻辑。它从 Employees 表 (e) 中查找所有员工, 然后将它们与前一次递归的结果集 SubordinateHierarchy (sh) 进行连接。连接条件 e.manager_id = sh.emp_id 意味着它查找的是**直接向** sh 中包含的员工 (前一次递归找到的员工) **汇报**的员工 (e)。每次递归都找到下一层级的直接下属。sh.level + 1 增加层级计数。
- 递归会不断进行, 找到所有直接下属, 然后找到这些下属的下属, 以此类推, 直到某个员工没有下属为止。
- SELECT ... FROM SubordinateHierarchy: 主查询从递归 CTE 中选择所有结果。

- **预期输出 23.1:** (Manager B (ID 5) 的所有下属, 包括 Manager B 自己)

- emp_id | emp_name | manager_id | level

- -----|-----|-----|-----

- 5 | Manager B | 1 | 0 -- 起始经理

- 6 | Emp B1 | 5 | 1 -- Manager B 的直接下属

- 8 | Emp C1 | 5 | 1 -- Manager B 的另一个直接下属

- 7 | Emp B1a | 6 | 2 -- Emp B1 的直接下属 (Manager B 的间接下属)

(如果将 WHERE emp_id = 1 (CEO) 作为起始, 将返回整个公司层级结构。)

- **处理层级结构总结:** 递归 CTE 是处理层级数据的标准且强大的方法。它通过定义起始点和递归规则 (父子关系) 来遍历整个层级结构, 常用于查询组织结构、物料清单、网络路径等。

24. 比较相关行的数据 (Comparing Related Rows)

- **目的:** 比较表中相互关联但位于不同行的记录的某个属性。这种关联通常是基于同一个实体 (如同一个客户的不同订单, 同一个产品的不同版本) 按某个顺序 (如时间、版本号) 排列的。
- **常用技术:**
 - **窗口函数** LAG(列名, offset, default) 和 LEAD(列名, offset, default): 用于获取窗口内当前行上方 (LAG) 或下方 (LEAD) 指定偏移量的行的值。这非常适合比较序列中相邻的行。
 - **自连接 (Self-Join):** 将表与自身的别名连接起来。通过在连接条件中指定关联键相同 (如同一个客户 ID) 但其他条件不同 (如订单 ID 不同, 版本号不同) 来比较相关的行。找出“紧邻前/后”的行用自连接实现比 LAG/LEAD 复杂, 通常需要额外的排序和过滤步骤。
- **应用案例:** 分析客户的购买习惯。对于每个订单, 比较它的金额与该客户的前一笔订单金额以及第一笔订单金额。
- **涉及表格:** orders 表 (包含 order_id, customer_id, order_date, amount)。我们将使用窗口函数 LAG() 和 FIRST_VALUE() 来实现, 这是比较序列中相关行的常用且高效的方法。

首先, 请确保您已经执行了我们之前包含多笔订单数据的 orders 表的建表和数据插入代码。

查询代码 24.1: 使用 LAG() 和 FIRST_VALUE() 比较订单金额

SQL

SELECT

```
order_id, -- 当前订单 ID
customer_id, -- 客户 ID
order_date, -- 订单日期
amount AS CurrentOrderAmount, -- 当前订单金额
```

```
-- 获取前一笔订单的金额
```

```
LAG(amount, 1, 0) OVER ( -- LAG 用法: 获取按日期排序的前一行 amount 值
```

```
    PARTITION BY customer_id -- 按客户 ID 分区
```

```
    ORDER BY order_date ASC -- 按订单日期升序排序 (决定“前一行”是谁)
```

```
) AS PreviousOrderAmount, -- 前一笔订单金额 (如果当前是第一笔订单, 则为 0)
```

```
-- 获取第一笔订单的金额
```

```
FIRST_VALUE(amount) OVER ( -- FIRST_VALUE 用法: 获取分区内第一行 (按日期排序) 的
amount 值
```

```
    PARTITION BY customer_id -- 按客户 ID 分区
```

```
    ORDER BY order_date ASC -- 按订单日期升序排序 (决定“第一行”是谁)
```

```
) AS FirstOrderAmount, -- 第一笔订单金额 (对于该客户的所有订单, 这个值都相同)
```

```
-- 计算当前订单与前一笔订单的金额差
```

```
amount - LAG(amount, 1, 0) OVER (
```

奕道远信息

```

PARTITION BY customer_id
ORDER BY order_date ASC
) AS DiffFromPrevious, -- 当前订单金额 - 前一笔订单金额

-- 计算当前订单与第一笔订单的金额差
amount - FIRST_VALUE(amount) OVER (
    PARTITION BY customer_id
    ORDER BY order_date ASC
) AS DiffFromFirst -- 当前订单金额 - 第一笔订单金额

FROM
    orders -- 从 orders 表查询
ORDER BY -- 最后排序结果
    customer_id ASC,
    order_date ASC;

```

- **代码解释:**
 - LAG(amount, 1, 0) OVER (PARTITION BY customer_id ORDER BY order_date ASC): **用途:** 在每个客户分区内, 按订单日期升序排序后, 获取当前行 (当前订单) 的上一行 (前一笔订单) 的 amount 值。1 是偏移量 (上一行), 0 是默认值 (如果不存在前一行, 即这是第一笔订单)。
 - FIRST_VALUE(amount) OVER (PARTITION BY customer_id ORDER BY order_date ASC): **用途:** 在每个客户分区内, 按订单日期升序排序后, 获取该分区内第一行的 amount 值, 即该客户第一笔订单的金额。
 - PARTITION BY customer_id ORDER BY order_date ASC: **用途:** 定义窗口分区 (按客户) 和分区内排序 (按日期), 这决定了 LAG 和 FIRST_VALUE 操作的对象和顺序。
 - amount - LAG(...) AS DiffFromPrevious: 使用算术运算符 - 计算当前订单金额与前一笔订单金额的差值。
 - amount - FIRST_VALUE(...) AS DiffFromFirst: 计算当前订单金额与第一笔订单金额的差值。
 - ORDER BY customer_id ASC, order_date ASC: 最后对整个结果集排序, 便于查看。
- **自连接实现思路 (简述):**
 - 要使用自连接比较当前订单与**前一笔**订单, 需要先找到每个订单的前一笔订单日期。这通常需要先连接表到自身, 找到所有日期小于当前订单日期的订单, 然后对这些“前”订单按日期降序排序, 取第一个。

SQL

```

-- 查找每个订单的前一笔订单金额 (使用自连接的一种复杂方式)
SELECT
    o1.order_id, o1.order_date, o1.amount,
    ( -- 子查询找到前一笔订单金额
        SELECT o2.amount
        FROM orders o2
        WHERE o2.customer_id = o1.customer_id -- 同一客户
        ORDER BY o2.order_date DESC
        LIMIT 1
    ) AS prev_amount
FROM orders o1

```

```

AND o2.order_date < o1.order_date -- 日期早于当前订单
ORDER BY o2.order_date DESC -- 按日期倒序，最接近当前日期的排最前
LIMIT 1 -- 只取一个 (最近的一个)
) AS PreviousOrderAmount_SelfJoin
FROM orders o1
ORDER BY o1.customer_id, o1.order_date;
-- 这种方法通常不如 LAG 简洁高效，但能实现类似目的。
    ○ 自连接更适用于比较任意相关行，不局限于“紧邻”关系，例如比较同一产品在不同仓库的库存（Join on product_id, warehouse_id != other_warehouse_id）。

```

- **预期输出 24.1:** (显示所有订单，并附加比较结果)
- order_id | customer_id | order_date | CurrentOrderAmount | PreviousOrderAmount | FirstOrderAmount | DiffFromPrevious | DiffFromFirst
- -----|-----|-----|-----|-----|-----|-----|-----
- -----|-----|-----|-----|-----|-----|-----|-----
- 101 | 1 | 2023-01-15 | 150.00 | 0.00 | 150.00 | 150.00 | 0.00 -- Alice 第 1 单
- 103 | 1 | 2023-01-17 | 50.00 | 150.00 | 150.00 | -100.00 | -100.00 -- Alice 第 2 单
- 106 | 1 | 2023-06-01 | 450.75 | 50.00 | 150.00 | 400.75 | 300.75 -- Alice 第 3 单
- 111 | 1 | 2023-07-30 | 600.00 | 450.75 | 150.00 | 149.25 | 450.00 -- Alice 第 4 单
- ... | ... | ... | ... | ... | ... | ... | ...
- 116 | 9 | 2024-01-15 | 50.00 | 0.00 | 50.00 | 50.00 | 0.00 -- Alice (LA) 第 1 单
- **比较相关行总结:** 窗口函数 LAG() 和 LEAD() 是比较序列中相邻行属性值的最直接和高效方法。自连接也是一种强大的比较工具，尤其适用于比较非序列性的、基于任意条件的关联行。

25. 比较个体与群体规范 (Comparing Item to Group Norm)

- **目的:** 比较数据集中的单条记录（个体）的某个值与其所属分组（群体）的汇总统计量（如平均值、最大值、最小值）。这用于评估个体在群体中的表现或定位。
- **常用技术:**
 - **聚合窗口函数:** 使用 AVG() OVER(...), MIN() OVER(...), MAX() OVER(...) 等函数，结合 PARTITION BY 按分组列进行分区。这样可以在不折叠原始行的情况下，将每个分组的汇总值计算并显示在分组内的每一行旁边。
 - **子查询 或 CTE 结合连接:** 先计算每个分组的汇总值（在子查询/CTE 中进行 GROUP BY），然后在主查询中将这个汇总结果与原始明细数据连接起来进行比较。
- **应用案例:** 分析产品定价。对于每个产品，比较它的价格与它所属类别的平均价格、

最低价格和最高价格。

- **涉及表格:** products 表 (包含 product_name, category, price)。我们将使用聚合窗口函数来实现。

首先, 请确保您已经执行了我们之前包含所有产品数据的 products 表的建表和数据插入代码。

查询代码 25.1: 使用聚合窗口函数比较产品价格与类别均价/最高价/最低价

SQL

SELECT

```
product_id, -- 产品 ID
product_name, -- 产品名称
category, -- 产品类别
price AS CurrentPrice, -- 当前产品价格
```

```
-- 计算类别平均价格
```

```
AVG(price) OVER ( -- AVG() OVER(): 将 AVG 作为窗口函数使用
```

```
    PARTITION BY category -- PARTITION BY 用法: 按产品类别分区, 平均值在每个类别内独立计算
```

```
) AS CategoryAveragePrice, -- 每个产品类别内的平均价格 (显示在类别内的所有行上)
```

```
-- 计算类别最低价格
```

```
MIN(price) OVER ( -- MIN() OVER(): 将 MIN 作为窗口函数使用
```

```
    PARTITION BY category -- 按产品类别分区
```

```
) AS CategoryMinimumPrice, -- 每个产品类别内的最低价格 (显示在类别内的所有行上)
```

```
-- 计算类别最高价格
```

```
MAX(price) OVER ( -- MAX() OVER(): 将 MAX 作为窗口函数使用
```

```
    PARTITION BY category -- 按产品类别分区
```

```
) AS CategoryMaximumPrice, -- 每个产品类别内的最高价格 (显示在类别内的所有行上)
```

```
-- 计算当前价格与类别平均价格的差值
```

```
price - AVG(price) OVER (PARTITION BY category) AS DiffFromCategoryAvg,
```

```
-- 计算当前价格占类别最高价格的比例 (需要处理类别最高价为 0 的情况, 虽然本例价格都大于 0)
```

```
CASE WHEN MAX(price) OVER (PARTITION BY category) = 0 THEN NULL
```

```
    ELSE price / MAX(price) OVER (PARTITION BY category) * 100
```

```
END AS PctOfCategoryMax
```

FROM

```
products -- 从 products 表查询
```

ORDER BY -- 最后排序结果

```
category ASC,
```

```
price DESC; -- 按类别排序, 类别内按价格降序
```

奕道远信息

- **代码解释:**

- AVG(price) OVER (PARTITION BY category): **用途:** 将 AVG 函数用作窗口函数。PARTITION BY category 将产品按其类别分组。OVER (PARTITION BY category) 定义了一个窗口，该窗口包含与当前行属于同一类别的所有行。AVG(price) 在这个窗口内计算平均值。结果是每个类别内的平均价格，并且这个值会重复显示在该类别的所有产品行上。
- MIN(price) OVER (PARTITION BY category): **用途:** 将 MIN 函数用作窗口函数，计算每个类别的最低价格，并显示在类别内的所有行上。
- MAX(price) OVER (PARTITION BY category): **用途:** 将 MAX 函数用作窗口函数，计算每个类别的最高价格，并显示在类别内的所有行上。
- 注意：这些聚合窗口函数 (AVG, MIN, MAX) 的 OVER() 子句中**没有** ORDER BY。在这种情况下，窗口框架默认是整个分区 (ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)，聚合函数计算的是整个分区（整个类别）的汇总值。如果加入了 ORDER BY，它们会计算**累计**平均值/最大值/最小值（默认窗口是分区开始到当前行）。
- price - AVG(...) AS DiffFromCategoryAvg: 计算当前产品价格与该类别平均价格的差值。
- price / MAX(...) * 100 AS PctOfCategoryMax: 计算当前产品价格占该类别最高价格的百分比。使用了 CASE 处理分母为 0 的情况。
- ORDER BY category ASC, price DESC: 最后对结果集排序，方便查看。

- **子查询实现思路 (简述):**

- 另一种方法是先计算每个类别的平均价格（使用 GROUP BY category 的子查询），然后将这个子查询的结果与原始 products 表通过 category 进行连接，再进行比较计算。

SQL

-- 计算类别平均价格 (使用子查询和 JOIN)

SELECT

p.product_id, p.product_name, p.category, p.price, ca.CategoryAveragePrice,
p.price - ca.CategoryAveragePrice AS DiffFromCategoryAvg_Subquery

FROM

products p

INNER JOIN (-- 子查询计算类别平均价

SELECT category, AVG(price) AS CategoryAveragePrice

FROM products

GROUP BY category

) AS ca ON p.category = ca.category; -- 将原始产品表与类别平均价结果连接

- 使用窗口函数通常更简洁，尤其当需要同时比较多个汇总值（如平均、最小、最大）时。

- **预期输出 25.1:** (显示所有产品，并附加与类别规范的比较结果)

- product_id | product_name | category | CurrentPrice | CategoryAveragePrice | CategoryMinimumPrice | CategoryMaximumPrice | DiffFromCategoryAvg | PctOfCategoryMax

- -----|-----|-----|-----|-----|-----|-----|-----|-----|

变道远信息

```

-----|-----|-----|-----|-----|-----|-----|-----|
-----|
• Books      | Python Book | Books      | 60.00      | 40.000000      | 15.00
  | 60.00      | 20.000000      | 100.000000
• Books      | SQL Book    | Books      | 45.00      | 40.000000      | 15.00      |
  | 60.00      | 5.000000      | 75.000000
• Books      | Novel       | Books      | 15.00      | 40.000000      | 15.00      |
  | 60.00      | -25.000000     | 25.000000
• Electronics| Laptop      | Electronics | 800.00     | 274.999000     | 25.00
  | 800.00     | 525.001000     | 100.000000
• Electronics| Monitor     | Electronics | 300.00     | 274.999000     | 25.00
  | 800.00     | 25.001000     | 37.500125
• ...        | ...         | ...         | ...        | ...            | ...        | ...
  | ...
• Misc       | Mystery Item | Misc       | 999.99     | 497.495000     | -5.00      |
  | 999.99     | 502.495000     | 100.000000
• Misc       | Defective Item | Misc      | -5.00      | 497.495000     | -5.00      |
  | 999.99     | -502.495000    | -0.500005

```

(注：类别平均价是基于我们更新后的包含负价格产品的 *products* 表计算的。)

- **比较个体与群体规范总结：**利用聚合窗口函数配合 PARTITION BY 是在不折叠行的情况下，将分组汇总值附加到每行并进行比较的强大模式。它比子查询/JOIN 方法在需要比较多个汇总指标时更简洁。

